

IDentify: Integration von Canvas Fingerprinting für Identity Management

Untersuchung verschiedener Machine Learning Ansätze für das Canvas
Fingerprinting Projekt IDentify

Lion Witte
Matrikelnummer: 675382

Eine Masterarbeit in Kooperation mit KOBIL und Herrn Dr. Federooni
und Herrn Prof. Dr. Jens Kohler



Fachbereich Informatik
Hochschule Worms
Datum: 14. Februar 2025

Erstkorrektor: Prof. Dr. Jens Kohler
Zweitkorrektor: Prof. Dr. Elisabeth Heinemann

INHALTSVERZEICHNIS

I	ABSTRACT	3	V-D4	Trainingsmodul (train_model.py[10])	15
II	Einleitung	3	V-D5	Technologien	15
II-A	Browser Fingerprinting	4	V-E	Experiment Setup der Machine- Learning-Komponenten	16
II-B	Canvas Fingerprinting	4	V-E1	Architektur	16
II-C	Problemstellung	5	V-E2	Datenfluss und Trainingspro- zess	16
II-D	Zielsetzung	5	V-E3	Technologien	16
II-E	Forschungsfrage	6	V-E4	Unabhängigkeit und Flexibi- lität	16
II-F	Random Forest und Isolation Forest . .	6	V-E5	Hardware und Leistung . . .	17
II-G	Autoencoder	6	V-E6	Hardware-Spezifikationen . .	17
II-H	Convolutional Neural Networks	6	V-E7	Einsatzbereiche der Hardware	17
II-I	Komplexere Ansätze	7	V-F	Ansätze	17
II-J	Relevanz	7	VI	Ansatz I	18
II-K	Aufbau der Arbeit	7	VI-A	Bisheriger Ansatz	18
III	Stand der Technik	8	VI-B	Modelle	18
III-A	Kommerzielle Nutzung und FingerprintJS-Bibliothek	8	VI-B1	Model 1	18
III-B	Forschungsergebnisse zu Canvas Fin- gerprinting	8	VI-B2	Model 2	18
III-C	Kritik und Herausforderungen	9	VI-B3	Model 3	19
III-C1	Technische Herausforderun- gen und Gegenmaßnahmen . .	9	VI-B4	Model 4	19
III-D	Related Work	9	VI-B5	Model 5	19
IV	Methodik	10	VI-B6	Model 6	20
IV-A	Code-Backbone: Das technologische Fundament	10	VI-C	Mehrheitsabstimmung	20
IV-B	Implementierung des Canvas Fingerprints	10	VI-D	Implementierung der Mehrheitsabstim- mung	20
IV-C	Ziele der Datensammlung	10	VI-E	Ergebnisse des bisherigen Ansatzes . .	21
IV-D	Geplante Modelle und Architekturen . .	10	VI-F	Begriffe und Konzepte	21
IV-D1	Isolation Forest Classifier . .	10	VI-F1	Username	21
IV-D2	Convolutional Neural Net- works (CNNs)	11	VI-F2	Fingerabdruck	21
IV-D3	Autoencoder	11	VI-F3	Canvas	21
IV-D4	Siamese Neural Networks (SNNs)	11	VI-G	Leistung der Modelle	21
IV-D5	Ensemble-Learning-Techniken	11	VI-G1	Metriken zur Modellbewertung	21
IV-E	Implementierungsstrategie	11	VI-H	Testdurchlauf der Modelle 1	21
IV-F	Zusammenfassung	11	VI-H1	Graphen zur Modellleistung .	22
V	Implementierung	12	VI-H2	Graphen zur Unterschei- dungsfähigkeit der Modelle .	23
V-A	Systemarchitektur	12	VI-H3	Zusammenfassung der Er- gebnisse 1	24
V-B	Containerisierte Systemarchitektur . . .	12	VI-I	Testdurchlauf der Modelle 2	24
V-C	Systemkomponenten	14	VI-I1	Ergebnisse	24
V-C1	Frontend	14	VI-I2	Grafische Darstellung der Ergebnisse	26
V-C2	Logik des Frontends	14	VI-I3	Zusammenfassung der Er- gebnisse 2	26
V-C3	Backend	14	VI-J	Gesamtbewertung	26
V-D	Zusammenarbeit mit anderen Kompo- nenten	15	VII	Ansatz II	27
V-D1	Backend-Logik	15	VII-1	Ziele und Herangehensweisen	27
V-D2	Machine-Learning- Komponenten	15	VII-2	Einleitung Modelle und Eva- luierungsmethoden	27
V-D3	ML-Server (ml_server.py[10])	15	VII-3	Zusammenfassung der Me- thodik	27
			VII-A	Beschreibung der Daten und Ergebnisse	27

	VII-A1	Einführung	27	VII-G3	Ergebnisse	38
	VII-A2	Datenbankverwaltung und Docker-Setup	27	VII-G4	Grafische Darstellung der Ergebnisse des CNN-Modells	38
	VII-A3	Beispiele für Canvas Finger- prints	28	VII-G5	Optimierungsversuche und finale Modellarchitektur . . .	45
VII-B		Verwendete Geräte	28	VII-G6	Diskussion der finalen Mo- dellarchitektur	46
	VII-B1	Verteilung der Geräte, Be- triebssysteme, Hersteller und Browser	29	VII-G7	Zusammenfassung	46
	VII-B2	1. Geräteverteilung	29	VII-G8	Isolation Forest	46
	VII-B3	2. Betriebssysteme	30	VII-G9	Autoencoder	46
	VII-B4	3. Herstellerverteilung	30	VII-G10	Convolutional Neural Net- works (CNNs)	47
	VII-B5	4. Browserverteilung	30	VII-H	Klassifikationsproblematik	47
	VII-B6	Statistische Analyse	30	VII-I	Diskussion der Ergebnisse	47
	VII-B7	Ähnlichkeitsanalyse mit SSIM	31	VII-J	Zusammenfassung	47
VII-C		Datenstruktur	32			
VII-D		Isolation Forest	33	VIII Diskussion der Ergebnisse		48
	VII-D1	Isolation Forest: Erste Mo- dellimplementierung	33	VIII-A	Ansatz I: Begrenzte Datenmenge	48
	VII-D2	Zusammenfassung des Codes	33	VIII-B	Ansatz II: Erweiterte Datenbasis	48
	VII-D3	Ergebnisse des Modells . . .	33	VIII-C	Zentrale Herausforderungen	48
	VII-D4	Grafische Darstellung der Ergebnisse	33	IX Fazit und Ausblick		49
	VII-D5	Isolation Forest: Zweite Mo- dellimplementierung	34	IX-A	Ausblick	49
	VII-D6	Zusammenfassung des Codes	34	X Danksagung		49
	VII-D7	Ergebnisse des Modells . . .	34	X-A	Zusätzliche Informationen	49
	VII-D8	Grafische Darstellung der Ergebnisse	34			
					ABBILDUNGSVERZEICHNIS	
VII-E		Autoencoder: Modellimplementierung und Evaluierung	35	1	Systemübersicht: Verarbeitung von Canvas- Fingerabdrücken mit ML-Modellen.	5
	VII-E1	Was ist ein Autoencoder? . .	35	2	Framework des Projekts IDentify	12
	VII-E2	Zusammenfassung des Codes	35	3	Framework des Projekts IDentify	13
	VII-E3	Ergebnisse des Modells . . .	35	4	Experiment-Setup des Projekts IDentify	13
	VII-E4	Grafische Darstellung der Ergebnisse	35	5	Vereinfachte Übersicht über die Hauptlogik des Frontends	14
VII-F		Autoencoder: Zweite Modellimplemen- tierung	36	6	Übersicht über die Hauptinteraktionen im Backend	15
	VII-F1	Unterschiede zur ersten Ver- sion	36	7	Übersicht über die ML-Komponenten und ihre Interaktionen	16
	VII-F2	Ergebnisse des Modells . . .	36	8	Datenfluss und Komponenten des ML- Trainingsmoduls	17
	VII-F3	Grafische Darstellung der Ergebnisse für das zweite Modell	36	9	Einsatzbereiche der Hardware-Systeme	17
	VII-F4	CNN Autoencoder	36	10	Architektur von Model 1	18
	VII-F5	Modellbewertung	37	11	Architektur von Model 2	18
	VII-F6	Grafische Darstellung der Ergebnisse	37	12	Architektur von Model 3	19
	VII-F7	Zusammenfassung der Er- gebnisse	37	13	Architektur von Model 4	19
				14	Architektur von Model 5	20
				15	Architektur von Model 6	20
				16	Illustrationen von Fingerabdruck-Canvases	21
VII-G		Übersicht der CNN-Modelle	37	17	Leistung der sechs Modelle von <i>Benutzer 1</i> gegen Fingerabdrücke von <i>Benutzer 1</i>	22
	VII-G1	Zusammenfassung der Ent- wicklungsphasen und Mo- delle	38	18	Leistung der sechs Modelle von <i>Benutzer 4</i> gegen Fingerabdrücke von <i>Benutzer 4</i>	22
	VII-G2	CNNV7: Evaluierung und Optimierung	38	19	Leistung der sechs Modelle von <i>Benutzer 4</i> gegen zufällige Fingerabdrücke von <i>Benutzer 1</i> und <i>Benutzer 4</i>	23

20	Leistung der sechs Modelle von <i>Benutzer 1</i> gegen zufällige Fingerabdrücke von <i>Benutzer 1</i> und <i>Benutzer 4</i>	23	60	Performance 5 besten der Modelle	44
21	Konfusionsmatrix für Model 1	24	61	Performance der Top 10 Modelle	44
22	Konfusionsmatrix für Model 2	24	62	Zusammengeführte Liste des Precision Scores je Architektur	44
23	Konfusionsmatrix für Model 3	24	63	Zusammengeführte Liste des Precision Scores der 5 besten Modelle	44
24	Konfusionsmatrix für Model 4	24	64	Metriken Vergleich der besten Modelle je Architektur	44
25	Konfusionsmatrix für Model 5	25	65	Metriken Vergleich der besten 5 Modelle	45
26	Konfusionsmatrix für Model 6	25	66	Architektur des optimierten CNN-Modells	45
27	Vergleich der Metriken für die Modelle	26	67	CNN Konfusionsmatrix	46
28	Canvas Fingerprint Beispiel für Benutzer 1	28		<i>Zusammenfassung—</i>	
29	Canvas Fingerprint Beispiel für Benutzer 2	28			
30	Canvas Fingerprint Beispiel für Benutzer 3	28			
31	Geräteverteilung in der Stichprobe	29			
32	Betriebssystemverteilung in der Stichprobe	30			
33	Herstellerverteilung in der Stichprobe	30			
34	Browserverteilung in der Stichprobe	30			
35	Boxplot der Länge der Base64-codierten Strings nach Benutzern	31			
36	Heatmap der SSIM-Werte zwischen den Benutzern	32			
37	Datenstruktur des zweiten Ansatzes	33			
38	Konfusionsmatrix: Zeigt die Anzahl der True Positives, True Negatives, False Positives und False Negatives.	34			
39	Konfusionsmatrix: Zeigt die Anzahl der True Positives, True Negatives, False Positives und False Negatives.	34			
40	Konfusionsmatrix: Zeigt die Anzahl der True Positives, True Negatives, False Positives und False Negatives.	34			
41	Rekonstruktionsfehler für den Beispielnutzer (Train, Validation, Test)	35			
42	Rekonstruktionsfehler für Nutzer 1 und zufälligen Nutzer 3	35			
43	Rekonstruktionsfehler für alle Nutzer	35			
44	Rekonstruktionsfehler für den Beispielnutzer und andere Nutzer	36			
45	Rekonstruktionsfehler für Nutzer 1 und zufälligen Nutzer 3 – Zweites Modell	36			
46	Rekonstruktionsfehler für den Beispielnutzer (Train, Validation, Test)	37			
47	Rekonstruktionsfehler für Nutzer 1 und zufälligen Nutzer 3	37			
48	Rekonstruktionsfehler für alle Nutzer	37			
49	Bewertung verschiedener Nutzer	39			
50	Kreisdiagramm False Positives	39			
51	Metrische Tabelle	40			
52	Konfusionsmatrix	40			
53	Bewertung verschiedener Nutzer	41			
54	Kreisdiagramm False Positives	41			
55	Metrische Tabelle	42			
56	Konfusionsmatrix	42			
57	Gesamte Genauigkeit (Accuracy) des CNN-Modells	42			
58	Genauigkeit der besten 5 Modelle	43			
59	Genauigkeit der besten Modelle pro Architekturen	43			

I. ABSTRACT

In dieser Arbeit werden die Entwicklung und Evaluierung verschiedener Machine-Learning-Modelle zur Benutzererkennung basierend auf Fingerabdruckdaten vorgestellt. Ziel des Projekts ist es, präzise und robuste Modelle zu entwickeln, die Fingerabdrücke effektiv analysieren und Benutzer zuverlässig identifizieren können. Die Daten wurden aus einer MongoDB-Datenbank extrahiert, vorverarbeitet und für das Training verschiedener Modelle genutzt, darunter Convolutional Neural Networks (CNNs), Autoencoder und Isolation Forests.

Die Modelle wurden auf einem umfangreichen Datensatz trainiert, der insgesamt 2 Millionen Canvas-Fingerprints umfasst, wobei für jeden der 20 Benutzer 100.000 Samples gesammelt wurden. Zur Evaluierung der trainierten Modelle wurden Metriken wie Genauigkeit, Präzision, Recall und F1-Score herangezogen[1]. Zusätzlich wurden Konfusionsmatrizen und Visualisierungen erstellt, um die Ergebnisse zu analysieren[2].

Die Ergebnisse zeigen, dass verschiedene Machine-Learning-Ansätze, wie Convolutional Neural Networks (CNNs) vielversprechende Ergebnisse für die Benutzererkennung liefern. Diese Ansätze bieten eine solide Grundlage für zukünftige Forschungen und Weiterentwicklungen in diesem Bereich. Diese Arbeit trägt zur Weiterentwicklung von Authentifizierungsmethoden bei, indem sie Canvas Fingerprinting als zusätzlichen Sicherheitsmechanismus untersucht und dessen Potenzial zur Erweiterung bestehender Authentifizierungsverfahren aufzeigt.

II. EINLEITUNG

Im digitalen Zeitalter ist die sichere Authentifizierung von Benutzern von entscheidender Bedeutung. Traditionelle Methoden wie Passwörter und Sicherheitstoken weisen erhebliche Schwachstellen auf. Passwörter können durch Brute-Force-Angriffe[3] geknackt oder durch Phishing-Attacken gestohlen werden[4], während Sicherheitstoken verloren gehen oder entwendet[5][6] werden können. Diese Authentifizierungsmethoden können zudem oft benutzerunfreundlich[7][8] sein, da sie regelmäßige Eingaben oder das Mitführen zusätzlicher Hardware erfordern[9]. Das Projekt IDentify[10] adressiert diese Herausforderungen durch die Implementierung von Canvas Fingerprinting[11] als zusätzliche Sicherheitsebene. Diese Technologie ermöglicht eine transparente Überprüfung der Benutzeridentität, besonders in Situationen, in denen traditionelle Authentifizierungsmethoden kompromittiert wurden. Wenn sich ein Benutzer beispielsweise von einem unbekannten Gerät anmeldet, kann Canvas Fingerprinting als zusätzliche Authentifizierungsbarriere dienen, selbst wenn die primären Anmeldedaten korrekt sind. Diese Technologie könnte theoretisch auch eigenständig eingesetzt werden. Allerdings ist sie

für diesen Anwendungszweck derzeit noch nicht ausreichend ausgereift, da sie potenziell Fehlklassifikationen verursachen kann. Dies könnte, ähnlich wie bei einem schwachen Passwort, zu erhöhten Sicherheitsrisiken führen. Daher sollte diese Technologie zunächst als zusätzliche Sicherheitsebenen betrachtet und entsprechend genutzt werden. Um die folgenden Kapitel besser nachvollziehen zu können, werden zunächst die wichtigsten Begriffe dieser Arbeit erläutert.

A. Browser Fingerprinting

Browser Fingerprinting[12][13] ist eine Methode, mit der Benutzer im Web anhand der spezifischen Konfigurationen und Eigenschaften ihres Browsers identifiziert und verfolgt werden können. Diese Technik nutzt verschiedene Parameter wie installierte Plugins, Schriftarten, Bildschirmauflösung, Zeitzone, Betriebssystem und andere Gerätespezifikationen, um einen eindeutigen digitalen Fingerabdruck zu erstellen. Eine bekannte Bibliothek für diese Technik ist die FingerprintJS-Bibliothek[14], die sich auf die Erstellung von Fingerabdrücken fokussiert hat. Diese Technik basiert auf der Erfassung verschiedener Browser- und Systemeigenschaften, darunter auch Canvas Fingerprinting, um einen einzigartigen Fingerabdruck zu generieren. Dieser Fingerabdruck kann verwendet werden, um Benutzer wiederzuerkennen, auch wenn diese vorhandenen Cookies löschen oder den Inkognito-Modus verwenden.

Diese Technik bietet einige Vorteile, darunter die folgenden:

- 1) **Hohe Genauigkeit:** Die Kombination aus verschiedenen Parametern führt zu einem nahezu einzigartigen Fingerabdruck für jeden Benutzer
- 2) **Keine Abhängigkeit von Cookies:** Im Gegensatz zu traditionellen Tracking-Methoden wie Cookies ist Browser-Fingerprinting schwerer zu umgehen, da es ohne Zustimmung des Nutzers funktioniert und nicht direkt blockiert werden kann.¹
- 3) **Erkennung von Betrug:** Durch die Identifikation ungewöhnlicher Konfigurationen oder Verhaltensmuster kann Browser Fingerprinting helfen, betrügerische Aktivitäten zu erkennen und zu verhindern.

Neben diesen Vorteilen birgt Browser Fingerprinting auch potenzielle Risiken. Da diese Technik im Hintergrund arbeitet und keine direkte aktive Zustimmung des Nutzers erfordert[15], kann sie theoretisch dazu verwendet werden, Nutzer ohne deren Wissen oder Einverständnis zu überwachen. Dies wirft erhebliche Datenschutzbedenken[16] auf, insbesondere wenn Fingerprinting-Techniken mit anderen Datenquellen kombiniert werden, um detaillierte Nutzerprofile zu erstellen[17]. Gleichzeitig bietet diese Methode jedoch auch

¹Browser-Fingerprinting erfasst eine Vielzahl von Geräte und Browsermerkmalen, wie Bildschirmauflösung, installierte Plugins und Schriftarten, um einen digitalen Fingerabdruck zu erstellen. Diese Daten können weder durch das Löschen von Cookies noch durch den Einsatz von Tools wie VPNs entfernt werden. Verschiedene Browser-Plugins oder spezialisierte Browser wie der Tor-Browser minimieren die Einzigartigkeit des Fingerabdrucks, allerdings auf Kosten der Benutzerfreundlichkeit.

positive Anwendungen, wie beispielsweise die Erkennung von betrügerischem Verhalten. Durch die Analyse von Abweichungen in den Fingerabdrücken lassen sich Anomalien erkennen, die auf potenziellen Betrug hinweisen können, beispielsweise bei der Nutzung gestohlener Geräte oder manipulierten Softwareumgebungen.

In der vorliegenden Arbeit liegt der Fokus jedoch auf einer spezifischen Variante des Browser-Fingerprintings, dem Canvas Fingerprinting.

B. Canvas Fingerprinting

Canvas Fingerprinting ist eine spezielle Form des Browserfingerprintings, die das HTML5-Canvas-Element[18] nutzt, um einzigartige Fingerabdrücke zu erstellen. Diese Technik wurde erstmals von Mowery und Shacham [19] in ihrem Paper Pixel Perfect: Fingerprinting Canvas in HTML5 beschrieben, das diese Methode detailliert erklärt. Die Technik lässt sich wie folgt zusammenfassen:

- 1) **Zeichnen:** Ein Skript auf der Webseite zeichnet Text oder Grafiken auf das Canvas-Element².
- 2) **Extraktion:** Das Skript extrahiert die Pixel-Daten des gezeichneten Bildes und konvertiert sie in eine Base64-codierte Darstellung.
- 3) **Hashing:** Diese Darstellung wird dann durch eine Hash-Funktion geleitet, um einen eindeutigen Fingerabdruck zu erzeugen.

Die Unterschiede in der Art und Weise, wie verschiedene Geräte und Browser das Bild rendern, führen zu einzigartigen Fingerabdrücken[20]. Diese Unterschiede entstehen durch Faktoren wie die Grafikkarte, den Grafiktreiber, das Betriebssystem und die installierten Schriftarten³. Durch die Analyse der durch das Canvas-Element erzeugten Fingerabdrücke können Nutzer über mehrere Sitzungen hinweg wiedererkannt werden, solange keine Änderungen an der Hardware oder Software vorgenommen werden.

Diese Methode ist besonders effektiv, da sie auch im Inkognito-Modus oder bei deaktivierten Cookies funktioniert⁴.

Vorteile von Canvas Fingerprinting:

- **Hohe Präzision:** Die Methode erkennt selbst kleinste Unterschiede in der Hardware- und Softwarekonfiguration eines Geräts⁵.
- **Transparenz für den Benutzer:** Der gesamte Prozess läuft im Hintergrund ab und beeinträchtigt die Benutzererfahrung und die Performance des Browsers nicht⁶.

²Das Canvas-Element ist ein HTML5-Element, das es ermöglicht, dynamische, pixelbasierte Grafiken direkt im Browser zu erstellen und zu manipulieren.

³Das Canvas-Rendering variiert durch hardware- und softwarebedingte Faktoren wie Subpixel-Rendering, Anti-Aliasing und GPU-Optimierungen [21].

⁴Canvas Fingerprinting ermöglicht Tracking ohne lokale Datenspeicherung [22].

⁵GPU-Treiber und Rendering-Algorithmen erzeugen einzigartige Pixelmuster [23].

⁶Im Gegensatz zu Cookies hinterlässt Canvas Fingerprinting keine sichtbaren Spuren auf dem Gerät des Nutzers.

- **Persistenz:** Fingerabdrücke bleiben stabil über mehrere Browsersitzungen hinweg bestehen, selbst bei anonymem Surfen⁷
- **Vielfältige Anwendungen:** Neben der Nutzerverfolgung wird es zur Betrugserkennung, Personalisierung von Inhalten und Sicherheitsmaßnahmen eingesetzt⁸.

C. Problemstellung

Die Authentifizierung von Nutzern in Webanwendungen stellt eine zentrale Herausforderung in der IT-Sicherheit dar. Wie bereits in der Einleitung angesprochen, sind traditionelle Authentifizierungsmethoden wie Passwörter anfällig, für Brute-Force-Angriffe[3] und Phishing[4], während Sicherheitstoken gestohlen oder kompromittiert werden können[6][5]. Diese Arten von Sicherheitsmaßnahmen können die Benutzerfreundlichkeit beeinträchtigen, da Nutzer beispielsweise gezwungen sind, eine Vielzahl von Passwörtern zu verwalten [8]. Falls jedoch schwache Passwörter verwendet werden oder Authentifizierungsdaten gestohlen wurden, kann Canvas-Fingerprinting helfen, zusätzliche Sicherheitsmechanismen zu implementieren.

Canvas Fingerprinting bietet hier eine zusätzliche Sicherheitsebene, indem es subtile Unterschiede in der Darstellung von HTML5-Canvas-Elementen nutzt[20], welche bei jedem Besuch der Webseite mit dieser geteilt werden und so eine Wiedererkennung des Nutzers ermöglichen. Diese Technik ermöglicht es, verdächtige Anmeldeaktivitäten zu erkennen, wenn ein Angreifer versucht, sich mit gestohlenen Zugangsdaten von einem anderen Gerät aus einzuloggen[24]. Da die Identifizierung im Hintergrund erfolgt, sind keine zusätzlichen Aktionen vom Benutzer erforderlich, was die Benutzerfreundlichkeit erhöht.

Allerdings weist die derzeitige Technik von Canvas Fingerprinting Methoden noch Einschränkungen hinsichtlich der Genauigkeit, Skalierbarkeit und Robustheit auf. Des Weiteren besteht die Möglichkeit, dass sich der Fingerabdruck ändert, falls der Nutzer seine Systemkonfiguration durch Updates, wie etwa ein Betriebssystem- oder Grafiktreiber-Update, verändert. Solche Änderungen beeinflussen die Rendering-Mechanismen und können dazu führen, dass der Fingerabdruck nicht mehr mit dem ursprünglichen übereinstimmt, was die Wiedererkennung des Nutzers erschwert.

D. Zielsetzung

Um die beschriebenen Einschränkungen des Canvas Fingerprintings zu adressieren, wird im Rahmen dieser Arbeit ein System entwickelt, das Canvas-Fingerabdrücke von Nutzern durch zufällig generierte Strings erfasst und analysiert. Die gesammelten Daten werden in einer Datenbank gespeichert und dienen dem Training verschiedener Machine-Learning-Modelle. Ziel dieser Arbeit ist es, die bestehende Technologie weiterzuentwickeln, zu implementieren und umfassend

zu testen. Dabei soll untersucht werden, wie verschiedene Ansätze und Machine-Learning-Techniken die Leistung und Anwendbarkeit dieser Technologie beeinflussen. Durch diese Experimente soll die Technologie auf ihre Eignung für Endanwendungen bewertet und optimiert werden. Die praktische Implementierung erfolgt im Rahmen des Projekts IDentify[10], dessen Quellcode öffentlich auf GitHub verfügbar ist.

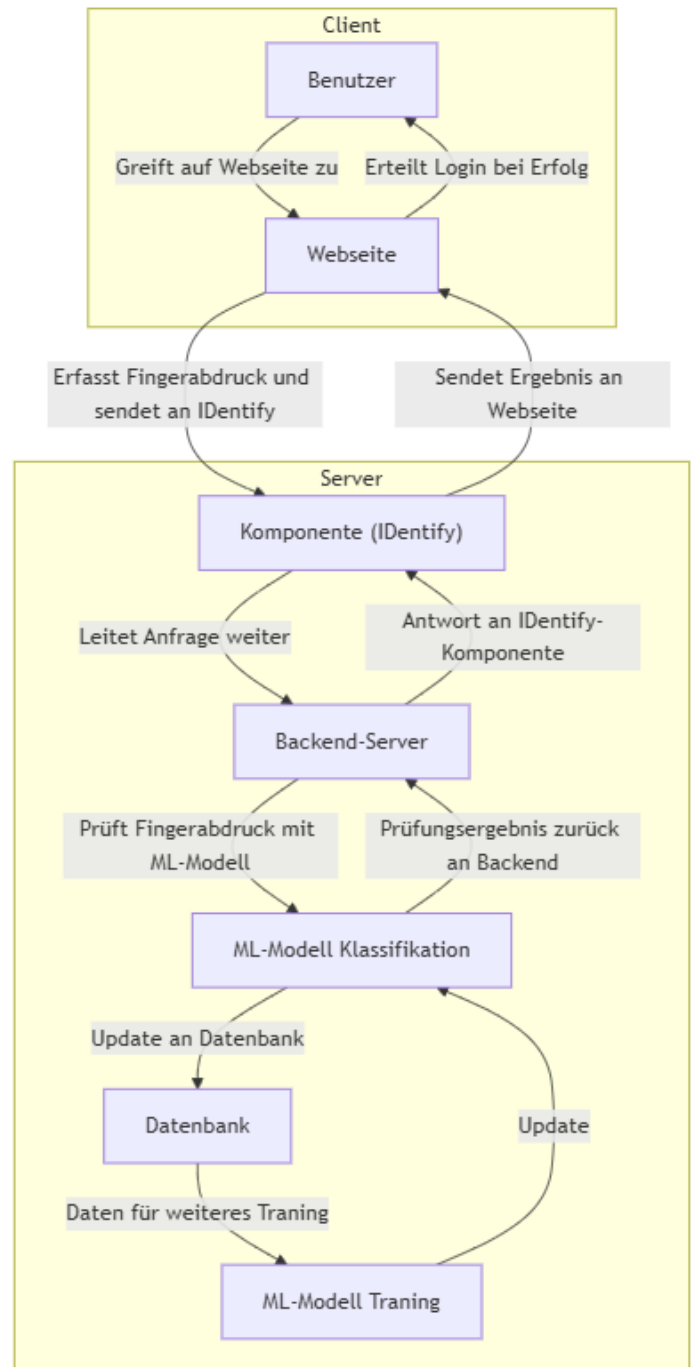


Abbildung 1: Systemübersicht: Verarbeitung von Canvas-Fingerabdrücken mit ML-Modellen.

Die Abbildung 1 zeigt die Architektur des Systems.

⁷Fingerabdrücke bleiben nur stabil bis Browser-Updates oder Systemänderungen erfolgen.

⁸Beispielsweise können Banken Canvas-Fingerabdrücke nutzen, um verdächtige Login-Versuche zu erkennen, was eines der Anwendungsgebiete innerhalb KOBILs sein könnte.

Der Benutzer greift über eine Webseite auf das System zu, wobei der Fingerabdruck erfasst und an das Backend weitergeleitet wird. Dort prüft ein Machine-Learning-Modell die Übereinstimmung mit den bereits erlernten Fingerabdrücken. Falls eine Übereinstimmung festgestellt wird, wird der Nutzer angemeldet.

Aufbauend auf einem bereits erfolgreich absolvierten Praxissemester erweitert diese Masterarbeit das bestehende Thema unter der Betreuung von Herrn Dr. Hossein Federooni[25], dessen umfangreiche Forschung und Veröffentlichungen im Bereich der Künstlichen Intelligenz und des Modelltrainings maßgeblich zur Auswahl der Projektziele beigetragen haben[26]. Dies führt uns zur Forschungsfrage dieser Arbeit.

E. Forschungsfrage

Die zentrale Forschungsfrage dieser Masterarbeit lautet:

Wie können bestehende Machine-Learning-Modelle und Techniken im Rahmen des Projekts IDentify optimiert werden, um die Benutzererkennung mittels Canvas-Fingerprinting zu verbessern, und wie lässt sich überprüfen, ob diese Ansätze auch bei größeren Datenmengen zuverlässig funktionieren?

Im Rahmen dieser Forschungsfrage wird untersucht, ob die Ergebnisse der vorherigen Arbeit validiert werden können, da diese bislang nur rudimentär getestet wurden. Darüber hinaus wird geprüft, ob es alternative, einfachere Machine-Learning-Ansätze gibt, die die gestellte Aufgabe ebenfalls erfolgreich bewältigen können. Zusätzlich werden mehrere Modelle entwickelt und getestet sowie eine umfangreichere Datenbasis erhoben, um die Validierung der Ergebnisse zu untermauern

Unter anderem wurden die folgenden Projektziele herausgearbeitet.

Projektziele

• Ziele der Datensammlung:

- **Anzahl der Stichproben:** Sammlung von insgesamt 100.000 Stichproben zur Gewährleistung einer robusten Datenbasis.
- **Geräteanzahl:** Datensammlung von mindestens 10, angestrebt sind bis zu 20 Geräte zur Erhöhung der Variabilität im Vergleich zu bisherigen Ansätzen.
- **Datenbalance:** Sicherstellung eines ausgewogenen Datenverhältnisses zur Vermeidung von Modellverzerrungen⁹. Dies soll durch unterschiedliche Gerätetypen wie Smartphones, Desktops und PCs mit verschiedenen Betriebssystemen erreicht werden.
- **Evaluierung:** Test verschiedener Machine-Learning-Modelle, von einfachen Klassifikatoren bis hin zu komplexeren neuronalen Netzen, um die effizienteste Lösung zu identifizieren.

Im Rahmen der Zusammenarbeit und fachlichen Beratung durch Herrn Dr. Federooni[26] wurde angeregt, verschiedene

Machine-Learning-Methoden systematisch zu evaluieren. Ziel dieser Untersuchung ist es, zu überprüfen, ob einfachere Ansätze ebenfalls in der Lage sind, die gestellte Aufgabe adäquat zu lösen, wodurch potenziell Ressourcen eingespart werden könnten.

Um die verschiedenen Evaluierungsansätze besser nachvollziehen zu können, werden im Folgenden die eingesetzten Machine-Learning-Techniken detailliert vorgestellt. Dabei wird systematisch untersucht, ob bereits einfache Klassifikatoren ausreichen, um Nutzerunterschiede zuverlässig zu erkennen, oder ob der Einsatz komplexerer neuronaler Netzwerke erforderlich ist. Diese Herangehensweise erlaubt eine fundierte Bewertung der Vor- und Nachteile der einzelnen Modelle und liefert wertvolle Erkenntnisse über ihre Eignung für die Benutzererkennung.

F. Random Forest und Isolation Forest

Der Random Forest und der Isolation Forest dienen als Baseline-Modelle aufgrund ihrer Robustheit und der Fähigkeit, auch bei kleineren Datensätzen effektiv zu arbeiten[27]. Diese funktionieren wie folgt: Der Isolation Forest isoliert Anomalien, indem er die Daten durch zufällige Aufteilungen in sogenannte Isolation Trees unterteilt. Datenpunkte, die ungewöhnlich sind, werden dabei meist schneller (also mit weniger Partitionen) isoliert, was sich in kürzeren Pfadlängen widerspiegelt. Wir haben uns entschieden, diese einfachen Methoden zu nutzen, um zu prüfen, ob bereits simple Ansätze ein funktionsfähiges Modell liefern können.

G. Autoencoder

Autoencoder werden häufig eingesetzt, um komplexe Merkmale aus Daten zu extrahieren und die Dimension der Daten zu reduzieren[28][29]. Sie bestehen aus zwei Hauptkomponenten: einem Encoder, der die Eingabedaten in eine niedrigdimensionale Darstellung komprimiert, und einem Decoder, der versucht, die ursprünglichen Daten aus dieser Darstellung zu rekonstruieren. Diese Struktur ermöglicht es, relevante Merkmale der Daten hervorzuheben und irrelevante Informationen zu entfernen. In dieser Arbeit nutzen wir Autoencoder, um die relevanten Merkmale aus Canvas-Fingerprints zu identifizieren und zu komprimieren. Diese Technik ermöglicht es uns zu prüfen, ob ein Modell, das nicht speziell für Bilddaten optimiert ist, dennoch in der Lage ist, sinnvolle Ergebnisse zu liefern

H. Convolutional Neural Networks

CNNs haben sich bei der Analyse von Bildmustern als besonders effektiv erwiesen[30]. In unserem Kontext werden sie zur Verarbeitung der Canvas-Fingerprints und zur Extraktion relevanter Merkmale eingesetzt. Wir haben uns entschieden, diese Technik zu nutzen, da sie in vorherigen Experimenten bereits gute Ergebnisse geliefert hat. CNNs sind in der Lage, durch ihre Schichtenstruktur komplexe Muster und Zusammenhänge in den Daten zu

⁹Modellverzerrungen entstehen, wenn ein Machine-Learning-Modell durch unausgewogene Trainingsdaten bestimmte Klassen bevorzugt oder benachteiligt.

erkennen, was sie besonders geeignet für die Analyse von Fingerabdruckdaten macht.

I. Komplexere Ansätze

Falls die vorherigen Modelle keine zufriedenstellenden Ergebnisse liefern, werden Siamese Neural Networks[31] als komplexere Alternative in Betracht gezogen. Siamese Neural Networks bestehen aus zwei identischen Subnetzwerken, die dieselben Gewichte teilen und darauf trainiert sind, die Ähnlichkeit oder Unterschiede zwischen zwei Eingaben zu bewerten. Sie nehmen zwei Eingabedaten gleichzeitig entgegen und berechnen einen Distanzwert, der angibt, wie ähnlich oder unterschiedlich die beiden Datenpunkte sind.

Das Ziel ist es, ein robustes System zu entwickeln, das auf den effektivsten Ansätzen basiert und eine präzise Benutzererkennung ermöglicht. Um dies zu erreichen, wird ein systematischer Evaluierungsansatz verfolgt, der verschiedene Aspekte des maschinellen Lernens berücksichtigt.

- **Modellevaluierungsmethoden:** Die Modelle werden mittels verschiedener Evaluierungsmethoden bewertet. Dabei kommen unter anderem die Analyse von falsch-positiven und falsch-negativen Ergebnissen mithilfe einer Konfusionsmatrix[2] sowie die Berechnung des F1-Scores zur Bewertung von Präzision und Recall[1] zum Einsatz. Weitere Metriken werden berücksichtigt, um eine umfassende Bewertung der Modelle zu gewährleisten.
- **Datensatzaufteilung:** Der Datensatz wird in ein Trainingsset (70%), ein Validierungsset (10%) und ein Testset (20%) aufgeteilt[32], um eine effektive Trainings- und Evaluationsphase zu ermöglichen.

Durch die Erweiterung der Projektziele wird die bestehende Implementierung um verschiedene Methoden des maschinellen Lernens ergänzt. Dies ermöglicht eine Evaluation unterschiedlicher Ansätze zur Benutzererkennung, beginnend mit einfachen Klassifikatoren bis hin zu komplexeren Modellen. Insgesamt soll das Projekt die Machbarkeit der Canvas Fingerprinting-Technik als zusätzliche Sicherheitsebene demonstrieren sowie deren Beitrag zur Erhöhung der Sicherheit und Benutzerfreundlichkeit von Webanwendungen evaluieren.

J. Relevanz

Die Relevanz dieser Arbeit ergibt sich aus den aktuellen Herausforderungen im Bereich der digitalen Authentifizierung. Traditionelle Methoden wie Passwörter sind nachweislich anfällig für Brute-Force-Angriffe[3] und Phishing[4], wie bereits erwähnt wurde. Diese Schwachstellen gefährden nicht nur die Sicherheit, sondern beeinträchtigen auch die Benutzerfreundlichkeit, da Nutzer eine Vielzahl von Passwörtern verwalten müssen[8].

Canvas Fingerprinting bietet hier einen innovativen Ansatz zur Erhöhung der Sicherheit von Webanwendungen[20]. Diese Technik kann als zusätzliche Authentifizierungsmethode genutzt werden, um verdächtige Anmeldeaktivitäten zu

erkennen. Dies erlaubt es, potenzielle Angreifer zu identifizieren, ohne dass Nutzer aktiv eingreifen müssen[33] und den Anmeldeversuch zu stoppen. Dadurch wird eine Balance zwischen erhöhter Sicherheit und Benutzerfreundlichkeit geschaffen, indem zusätzliche Authentifizierungsschritte nur bei signifikanten Abweichungen vom erwarteten Fingerabdruck erforderlich werden[34].

K. Aufbau der Arbeit

Diese Arbeit ist wie folgt gegliedert: Nach der Einführung in die Problemstellung, Zielsetzung und Relevanz des Themas wird im nächsten Abschnitt der aktuelle Stand der Technik dargestellt. Hierbei werden zentrale Technologien und Methoden des Browser-Fingerprintings erläutert, mit besonderem Fokus auf Canvas Fingerprinting. Ergänzend dazu wird im Abschnitt *Related Work* ein Überblick über relevante Forschungsarbeiten gegeben.

Im Anschluss daran wird die Methodik dieser Arbeit beschrieben. Hier werden die theoretischen Grundlagen sowie die geplante Vorgehensweise zur Implementierung des Systems basierend auf Canvas Fingerprinting detailliert erläutert. Dies umfasst die Datensammlung, Merkmalsextraktion und Modellarchitekturen.

Es folgt eine detaillierte Beschreibung der Implementierung des Systems. Dieser Abschnitt ist in zwei Hauptteile gegliedert:

Der erste Teil beschreibt den ursprünglichen Ansatz mit sechs CNN-Modellen, der auf einer kleineren Datenbasis von 1.900 Samples basiert. Hier werden die Modellarchitekturen, die Implementierung der Mehrheitsabstimmung und die ersten Ergebnisse vorgestellt.

Der zweite Teil präsentiert den erweiterten Ansatz, der auf einer größeren Datenbasis von 100.000 Samples pro Nutzer aufbaut. Hier werden verschiedene Machine-Learning-Ansätze systematisch untersucht, darunter Isolation Forest, Autoencoder und optimierte CNN-Modelle.

Im Ergebnisteil werden die Leistungen beider Implementierungsansätze verglichen. Die Effizienz und Genauigkeit der entwickelten Modelle werden anhand verschiedener Metriken wie Genauigkeit, Präzision und F1-Score detailliert evaluiert. Zudem werden spezifische Tests zur Unterscheidungsfähigkeit der Modelle durchgeführt, um die Skalierbarkeit und Robustheit beider Ansätze zu vergleichen.

Anschließend folgt eine Diskussion der Ergebnisse. Dabei werden die Leistungen der verschiedenen Modelle verglichen, Herausforderungen und Limitationen erörtert sowie Verbesserungsvorschläge und zukünftige Forschungsansätze diskutiert.

Abschließend wird ein Fazit gezogen, das die wichtigsten Erkenntnisse dieser Arbeit zusammenfasst. Zudem werden die methodischen Erkenntnisse, praktischen Implikationen und mögliche zukünftige Forschungsrichtungen aufgezeigt, einschließlich weiterführender Anwendungen von Canvas Fingerprinting in verschiedenen Bereichen.

III. STAND DER TECHNIK

Browser-Fingerprinting hat sich als leistungsstarke Technik zur Identifizierung von Nutzern anhand einzigartiger Merkmale ihrer Browser- und Geräteumgebung etabliert. Diese Methode findet sowohl im Tracking als auch in Sicherheitsanwendungen Verwendung, wobei Canvas Fingerprinting aufgrund seiner hohen Entropie und Persistenz eine zentrale Rolle spielt.

Die Entwicklung von Browser-Fingerprinting-Techniken hat in den letzten Jahren beachtliche Fortschritte gemacht. Laperdrix et al.[35] bieten in ihrer umfassenden Übersichtsarbeit einen detaillierten Einblick in die Forschung auf diesem Gebiet, einschließlich der Funktionsweise und Ursprünge dieser Technik. Ihre Analyse der Zusammensetzung moderner Fingerprints und deren aktueller Nutzung im Internet liefert Erkenntnisse für Forscher und Neueinsteiger in dieses Gebiet.

Im Bereich des Canvas Fingerprintings leisteten Mowery und Shacham Pionierarbeit. Ihre Studie "Pixel Perfect: Fingerprinting Canvas in HTML5"[19] legte das Fundament für diese spezifische Technik. Seitdem wurde Canvas Fingerprinting kontinuierlich weiterentwickelt und verfeinert. Die Forschung von Raschke und Küpper[36] zur Echtzeit-Erkennung von Canvas-Fingerprinting bietet Einblicke in die Verbreitung dieser Technik auf populären Websites und zeigt somit deren praktische Relevanz. Die Anwendungsmöglichkeiten von Browser-Fingerprinting haben sich zuletzt insbesondere im Bereich der Sicherheit und Authentifizierung erheblich erweitert. Das SWAT-Protokoll[37] (Seamless Web Authentication Technology) bietet einen innovativen Ansatz, der Canvas Fingerprinting mit maschinellem Lernen kombiniert, um eine zuverlässige Nutzerauthentifizierung zu ermöglichen. Diese Arbeit diente als Hauptinspiration für das Projekt. Die Technik wurde jedoch weiterentwickelt und an spezifische Anforderungen angepasst, um ihre Effektivität zu steigern. Das SWAT-Protokoll verdeutlicht das Potenzial von Browser-Fingerprinting als robuste und benutzerfreundliche Sicherheitslösung.

Acar et al.[38] haben mit FPDetective ein Framework entwickelt, das Browser-Fingerprinting-Skripte in großem Maßstab erkennen und analysieren kann. Ihre Arbeit verdeutlicht die Verbreitung und Techniken von Fingerprinting auf verschiedenen Websites und trägt zum besseren Verständnis dieser Praktiken bei. Die Arbeit von Alaca et al.[39] demonstriert die praktische Anwendbarkeit dieser Technologie, indem sie eine skalierbare und sichere HTML5 Canvas-basierte Nutzerauthentifizierungsmethode vorstellt. Ihr Ansatz demonstriert die Vielseitigkeit und das Potenzial von Canvas Fingerprinting in Authentifizierungssystemen und bestätigt damit die Relevanz unserer Forschungsrichtung.

Die Fortschritte in der Forschung und Anwendung von Browser-Fingerprinting-Techniken zeigen ihre wachsende Bedeutung in der digitalen Welt. Neben neuen Möglichkeiten zur Verbesserung von Sicherheit und Authentifizierung werfen sie jedoch auch wichtige Fragen zu Privatsphäre und Datenschutz auf, die in zukünftigen Arbeiten weiter untersucht werden sollten.

A. Kommerzielle Nutzung und FingerprintJS-Bibliothek

Browser-Fingerprinting hat sich über die Forschung hinaus zu einer bedeutenden kommerziellen Technologie entwickelt, wie verschiedenen Statistiken und Forschungen zeigen. Diese Technologie wird bereits von diversen Unternehmen genutzt. Eine führende Lösung in diesem Bereich ist die *FingerprintJS*-Bibliothek, die sowohl als Open-Source-Projekt auf GitHub[40] als auch als kommerzielle Version von *Fingerprint.com*[14] verfügbar ist. Diese Bibliothek ermöglicht Entwicklern die Implementierung fortschrittlicher Fingerprinting-Lösungen, die Nutzer selbst bei deaktivierten Cookies oder im Inkognito-Modus identifizieren können.

Laut Eigenangaben wird die kommerzielle Version von FingerprintJS auf 12% der Top 500 Websites eingesetzt und erreicht eine Wiedererkennungsrates von 99,5%[14]. Die Bibliothek kombiniert verschiedene Techniken wie Canvas Fingerprinting, WebGL und Audio-Fingerprinting, um präzise digitale Identitäten zu erstellen.

Die weite Verbreitung dieser Technologie zeigt ihre Bedeutung in Bereichen wie Betrugsprävention, Benutzeranalyse und personalisierter Werbung[14]. Allerdings wirft der kommerzielle Einsatz von Browser-Fingerprinting auch ernsthafte Datenschutzfragen auf, da Nutzer häufig nicht über den Einsatz solcher Tracking-Methoden informiert werden.

B. Forschungsergebnisse zu Canvas Fingerprinting

Canvas Fingerprinting wurde in verschiedenen Kontexten umfassend untersucht. Raschke und Küpper[36] entwickelten 2018 eine Echtzeit-Browsererweiterung zur Erkennung von Canvas-Fingerprints. Ihre Analyse zeigte, dass 4,4% der 500 beliebtesten deutschen Websites diese Technik nutzen, wobei viele dieser Skripte aus Open-Source-Bibliotheken wie *fingerprintjs*[14] stammen.

Das SWAT-Protokoll (Seamless Web Authentication Technology) von Rochet et al.[37] kombiniert Canvas Fingerprinting mit maschinellem Lernen zur Nutzerauthentifizierung und nutzt Challenge-Response-Mechanismen sowie Convolutional Neural Networks (CNNs). Diese Entwicklung zeigt das Potenzial von Browser-Fingerprinting als robuste und benutzerfreundliche Sicherheitslösung auf. In dieser Arbeit wurde das SWAT-Protokoll als Grundlage genutzt, jedoch angepasst und erweitert, um spezifische Anforderungen zu erfüllen.

Eine umfassende Übersicht von Zhang et al.[41] aus dem Jahr 2022 beleuchtet verschiedene Aspekte des Browser-Fingerprintings, einschließlich Techniken, Anwendungen und Gegenmaßnahmen. Diese Studie ist Teil einer wachsenden Forschungslandschaft, die sich mit den verschiedenen Facetten des Browser-Fingerprintings auseinandersetzt. Die Autoren diskutieren verschiedene Fingerprinting-Methoden und betonen deren Bedeutung für unterschiedliche Anwendungsgebiete. Ähnliche Überblicksarbeiten, wie die von Laperdrix et al.[35], unterstreichen die zunehmende Relevanz und das wachsende Interesse an Browser-Fingerprinting-Techniken in der Forschungsgemeinschaft. Diese Studien zeigen, dass auch an Umgehungsstrategien und Gegenmaßnahmen für Finger-

printing Techniken geforscht wird und welche Techniken dafür eingesetzt werden können.

Zusätzlich haben Alaca et al.[39] eine skalierbare und sichere HTML5 Canvas-basierte Benutzerauthentifizierungsmethode entwickelt wie bereits angesprochen, die die praktische Anwendbarkeit dieser Technologie demonstriert und das Potenzial von Canvas Fingerprinting in Authentifizierungssystemen unterstreicht und als Alternative zu dem Paper SWAT[37] steht.

C. Kritik und Herausforderungen

Trotz der beeindruckenden technischen Möglichkeiten von Browser-Fingerprinting gibt es erhebliche Herausforderungen. Ein zentrales Problem sind die Datenschutzbedenken, da diese Technik oft ohne Wissen oder Zustimmung der Nutzer eingesetzt wird[42]. Laut der Studie von Laperdrix et al.[35] steht dies im Konflikt mit Datenschutzgesetzen und erfordert in vielen Fällen die Einwilligung der Nutzer. Die Europäische Datenschutzbehörde (EDPB)[43] hat festgelegt, dass die Notwendigkeit einer Einwilligung vom Verwendungszweck des Fingerprinting abhängt. Während es für Authentifizierung und Sicherheitszwecke ohne Einwilligung eingesetzt werden kann, ist für Tracking und Profiling die Zustimmung des Nutzers erforderlich[35].

1) Technische Herausforderungen und Gegenmaßnahmen

Im Zuge der Entwicklung von Browser-Fingerprinting-Techniken haben sich auch verschiedene Gegenmaßnahmen etabliert. Diese zielen darauf ab, die Erfassung konsistenter Fingerabdrücke zu erschweren oder zu verhindern. Zwei Hauptansätze haben sich dabei herauskristallisiert.

Diese beiden sind API-Normalisierung und Randomisierung:

- **API-Normalisierung:** Browser wie Tor[44] und Brave[45] implementieren Techniken zur Normalisierung von API-Aufrufen, um die Einzigartigkeit von Fingerabdrücken zu reduzieren[46]. Diese Normalisierung sorgt dafür, dass API-Ausgaben standardisiert werden und somit weniger Rückschlüsse auf individuelle Geräte zulassen[47].
- **Randomisierung:** Ansätze wie FPRandom, eine modifizierte Version von Firefox, fügen Zufälligkeit hinzu, um moderne Fingerprinting-Algorithmen zu beeinträchtigen[35]

Zusätzlich bieten Browser-Erweiterungen Schutz vor Fingerprinting:

- uBlock Origin[48] und Privacy Badger[49] blockieren bekannte Fingerprinting-Skripte.
- Erweiterungen wie CanvasBlocker[50] verhindern spezifische Fingerprinting-Techniken wie Canvas Fingerprinting.

Das Bundesamt für Sicherheit in der Informationstechnik (BSI)[51] empfiehlt ebenfalls den Einsatz solcher Schutzmaßnahmen, um die Privatsphäre beim Surfen zu verbessern[42]. Studien zeigen jedoch auch, dass einige dieser Maßnahmen paradoxerweise zu einem einzig artigeren Fingerabdruck führen können, da sie das Verhalten eines Browsers verändern und ihn dadurch identifizierbarer machen können[46].

D. Related Work

Die Forschung im Bereich Browser-Fingerprinting umfasst mehrere Dimensionen, darunter die Erkennung von Tracking-Techniken, die Entwicklung von Gegenmaßnahmen und die Anwendung dieser Technologien in Sicherheitskontexten.

Wie bereits im Stand der Technik erwähnt, stellt das SWAT-Protokoll (Seamless Web Authentication Technology) von Rochet et al.[37] einen innovativen Ansatz in diesem Bereich dar. SWAT nutzt die Variationen beim Rendering von HTML5-Canvas-Elementen[18], die durch unterschiedliche Software- und Hardware-Konfigurationen entstehen, um ein nahtloses Challenge-Response-Authentifizierungsprotokoll zu implementieren. Die Autoren berichten von einer bemerkenswerten Effizienz, mit einer Lernphase von weniger als 3 Minuten pro Benutzer bei guter Genauigkeit und fast keinem Overfitting[52]. Dies unterstreicht die praktische Anwendbarkeit des Ansatzes, selbst für Systeme mit einer großen Anzahl von Benutzern.

Unsere Arbeit baut auf diesen Erkenntnissen auf, unterscheidet sich jedoch in wesentlichen Aspekten. Während SWAT direkt Convolutional Neural Networks (CNNs) einsetzt, verfolgen wir zunächst einen schrittweisen Ansatz mit einfacheren Modellen, um die Grundlagen zu analysieren und zu verstehen. Erst in späteren Phasen planen wir den Übergang zu komplexeren Modellen wie CNNs. Diese iterative Vorgehensweise ermöglicht es uns, die Technologie schrittweise zu evaluieren und anzupassen.

IV. METHODIK

Die Methodik dieser Arbeit konzentriert sich auf die praktische Implementierung und Evaluierung von Canvas Fingerprinting im Rahmen des Projektes IDentify[10]. Der Ansatz kombiniert Webentwicklungstechniken mit datengetriebener Analyse, um die technische Machbarkeit und Effizienz der Methode zu untersuchen.

A. Code-Backbone: Das technologische Fundament

Dieser Abschnitt beschreibt die technische Umsetzung des Canvas Fingerprintings zur Nutzeridentifikation. Ziel ist die Entwicklung eines Frameworks zur Erfassung und Analyse von Canvas-Fingerprints. Die Implementierung erfolgt in zwei Kernschritten:

- 1) **Erzeugung des Fingerprints:** Ein JavaScript-Skript sammelt gerätespezifische Merkmale, wie:
 - User-Agent und Browser-Version,
 - Bildschirmauflösung und Farbtiefe,
 - installierte Schriftarten und Plugins,
 - Eigenschaften des Canvas-Renderings.
- 2) **Nutzeridentifikation:** Der erstellte Fingerprint wird codiert und an den Server übertragen. Bekannte Fingerprints werden dem entsprechenden Benutzer (z. B. *Benutzername_n*) zugeordnet, während neue Einträge zu neuen IDs bzw. Benutzern führen.

Das Framework dient drei Hauptzwecken:

- Sammlung individueller Canvas-Fingerprints
- Validierung der Wiedererkennungsgenauigkeit
- Skalierungstests

Die technische Basis bildet eine Express.js-Anwendung[53] mit Node.js-Backend[54].

B. Implementierung des Canvas Fingerprints

Die Implementierung des Canvas Fingerprints erfolgt in zwei zentralen Schritten: der Sammlung von Canvas Fingerprints und der Extraktion der Pixel-Daten.

- **Sammlung von Canvas Fingerprints:** Nach der Identifikation des Nutzers wird ein Skript auf der Webseite ausgeführt, das ein HTML5-Canvas-Element erzeugt. Innerhalb dieses Canvas-Elements werden spezifische Inhalte wie Text oder Grafiken gezeichnet. Die Art und Weise, wie diese Inhalte gerendert werden, hängt von der individuellen Hardware- und Softwarekonfiguration des Geräts ab, wie beispielsweise dem Betriebssystem, der Grafikkarte oder den Treibern. Dadurch entsteht ein einzigartiger Fingerprint, der zur weiteren Verarbeitung verwendet wird.
- **Extraktion der Pixel-Daten:** Nachdem die Inhalte auf das Canvas-Element gezeichnet wurden, extrahiert das Skript die Pixel-Daten des gerenderten Bildes. Diese Daten werden anschließend in eine Base64-codierte Darstellung umgewandelt. Die Base64-Codierung dient dazu, die Daten effizient zu speichern und für die weitere

Verarbeitung bereitzustellen, beispielsweise für die Speicherung in einer Datenbank oder die Übertragung an einen Server.

Diese Schritte bilden die Grundlage für die Implementierung des Canvas Fingerprinting im Projekt.

C. Ziele der Datensammlung

Die Datensammlung soll, wie bereits erwähnt, mehrere Beispiele eines Canvas Fingerprints für jeden Nutzer enthalten. Diese Fingerprints werden in einer Datenbank gespeichert, um sie für das System verfügbar zu machen. Geplant ist, die Daten in einer MongoDB zu speichern. Die folgenden Punkte beschreiben die methodische Planung der Datensammlung:

- **Stichprobenanzahl (Samples):** Es ist vorgesehen, insgesamt 100.000 Stichproben pro Nutzer zu sammeln. Dies soll durch automatisierte Prozesse erfolgen und, wenn möglich, regelmäßig aktualisiert werden.
- **Diversität der Geräte:** Die Daten sollen gezielt von mindestens 10 verschiedenen Geräten erhoben werden, mit dem Ziel, diese Zahl auf bis zu 20 Geräte zu erhöhen. Hierbei werden sowohl interne Unternehmensgeräte als auch externe Geräte einbezogen, um eine hohe Variabilität und Generalisierbarkeit der Modelle sicherzustellen.
- **Sicherstellung der Datenbalance:** Um Verzerrungen in den Modellen zu vermeiden, wird ein ausgewogenes Verhältnis der gesammelten Daten angestrebt. Dies umfasst die gezielte Auswahl unterschiedlicher Gerätetypen (z. B. Smartphones, Desktops) sowie die kontinuierliche Überprüfung der Datenbalance anhand spezifischer Metriken.
- **Datentyp:** Die gesammelten Daten bestehen aus eindimensionalen (1D) String-Daten, die aus den Canvas Fingerprints der Benutzer generiert werden. Diese Struktur wurde gewählt, um eine effiziente Verarbeitung durch maschinelle Lernmodelle zu ermöglichen.

Dies bildet die Grundlage, um auf dem bestehenden Datensatz, der bisher nur 1.900 Samples pro Nutzer enthält, aufzubauen und die Funktionalität des Systems zu gewährleisten. Dabei wird besonderer Wert auf die Qualität und Vielfalt der Daten gelegt, um präzise Ergebnisse und eine hohe Skalierbarkeit der entwickelten Modelle sicherzustellen.

D. Geplante Modelle und Architekturen

Im Rahmen des Projekts IDentify werden verschiedene maschinelle Lernmodelle und Architekturen implementiert und getestet, um die Benutzererkennung auf Basis von Canvas Fingerprints zu optimieren. Die geplanten Modelle und deren Implementierung umfassen sowohl klassische Ansätze als auch fortschrittliche neuronale Netzwerke. Ziel ist es, eine robuste und skalierbare Lösung zu entwickeln, die präzise Ergebnisse liefert. Die folgenden Modelle und Architekturen stehen im Fokus:

1) Isolation Forest Classifier

Der Isolation Forest Classifier wird als Ensemble-Learning-Ansatz implementiert. Dieses Modell kombiniert mehrere Entscheidungsbäume, um die Klassifikationsgenauigkeit zu erhöhen. Für die Implementierung wird eine Python-Bibliothek

wie `scikit-learn` verwendet. Die Hyperparameter des Modells, wie die Anzahl der Bäume (`n_estimators`) und die maximale Tiefe (`max_depth`), werden initial auf Standardwerte gesetzt und später durch Hyperparameter-Tuning optimiert.

2) *Convolutional Neural Networks (CNNs)*

CNNs werden eingesetzt, um sowohl eindimensionale (1D) String-Daten als auch zweidimensionale (2D) Bilder zu verarbeiten:

- **1D-CNNs:** Diese Architektur wird direkt auf die String-Daten angewendet. Die Eingabedaten werden als eindimensionale Sequenzen verarbeitet, wobei Faltungsoperationen genutzt werden, um relevante Merkmale zu extrahieren.
- **2D-CNNs:** Hierbei werden die 1D-Daten in 2D-Bilder konvertiert, um räumliche Merkmale besser erfassen zu können. Diese Methode nutzt typische CNN-Architekturen wie ResNet oder VGG, die mit vortrainierten Gewichten initialisiert werden können.

Die Implementierung erfolgt mit Frameworks wie TensorFlow oder PyTorch. Das Training wird auf GPUs durchgeführt, um die Rechenzeit zu minimieren.

3) *Autoencoder*

Autoencoder werden untersucht, um ihre Eignung zur Benutzererkennung zu evaluieren. Diese Architektur besteht aus einem Encoder-Decoder-Modell, das versucht, die Eingabedaten zu rekonstruieren. Ziel ist es, latente Merkmale zu extrahieren, die für die Klassifikation genutzt werden können. Die Implementierung erfolgt mit TensorFlow, wobei der Fokus auf der Optimierung der Rekonstruktionsfehler liegt.

4) *Siamese Neural Networks (SNNs)*

SNNs werden als potenzieller Ansatz für komplexe Benutzererkennungsprobleme in Betracht gezogen. Diese Netzwerke sind darauf spezialisiert, Ähnlichkeiten zwischen Datenpaaren zu erkennen. Sie bestehen aus zwei identischen Subnetzwerken, die gemeinsame Gewichte teilen und einen Distanzwert zwischen den Eingaben berechnen. SNNs könnten dann implementiert werden, wenn andere Architekturen nicht zufriedenstellend performen.

5) *Ensemble-Learning-Techniken*

Neben dem Isolation Forest Classifier werden weitere Ensemble-Learning-Methoden eingesetzt. Diese Modelle kombinieren mehrere schwache Klassifikatoren, um eine robuste Gesamtklassifikation zu erzielen. Sowohl Soft-Voting als auch Hard-Voting-Methoden sollen implementiert und getestet werden.

E. Implementierungsstrategie

Die Implementierung der Modelle erfolgt schrittweise:

- 1) **Datenvorbereitung:** Die Daten werden vorverarbeitet und in Trainings-, Validierungs- und Testsets aufgeteilt (70% Training, 10% Validierung, 20% Test).
- 2) **Baseline-Modelle:** Zunächst werden Standardmodelle ohne Hyperparameter-Tuning implementiert, um eine Basisleistung zu messen.

- 3) **Hyperparameter-Tuning:** Nach der Baseline-Evaluierung erfolgt eine Optimierung der Hyperparameter.

- 4) **Modellvergleich:** Die Leistung der verschiedenen Modelle wird anhand von Metriken wie Genauigkeit, F1-Score[1] und Konfusionsmatrix[2] verglichen.

- 5) **Evaluation:** Die besten Modelle werden hinsichtlich ihrer Robustheit gegen unterschiedliche Nutzer getestet.

F. Zusammenfassung

Dieser strukturierte Ansatz ermöglicht eine detaillierte Untersuchung der eingesetzten Methoden und bietet eine solide Grundlage für zukünftige Arbeiten im Bereich der Datenanalyse und Modelloptimierung. In dem folgenden Kapitel zur Implementierung betrachten wir detailliert die eingesetzten Technologien und verschiedenen Ansätze. Die Implementierung wird in zwei Hauptteile gegliedert: den bisherigen Ansatz und den aktuellen Ansatz

V. IMPLEMENTIERUNG

Dieses Kapitel beschreibt die technische Umsetzung des Projekts. Es werden die Systemarchitektur und die wichtigsten Implementierungsdetails erläutert. Besonderes Augenmerk liegt auf der containerisierten Architektur und deren Vorteilen für das Projekt.

A. Systemarchitektur

Die Systemarchitektur des Projekts IDentify wurde so gestaltet, dass sie eine hohe Portabilität und Skalierbarkeit gewährleistet. Das System besteht aus mehreren Komponenten, die miteinander kommunizieren und sicherstellen, dass es korrekt funktioniert, darunter die folgenden Hauptkomponenten:

- **Frontend:** Das Frontend ist für die Benutzerinteraktion verantwortlich. Es ermöglicht den Benutzern, Fingerprints zu erstellen und zu verifizieren. Es wurde mit Webtechnologien wie HTML, CSS und JavaScript entwickelt und kommuniziert mit dem Backend über HTTP- und WebSocket-Protokolle.
- **Backend:** Das Backend dient als Vermittler zwischen dem Frontend und den Machine-Learning-Komponenten. Es verarbeitet Anfragen vom Frontend und leitet diese an die entsprechenden Machine-Learning-Modelle weiter. Das Backend wurde in Node.js implementiert und verwendet Express.js für die Erstellung der API-Endpunkte.
- **Machine-Learning-Komponenten:** Diese Komponenten sind für die Analyse und Klassifikation der Fingerprints verantwortlich. Sie verwenden die von uns ausgewählten Machine-Learning-Modelle, die mit unterschiedlichen Methoden trainiert wurden. Die Modelle wurden in Python entwickelt und über Flask als Webservice bereitgestellt.
- **Datenbank:** Zur Speicherung der Fingerprints sowie zugehöriger Informationen wird MongoDB verwendet. Die Datenbank ermöglicht eine effiziente Speicherung sowie schnelle Abfragen der Daten.

Die folgende Abbildung Abbildung 2 zeigt die Hauptkomponenten des Systems sowie deren Interaktionen. In der Abbildung kann man den Datenfluss zwischen den Hauptkomponenten des Systems erkennen. Das Frontend kommuniziert mit dem Backend über HTTP- und WebSocket-Protokolle. Das Backend verarbeitet Anfragen und leitet sie an die Machine-Learning-Komponenten weiter oder ruft Daten aus der Datenbank ab.

B. Containerisierte Systemarchitektur

Um eine flexible und skalierbare Lösung zu schaffen, wurde das gesamte System in Docker-Container verpackt. Die Containerisierung bietet mehrere Vorteile: Sie ermöglicht es beispielsweise, bei Fehlern einfach auf eine vorherige Version zurückzukehren oder einzelne Komponenten unabhängig voneinander zu aktualisieren. Zudem verbessert sie die Portabilität und Replizierbarkeit des Systems erheblich.

Im Rahmen des Projekts wurden mehrere Docker-Container implementiert:

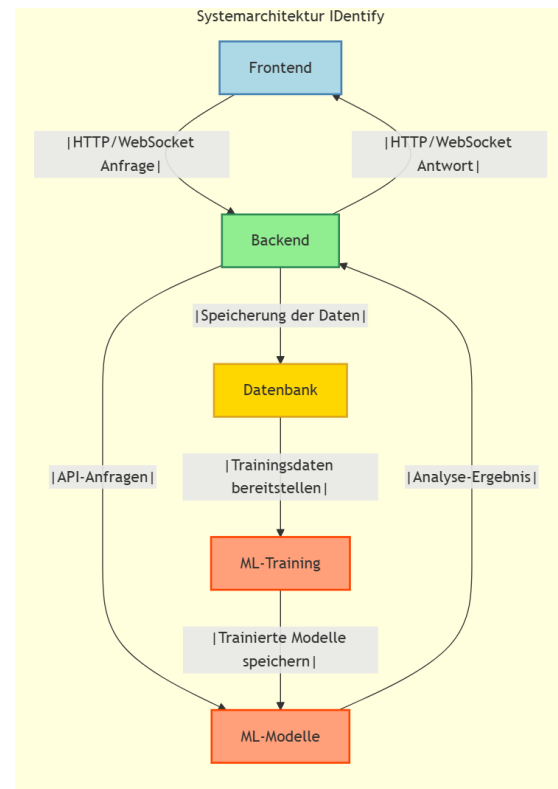


Abbildung 2: Framework des Projekts IDentify

- **Machine-Learning-Komponente:** Dieser Container führt die Machine-Learning-Modelle aus, die zur Analyse und Klassifikation der Fingerprints verwendet werden. Er basiert auf einem Python-Image und enthält alle notwendigen Bibliotheken wie TensorFlow und Flask.
- **Machine-Learning-Training:** In diesem Container werden die Modelle trainiert. TensorFlow wird verwendet, um die gesammelten Fingerprints zu analysieren und Modelle zu erstellen. Der Container enthält zudem Jupyter-Notebooks zur Dokumentation der Trainingsprozesse.
- **Frontend und Backend:** Dieser Container hostet sowohl das Frontend als auch das Backend zur Benutzerinteraktion. Er basiert auf einem Node.js-Image mit allen notwendigen Abhängigkeiten wie Express.js und Winston.
- **Datenbank (MongoDB):** Dieser Container nutzt das MongoDB-Image zur Speicherung der Fingerprints sowie zugehöriger Informationen. Die Datenbank ermöglicht eine effiziente Speicherung sowie schnelle Abfragen der Daten.
- **Jupyter Notebook:** Dieser Container stellt eine Umgebung für interaktive Datenanalysen und Modelltrainings zur Verfügung. Er basiert auf einem speziellen Dockerfile, das die erforderlichen Abhängigkeiten installiert.

Die Dateninitialisierung erfolgt durch ein Skript (init-mongo.sh), das im Docker-Setup definiert ist. Dieses Skript importiert die notwendigen JSON-Daten in die MongoDB-

Datenbank, sobald der db-Service gestartet wird. Somit ist der Service portabel, kann schnell migriert werden und setzt die Datenbank automatisiert wieder in den korrekten Zustand zurück

Die Verwendung von Docker ermöglicht es uns außerdem, verschiedene Teile des Systems unabhängig zu entwickeln und zu testen, während gleichzeitig eine konsistente Umgebung gewährleistet wird. Diese Struktur unterstützt nicht nur die Entwicklung und das Testen von Modellen, sondern sorgt auch dafür, dass die finale Version des Modells im Hauptframework einfacher integriert werden kann.

Die Machine-Learning-Komponente, die hier beschrieben wird, ist für die Produktionsumgebung vorgesehen. Sie übernimmt das finale Modell und führt das Training mit diesem durch. In unserem Fall testen wir jedoch noch verschiedene Ansätze und nutzen daher eine alternative Umgebung, die sich vollständig auf die Entwicklung der Modelle konzentriert. Sobald die optimale Architektur mit den passenden Hyperparametern gefunden wurde, wird diese an die Produktionsumgebung übergeben und dort integriert. Dadurch können die Modelle dynamisch trainiert werden, um Änderungen in den Fingerabdrücken auszugleichen. Diese alternative Umgebung, die als Experiment-Setup bezeichnet wird, wird innerhalb des Jupyter-Notebook-Containers ausgeführt. Zusätzlich ist sie vom GitHub-Repository und damit vom Projekt abgekapselt. Die folgenden Abbildungen Abbildung 3 und Abbildung 4 zeigen die Interaktion zwischen den Containern des Frameworks und des Experiment-Setsups:

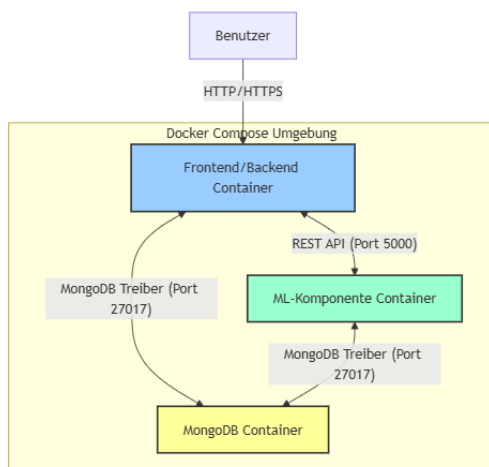


Abbildung 3: Framework des Projekts Identify

Die Abbildung Abbildung 3 zeigt die Hauptkomponenten des Frameworks:

- Der **Frontend/Backend-Container** ermöglicht die Benutzerinteraktion und verarbeitet Anfragen mit Node.js[54] und Express.js[53] als Frontend.
- Der **ML-Komponenten-Container** führt die Machine-Learning-Modelle aus, die zur Analyse und Klassifikation der Fingerprints verwendet werden.

- Die **MongoDB** dient als zentrale Datenbank, um Fingerprints und zugehörige Daten zu speichern.

Die Container kommunizieren über definierte Schnittstellen: Das Frontend sendet API-Anfragen an die ML-Komponente, während beide Container auf die MongoDB zugreifen, um Daten zu speichern oder abzurufen.

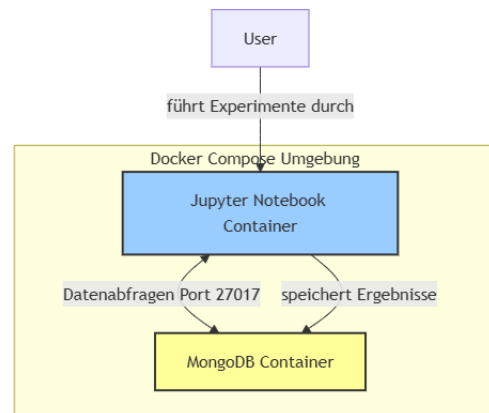


Abbildung 4: Experiment-Setup des Projekts Identify

Die Abbildung Abbildung 4 zeigt das Experiment-Setup:

- Der **Jupyter Notebook Container** wird verwendet, um Modelle zu erstellen und zu trainieren.
- Der **MongoDB Container** speichert die Trainingsdaten sowie Ergebnisse der Experimente.

Die Kommunikation erfolgt bidirektional: Der Jupyter Notebook Container ruft Daten aus der MongoDB ab und speichert Ergebnisse zurück in die Datenbank oder als Modell.

Docker Compose wird verwendet, um die verschiedenen Container des Projekts zu orchestrieren. Mithilfe einer YAML-basierten Konfigurationsdatei können alle Dienste, deren Abhängigkeiten sowie Netzwerke und Volumes definiert werden. Diese Datei ermöglicht eine zentrale Verwaltung der Container und bietet folgende Vorteile:

- **Skalierbarkeit:** Zusätzliche Dienste oder Container können einfach hinzugefügt werden, ohne die bestehende Architektur zu verändern. Dies ermöglicht eine flexible Anpassung an wachsende Anforderungen.
- **Konsistenz:** Die Konfiguration gewährleistet, dass das System in verschiedenen Umgebungen (Entwicklung, Test oder Produktion) identisch läuft.
- **Effiziente Verwaltung:** Mit einem einzigen Befehl (`docker-compose up`) können alle Container gestartet und miteinander verbunden werden.

Die YAML-Datei definiert die wichtigsten Dienste des Systems. Jeder Dienst wird in einem eigenen Abschnitt beschrieben und enthält grundlegende Konfigurationsparameter wie Build-Anweisungen, Umgebungsvariablen, Portweiterleitungen und Abhängigkeiten:

- **Frontend/Backend:** Dieser Dienst hostet die Benutzeroberfläche (Frontend) sowie das Backend zur Verarbeitung

von Anfragen. Ports wie 3000 (Frontend) und 80/443 (HTTP/HTTPS) werden weitergeleitet.

- **ML-Server:** Dieser Dienst führt die Machine-Learning-Modelle aus und kommuniziert mit der Datenbank. Er stellt API-Endpunkte für das Framework bereit.
- **Datenbank (MongoDB):** Die Datenbank speichert Fingerprints und zugehörige Informationen. Ein benanntes Volume (`mongo-data`) stellt sicher, dass Daten auch nach einem Neustart der Container erhalten bleiben.
- **Jupyter Notebook:** Dieser Dienst wird im Experiment-Setup verwendet, um Modelle zu erstellen und zu trainieren. Er verbindet sich mit der MongoDB-Datenbank, um Trainingsdaten abzurufen und Ergebnisse zu speichern.

Die containerisierte Architektur ermöglicht es flexibel auf wachsende Anforderungen zu reagieren und eine konsistente Umgebung für Entwicklung und Produktion bereitzustellen. Zudem könnte das System mit Orchestrierungstools wie Docker Swarm[55] oder Kubernetes[56] erweitert werden, um eine automatische Skalierung und effiziente Ressourcenverwaltung zu gewährleisten.

C. Systemkomponenten

Im vorherigen Kapitel wurde die Systemarchitektur des Projekts grob beschrieben. In diesem Kapitel wird detailliert auf die einzelnen Hauptkomponenten eingegangen, um deren Funktion, Technologien und Interaktionen im Gesamtsystem zu erläutern.

1) Frontend

Das Frontend bildet die Benutzeroberfläche des Systems und ermöglicht die Interaktion mit den Benutzern. Es wurde mit Webtechnologien wie HTML, CSS und JavaScript entwickelt, wobei JavaScript eine zentrale Rolle bei der Erfassung von Fingerprints spielt.

a) Funktion

Das Frontend stellt eine intuitive Benutzeroberfläche bereit, über die Benutzer Fingerprints erstellen und verifizieren können. Die Interaktivität wird hauptsächlich durch JavaScript ermöglicht, das Anfragen an das Backend sendet und Antworten verarbeitet.

b) Wichtige Funktionen

Die folgenden JavaScript-Dateien sind entscheidend für die Funktionalität des Frontends:

- **fingerprint.js[10]:** Diese Datei enthält Funktionen zur Erfassung verschiedener Eigenschaften des Browsers und Geräts, wie User-Agent, IP-Adresse, Bildschirmauflösung und unterstützte Technologien. Diese Informationen werden verwendet, um einen eindeutigen Fingerabdruck zu erstellen.
- **canvas.js[10]:** Hier wird eine Funktion zur Erstellung eines Canvas-Fingerabdrucks definiert. Ein HTML5-Canvas-Element wird erstellt, Text darauf gerendert und das resultierende Bild als Data-URL extrahiert. Diese Technik nutzt die Unterschiede in der Darstellung auf verschiedenen Geräten.
- **index.js[10]:** Diese Datei orchestriert die Hauptlogik der Anwendung. Sie importiert Funktionen aus den anderen

Dateien, um den Fingerabdruck zu generieren, an den Server zu senden und die Benutzeroberfläche zu aktualisieren. Wichtige Funktionen umfassen:

- Generierung und Hashing des Fingerabdrucks.
- Senden des Fingerabdrucks an den Server zur Verifizierung.
- Aktualisierung von Fortschrittsbalken und Terminal-Logs.
- Vorhersage des Fingerabdrucks basierend auf Serverantworten.

2) Logik des Frontends

Die Hauptlogik des Frontends wird in der Datei `index.js` implementiert. Der folgende Graph (Abbildung 5) zeigt die vereinfachte Logik des Frontends:

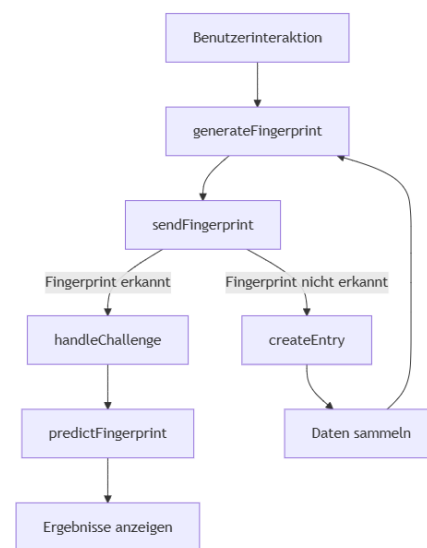


Abbildung 5: Vereinfachte Übersicht über die Hauptlogik des Frontends

Die Abbildung zeigt die grundlegenden Schritte:

- Der Benutzer startet den Prozess zur Fingerprint-Erstellung, in dem er die Webseite betritt.
- Fingerabdrücke werden generiert und an den Server gesendet.
- Je nach Serverantwort wird entweder eine Herausforderung gestartet oder ein neues Modell erstellt.
- Falls weitere Daten benötigt werden, wird der Prozess iterativ wiederholt.

Zusammenfassend arbeiten diese Dateien zusammen, um einen umfassenden Fingerabdruck des Geräts zu erstellen, ihn an den Server zu senden und die Ergebnisse in der Benutzeroberfläche darzustellen. Die Hauptlogik in `index.js` sorgt dafür, dass alle Schritte korrekt orchestriert werden.

3) Backend

Das Backend bildet die zentrale Schnittstelle zwischen dem Frontend, den Machine-Learning-Komponenten und der

Datenbank. Es wurde mit Node.js entwickelt und verwendet Express.js als Framework.

a) Funktion

Das Backend verarbeitet Anfragen vom Frontend, führt notwendige Logik aus und leitet Anfragen an die Machine-Learning-Komponenten oder die Datenbank weiter. Es ermöglicht die Speicherung von Fingerabdruckdaten, die Erstellung von Modellen sowie die Durchführung von Vorhersagen. Zudem verwaltet es den Fortschritt und stellt Statistiken bereit.

b) API-Endpunkte

Das Backend bietet mehrere API-Endpunkte, um verschiedene Funktionen bereitzustellen:

• Fingerabdruck-Verarbeitung:

- `POST /fingerprints`: Speichert Fingerabdruckdaten und aktualisiert den Fortschritt.
- `GET /progress`: Gibt den Fortschritt basierend auf dem Fingerabdruck-Hash zurück.

• Modellstatus und Erstellung:

- `GET /model-status`: Überprüft, ob Modelle für einen Benutzer existieren.
- `POST /create-model`: Erstellt Modelle durch Ausführen eines Python-Skripts.

• Herausforderungen und Vorhersagen:

- `POST /verify-challenge`: Überprüft Herausforderungen durch Kommunikation mit dem ML-Server.
- `POST /predict`: Führt Vorhersagen basierend auf Fingerabdruckdaten durch.

c) Technologien

- **Node.js[54]**: Serverseitige Laufzeitumgebung für JavaScript.
- **Express.js[53]**: Framework zur Erstellung von API-Endpunkten.
- **Mongoose[57]**: Bibliothek zur Interaktion mit MongoDB.

D. Zusammenarbeit mit anderen Komponenten

Das Backend kommuniziert mit dem ML-Server (z. B. über Flask) für Vorhersagen und Modelltraining sowie mit MongoDB zur Speicherung von Fingerabdruckdaten. Es stellt sicher, dass alle Komponenten des Systems effizient miteinander interagieren.

1) Backend-Logik

Die folgende Abbildung (Abbildung 6) zeigt die grundlegenden Interaktionen im Backend: Die Abbildung verdeutlicht die wichtigsten Interaktionen:

- Das Frontend sendet Anfragen an das Backend.
- Das Backend speichert und ruft Fingerprints aus der MongoDB ab.
- Es kommuniziert mit dem ML-Server für Modellstatusprüfungen und Vorhersagen.
- Ein Python-Skript wird zur Modellerstellung verwendet, wobei es Trainingsdaten aus der MongoDB lädt und das erstellte Modell dort speichert.
- Das Backend sendet Ergebnisse und Fortschrittsupdates zurück an das Frontend.

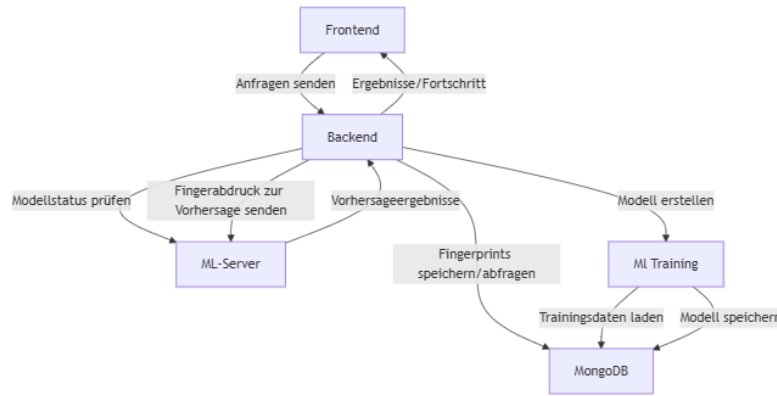


Abbildung 6: Übersicht über die Hauptinteraktionen im Backend

Diese Struktur ermöglicht eine effiziente Verarbeitung von Fingerabdruckdaten und Vorhersagen, während sie gleichzeitig die Skalierbarkeit und Modularität des Systems sicherstellt.

2) Machine-Learning-Komponenten

Die Machine-Learning-Komponenten sind für die Analyse, Klassifikation und Vorhersage der Fingerprints verantwortlich. Sie bestehen aus zwei Hauptteilen: dem ML-Server für Vorhersagen und dem Trainingsmodul für die Modellerstellung.

3) ML-Server (`ml_server.py[10]`)

Der ML-Server ist ein Flask-basierter Webservice, der folgende Funktionen bereitstellt:

- Empfang von Fingerabdruckdaten über den Endpunkt `POST /predict`
- Dekodierung und Vorverarbeitung der Bilddaten
- Durchführung von Vorhersagen mit mehreren geladenen Modellen
- Berechnung des Durchschnitts und der Mehrheitsentscheidung der Vorhersagen
- Rückgabe der Ergebnisse an das Backend

4) Trainingsmodul (`train_model.py[10]`)

Das Trainingsmodul ist für die Erstellung und das Training der Machine-Learning-Modelle zuständig:

- Verbindung zur MongoDB zur Abfrage von Trainingsdaten
- Datenvorbereitung und Aufteilung in Trainings- und Testsets
- Erstellung und Training mehrerer Modellvarianten
- Fortschrittsverfolgung und Logging über SocketIO
- Speicherung der trainierten Modelle im Dateisystem

5) Technologien

- **Python**: Hauptprogrammiersprache für ML-Komponenten
- **TensorFlow**: Framework für das Training und die Ausführung der Modelle
- **Flask**: Webframework für den ML-Server

- **NumPy**: Bibliothek für numerische Berechnungen und Datenverarbeitung
- **Scikit-learn**: Unterstützende Bibliothek für Datenvorverarbeitung

Die folgende Abbildung (Abbildung 7) zeigt die Interaktionen der ML-Komponenten: Der Graph verdeutlicht den Datenfluss

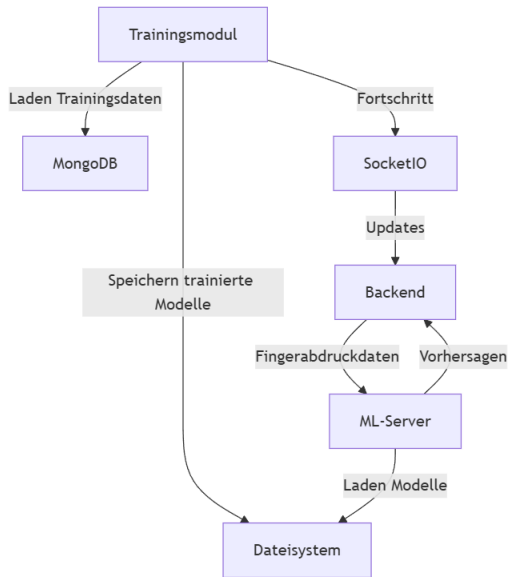


Abbildung 7: Übersicht über die ML-Komponenten und ihre Interaktionen

und die Interaktionen zwischen den ML-Komponenten, dem Backend und der Datenbank. Der ML-Server empfängt Anfragen vom Backend, lädt die Modelle aus dem Dateisystem, führt Vorhersagen durch und sendet die Ergebnisse zurück. Das Trainingsmodul greift auf die MongoDB zu, um Trainingsdaten zu laden, trainiert die Modelle und speichert sie im Dateisystem für spätere Verwendung durch den ML-Server. Der Trainingsfortschritt wird über SocketIO an das Backend gesendet.

E. Experiment Setup der Machine-Learning-Komponenten

Dieses Kapitel beschreibt den spezifischen Teil des Projekts, der für das Training und die Bereitstellung der Machine-Learning-Modelle verantwortlich ist. Dieser Bereich wurde als separates Modul entwickelt, um eine flexible Weiterentwicklung und Wiederverwendbarkeit der trainierten Modelle zu ermöglichen.

1) Architektur

Das Training erfolgt in einer containerisierten Umgebung, die für rechenintensive Aufgaben optimiert wurde:

- Ein dedizierter **Jupyter Notebook Server**, der TensorFlow und andere ML-Bibliotheken nutzt.
- Direkte Verbindung zur MongoDB-Datenbank für effizienten Datenzugriff.

- Separate Speicherung der trainierten Modelle im Dateisystem.

2) Datenfluss und Trainingsprozess

Der Trainingsprozess umfasst folgende Schritte:

1) Datenaufbereitung:

- Laden der Daten aus MongoDB-Collections (cavassamples und fingerprints).
- Dekodierung der Base64-codierten Bilder und Normalisierung.
- Aufteilung in Trainings-, Validierungs- und Testdatensätze.

2) Modelltraining:

Verschiedene Architekturen werden trainiert, darunter:

- Convolutional Neural Networks (CNNs)
- Autoencoder
- Isolation Forest
- Weitere Architekturen je nach Bedarf

3) Evaluierung:

Bewertung der Modelle anhand von Metriken wie Genauigkeit und F1-Score.

4) Speicherung:

Die besten Modelle werden im Dateisystem gespeichert.

3) Technologien

- **Jupyter Notebooks**: Für interaktive Entwicklung und Dokumentation.
- **TensorFlow**: Hauptframework für die Implementierung der ML-Modelle.
- **Pandas**: Für Datenmanipulation und -analyse.
- **Scikit-learn**: Für zusätzliche ML-Algorithmen und Metriken.
- **Docker**: Zur Containerisierung der Trainingsumgebung.

4) Unabhängigkeit und Flexibilität

Die Trennung des ML-Trainingsmoduls vom Hauptsystem ermöglicht:

- Kontinuierliche Verbesserung und Optimierung der Modelle.
- Experimentieren mit neuen Architekturen ohne Beeinträchtigung des laufenden Systems.
- Einfache Integration neuer Modelle in die Produktionsumgebung.

Die folgende Abbildung (Abbildung 8) zeigt den Datenfluss und die Komponenten des ML-Trainingsmoduls:

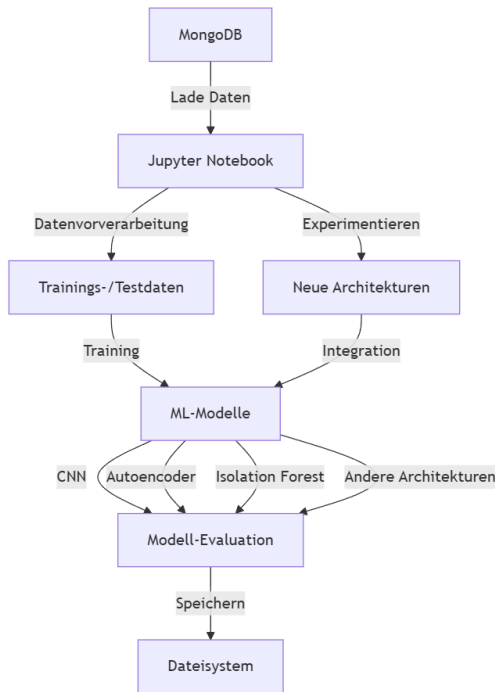


Abbildung 8: Datenfluss und Komponenten des ML-Trainingsmoduls

5) Hardware und Leistung

Das Projekt *IDentify*[10] nutzt zwei unterschiedliche Hardware-Setups, die jeweils spezifische Aufgaben erfüllen. Diese Trennung ermöglicht eine einfache Entwicklung des Frameworks sowie das Training der Machine-Learning-Modelle.

6) Hardware-Spezifikationen

Komponente	Spezifikation
Lokales System	
CPU	AMD Ryzen 7 2700X
RAM	32 GB
GPU	NVIDIA RTX 2080
OS	Windows 10
Server 1	
CPU	Intel Core i9-14900K
RAM	125 GB
GPU	2x NVIDIA RTX A6000
OS	Ubuntu 24.04.1 LTS
Server 2	
CPU	AMD Threadripper PRO 3955WX
RAM	125 GB
GPU	2x NVIDIA RTX A5000
OS	Ubuntu 24.04.1 LTS

Tabelle I: Hardware-Spezifikationen

Lokales System → Framework-Entwicklung

Server-Infrastruktur → ML-Modell-Training

Abbildung 9: Einsatzbereiche der Hardware-Systeme

7) Einsatzbereiche der Hardware

a) Lokales System für Framework-Entwicklung

Das lokale System wurde für die Entwicklung und Implementierung des Frameworks zur Datensammlung und Fingerabdruckbewertung genutzt. Es bot eine ideale Umgebung für schnelle Iterationen und einfache Anpassungen während der Entwicklungsphase.

b) Server-Infrastruktur für Machine-Learning-Training

Die Server wurden für das Training der Modelle genutzt. Dank ihrer leistungsstarken GPUs war es möglich, große Datensätze effizient zu verarbeiten und komplexe Modelle zu optimieren, was auf einem lokalen System entweder nicht möglich gewesen wäre oder deutlich mehr Zeit in Anspruch genommen hätte. Zunächst wurde ein Intel-basiertes System verwendet, das später durch ein AMD-System ersetzt wurde, da das ursprüngliche System anderweitig benötigt wurde innerhalb des Unternehmens[58]. Die parallele Verarbeitungsfähigkeit der GPUs und der großzügige Arbeitsspeicher ermöglichten eine wesentlich schnellere und effizientere Handhabung der komplexen Modelle.

c) Zusammenfassung

Die Kombination aus einem lokalen Entwicklungsrechner und leistungsstarken Servern ermöglichte eine klare Trennung zwischen der Entwicklung des Frameworks und dem Training der Modelle. Während das lokale System für die Entwicklung und das Training einfacher Modelle genutzt wurde, kamen die Server mit ihren leistungsstarken NVIDIA-GPUs für rechenintensive Trainingsprozesse großer Modelle zum Einsatz. Dies beschleunigte nicht nur das Training komplexer Modelle erheblich, sondern erleichterte auch die Durchführung von Experimenten, da die schnellere Verarbeitung und der größere verfügbare Grafikkartenspeicher mehr Spielraum für Tests boten. Dadurch konnte die Effizienz des gesamten Projekts gesteigert werden, indem rechenintensive Prozesse auf die leistungstärkere Hardware ausgelagert wurden, während einfache Tests lokal unkompliziert durchgeführt werden konnten.

d) Projekt und Fortschritt

Das Projekt *IDentify* ist vollständig auf GitHub[10] verfügbar, was eine Dokumentation und Nachvollziehbarkeit ermöglicht. Das öffentliche Repository[10] bietet Einblicke in die Entwicklungshistorie, den Quellcode und die gesamte Projektstruktur. Interessierte können den aktuellen Entwicklungsfortschritt einsehen, was das Projekt Transparent macht.

F. Ansätze

In den folgenden zwei Kapiteln werden zwei unterschiedliche Ansätze zur Implementierung dieses Projekts vorgestellt. Beide Ansätze sind für diese Arbeit relevant, wobei der zweite

Ansatz den Schwerpunkt auf die Weiterentwicklung legt. Der Ansatz 1 wurde in diesem Fall dokumentiert auf Basis des Praxissemesters.

VI. ANSATZ I

Im Rahmen eines vorangegangenen Praxissemesters wurde bereits die Basis für die Implementierung des Canvas Fingerprintings gelegt und erste Experimente mit kleineren Datensätzen durchgeführt. Die dort erzielten Ergebnisse dienen als Ausgangspunkt für weiterführende Untersuchungen. Im zweiten Teil dieser Arbeit wird erörtert, ob und wie sich diese Technik auch bei größeren Datenmengen und einer breiteren Nutzerbasis anwenden lässt. Auf diese Weise soll überprüft werden, ob die im Praxissemester gewonnenen Erkenntnisse und Methoden in einem skalierbaren Umfeld reproduzierbar sind.

A. Bisheriger Ansatz

Der bisherige Ansatz umfasst die Entwicklung von sechs unterschiedlichen Modellarchitekturen, die auf Convolutional Neural Networks (CNNs) basieren. Diese Modelle wurden speziell für die Analyse und Klassifikation von Canvas Fingerprints entwickelt. Jedes Modell verfolgt einen anderen architektonischen Ansatz, inspiriert von bekannten CNN-Architekturen wie VGG[59] und ResNet[60]. Ziel war es, durch Diversität in den Modellen eine robuste Klassifikation zu gewährleisten.

Im Rahmen dieses bisherigen Ansatzes wurden Daten von insgesamt fünf verschiedenen Geräten erhoben, was insgesamt 1.900 Fingerabdrücke (Samples) umfasste im Gegensatz zu dem aktuellen Ansatz. Diese wurden aufbereitet, um frühe Erkenntnisse über die Stabilität und Wiedererkennbarkeit der Fingerabdruckdaten zu gewinnen. Dies lieferte die Grundlage für die anschließende Entwicklung, Optimierung und Validierung der verschiedenen Modellarchitekturen.

B. Modelle

Im Folgenden werden die entwickelten Modellarchitekturen vorgestellt, die für das Canvas Fingerprinting eingesetzt wurden, des bisherigen Ansatzes. Diese Modelle stellen den Kern der Untersuchung dar und wurden speziell für diese Aufgabe getestet.

1) Model 1

- **Conv2D:** 32 Filter, Kernelgröße (3, 3), Aktivierungsfunktion ReLU
- **MaxPooling2D:** Pooling-Schicht mit einer Poolinggröße von (2, 2)
- **Flatten:** Wandelt die mehrdimensionalen Feature-Maps in einen flachen Vektor um
- **Dense:** Vollständig verbundene Schicht mit 128 Neuronen und der Aktivierungsfunktion ReLU
- **Dense:** Vollständig verbundene Schicht mit 1 Neuron und der Aktivierungsfunktion Sigmoid (geeignet für binäre Klassifikationsaufgaben)

Für Model 1 haben wir eine einfache CNN-Architektur gewählt. Sie besteht aus einer Convolutional-Schicht (32 Filter,

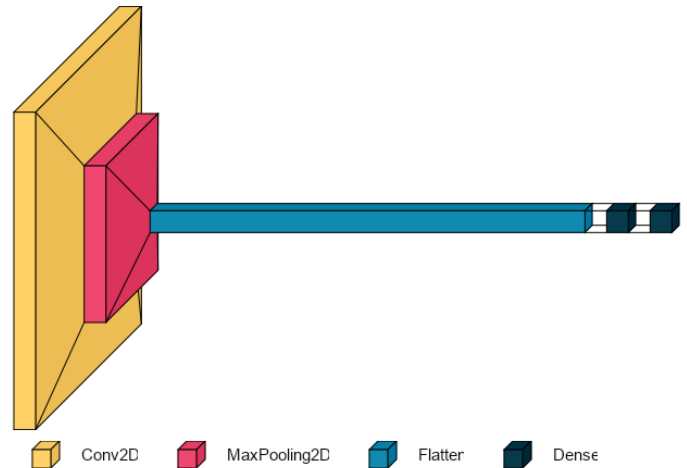


Abbildung 10: Architektur von Model 1

3x3 Kernel), gefolgt von Max-Pooling (2x2), einer Flatten-Schicht und zwei Dense-Schichten (128 Neuronen mit ReLU, 1 Neuron mit Sigmoid). Diese Struktur dient als Basislinie für unsere Bildklassifizierungsaufgabe und ermöglicht grundlegende Merkmalsextraktion. Inspiriert wurde sie von frühen CNN-Architekturen, die sich bei einfachen Bildererkennungsaufgaben bewährt haben.

2) Model 2

- **Conv2D:** 64 Filter (statt 32), Kernelgröße (3, 3), Aktivierungsfunktion ReLU
- **MaxPooling2D:** Pooling-Schicht mit einer Poolinggröße von (2, 2)
- **Flatten:** Wandelt die mehrdimensionalen Feature-Maps in einen flachen Vektor um
- **Dense:** Vollständig verbundene Schicht mit 256 Neuronen (statt 128) und der Aktivierungsfunktion ReLU
- **Dense:** Vollständig verbundene Schicht mit 1 Neuron und der Aktivierungsfunktion Sigmoid (geeignet für binäre Klassifikationsaufgaben)

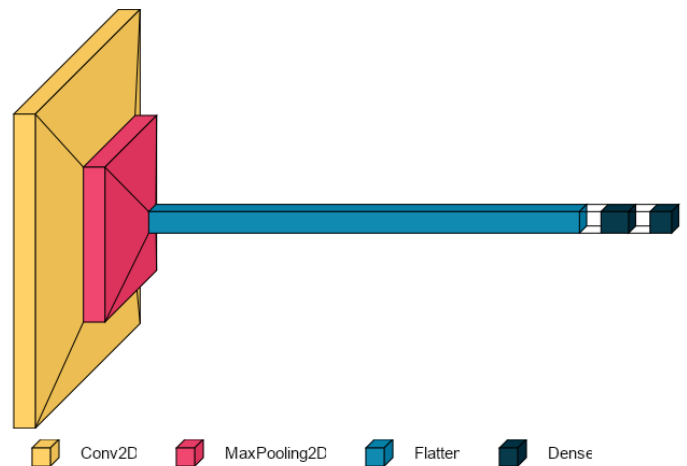


Abbildung 11: Architektur von Model 2

Model 2 erhöht die Komplexität von Model 1 durch die

Erhöhung der Anzahl der Filter und der Größe der Denseschicht. Diese Architektur bietet mehr Kapazität zur Merkmalsextraktion und kann feinere Details in den Eingabedaten erfassen. Es ist ein Schritt in Richtung tieferer Netzwerke und zeigt, wie zusätzliche Filter und Neuronen die Modellleistung verbessern können.

3) Model 3

- **Conv2D:** 64 Filter, Kernelgröße (3, 3), Aktivierungsfunktion ReLU
- **MaxPooling2D:** Pooling-Schicht mit einer Poolinggröße von (2, 2)
- **Conv2D: 128 Filter** (zusätzliche Schicht), Kernelgröße (3, 3), Aktivierungsfunktion ReLU
- **MaxPooling2D: Zweite Pooling-Schicht** mit einer Poolinggröße von (2, 2)
- **Flatten:** Wandelt die mehrdimensionalen Feature-Maps in einen flachen Vektor um
- **Dense:** Vollständig verbundene Schicht mit 256 Neuronen und der Aktivierungsfunktion ReLU
- **Dense:** Vollständig verbundene Schicht mit 1 Neuron und der Aktivierungsfunktion Sigmoid (geeignet für binäre Klassifikationsaufgaben)

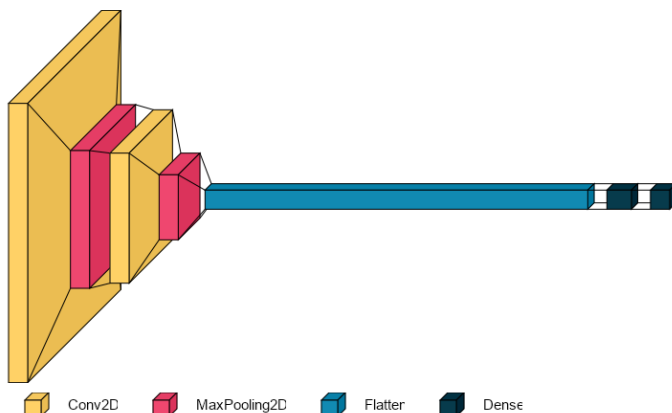


Abbildung 12: Architektur von Model 3

Model 3 erweitert die Architektur durch eine zusätzliche Convolutional-Schicht, wodurch es besser in der Lage ist, komplexere Merkmale aus den Eingabedaten zu extrahieren. Die Struktur orientiert sich an der VGG-Architektur[61], die für ihre Einfachheit und Tiefe bekannt ist. Diese nutzt mehrere Convolutional-Schichten, um tiefere Merkmalsrepräsentationen zu lernen. Durch diese Eigenschaften eignet sich Model 3 besonders gut, um die Auswirkungen zusätzlicher Schichten auf die Leistungsfähigkeit des Modells zu untersuchen.

4) Model 4

- **Conv2D: 32 Filter** (statt 64), Kernelgröße (3, 3), Aktivierungsfunktion ReLU
- **Conv2D: Zweite Conv2D-Schicht mit 32 Filtern** (zusätzliche Schicht), Kernelgröße (3, 3), Aktivierungsfunktion ReLU
- **MaxPooling2D:** Pooling-Schicht mit einer Poolinggröße von (2, 2)

- **Conv2D:** 64 Filter, Kernelgröße (3, 3), Aktivierungsfunktion ReLU
- **Conv2D: Zweite Conv2D-Schicht mit 64 Filtern** (zusätzliche Schicht), Kernelgröße (3, 3), Aktivierungsfunktion ReLU
- **MaxPooling2D:** Zweite Pooling-Schicht mit einer Poolinggröße von (2, 2)
- **Flatten:** Wandelt die mehrdimensionalen Feature-Maps in einen flachen Vektor um
- **Dense:** Vollständig verbundene Schicht mit **512 Neuronen** (statt 256) und der Aktivierungsfunktion ReLU
- **Dense:** Vollständig verbundene Schicht mit 1 Neuron und der Aktivierungsfunktion Sigmoid (geeignet für binäre Klassifikationsaufgaben)

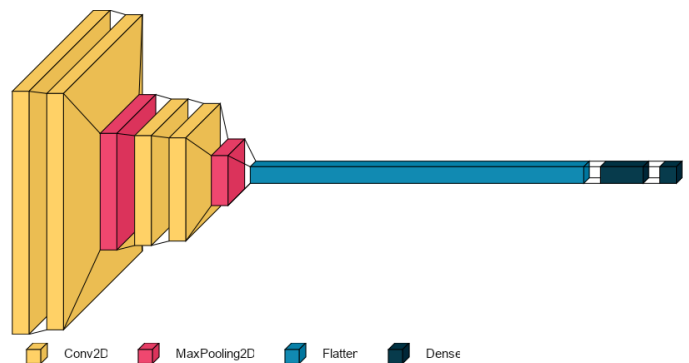


Abbildung 13: Architektur von Model 4

Model 4 verwendet mehrere Convolutional-Schichten mit Max-Pooling und ist inspiriert von tieferen Architekturen wie VGG16[62]. Diese Architektur nutzt mehrere Schichten, um komplexere Merkmale zu extrahieren und hat eine größere Dense-Schicht, um die extrahierten Merkmale zu verarbeiten. VGG16[62] ist bekannt für seine Fähigkeit, feine Details in Bildern zu erfassen, was es effektiv für die Fingerprinting-Aufgabe macht.

5) Model 5

- **Conv2D: 64 Filter mit einer Kernelgröße von (7, 7)** (statt 32 Filter mit (3, 3)), Aktivierungsfunktion ReLU
- **MaxPooling2D:** Pooling-Schicht mit einer Poolinggröße von (2, 2)
- **Conv2D:** 128 Filter, Kernelgröße (3, 3), Aktivierungsfunktion ReLU
- **MaxPooling2D:** Zweite Pooling-Schicht mit einer Poolinggröße von (2, 2)
- **Conv2D: 256 Filter** (zusätzliche Schicht), Kernelgröße (3, 3), Aktivierungsfunktion ReLU
- **MaxPooling2D:** Dritte Pooling-Schicht mit einer Poolinggröße von (2, 2)
- **Flatten:** Wandelt die mehrdimensionalen Feature-Maps in einen flachen Vektor um
- **Dense:** Vollständig verbundene Schicht mit 512 Neuronen und der Aktivierungsfunktion ReLU
- **Dense:** Vollständig verbundene Schicht mit 1 Neuron und der Aktivierungsfunktion Sigmoid (geeignet für binäre Klassifikationsaufgaben)

Klassifikationsaufgaben)

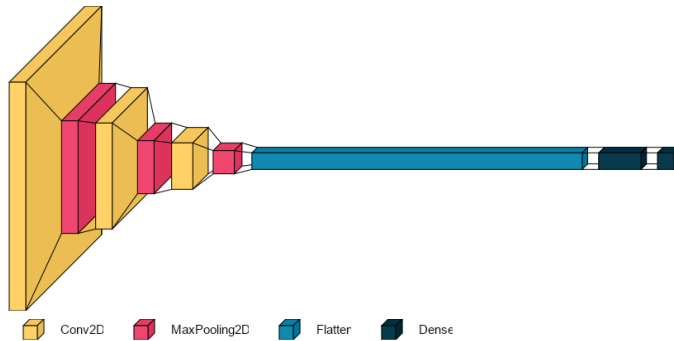


Abbildung 14: Architektur von Model 5

Model 5 verwendet eine tiefere Struktur mit mehreren Schichten, um komplexere Merkmale zu analysieren. Es kombiniert größere Filter in der ersten Schicht mit einer zunehmenden Anzahl an Filtern in den folgenden Schichten und nutzt Max-Pooling zur Datenreduktion. Ohne zusätzliche Residual-Verbindungen, wie sie in ResNet[60] zu finden sind, konzentriert sich dieses Modell auf die Extraktion detaillierter Merkmale. Ziel ist es, die Leistung bei der Geräteerkennung zu verbessern, indem feinere und abstraktere Merkmale identifiziert werden.

6) Model 6

- **Conv2D:** 32 Filter mit einer Kernelgröße von (3, 3) (statt 64 Filter mit (7, 7)), Aktivierungsfunktion ReLU
- **MaxPooling2D:** Pooling-Schicht mit einer Poolinggröße von (2, 2)
- **Conv2D:** 64 Filter, Kernelgröße (3, 3), Aktivierungsfunktion ReLU
- **MaxPooling2D:** Zweite Pooling-Schicht mit einer Poolinggröße von (2, 2)
- **Conv2D:** 128 Filter, Kernelgröße (3, 3), Aktivierungsfunktion ReLU
- **MaxPooling2D:** Dritte Pooling-Schicht mit einer Poolinggröße von (2, 2)
- **Flatten:** Wandelt die mehrdimensionalen Feature-Maps in einen flachen Vektor um
- **Dense:** Vollständig verbundene Schicht mit **256 Neuronen** (statt 512) und der Aktivierungsfunktion ReLU
- **Dense:** Vollständig verbundene Schicht mit 1 Neuron und der Aktivierungsfunktion Sigmoid (geeignet für binäre Klassifikationsaufgaben)

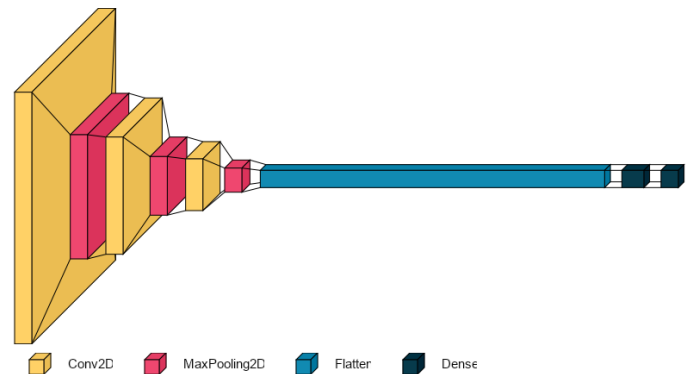


Abbildung 15: Architektur von Model 6

Model 6 kombiniert mehrere Convolutional-Schichten mit Max-Pooling und ist inspiriert von Architekturen wie ResNet[60]. Es zielt darauf ab, eine Balance zwischen Tiefe und Komplexität zu erreichen. Inception-Architekturen sind bekannt für ihre Fähigkeit, verschiedene Filtergrößen in einem einzigen Netzwerk zu kombinieren, was die Flexibilität und Leistungsfähigkeit des Modells erhöht.

Diese sechs Modelle wurden so ausgewählt, dass sie eine zunehmende Komplexität aufweisen. Ziel ist es, zu analysieren, ab welcher Tiefe oder Größe die beste Performance erzielt werden kann. Dies soll dabei helfen, zu verstehen, wie verschiedene Architekturen auf dieselbe Aufgabe reagieren und welche Architektur sich am besten für die spezifischen Anforderungen eignet.

C. Mehrheitsabstimmung

Es wurde sich zudem noch dazu entschieden die Modelle so zu nutzen, dass sie gemeinsam Abstimmen. Bei der Mehrheitsabstimmung wird die Vorhersage jedes Modells berücksichtigt, wobei die endgültige Entscheidung auf der Mehrheit der Vorhersagen basiert. Ein Fingerprint gilt als korrekt klassifiziert, wenn mindestens vier der sechs Modelle übereinstimmen. Dieser Ansatz zielt darauf ab, die Wahrscheinlichkeit von Fehlklassifikationen zu reduzieren und die Gesamtrüstbarkeit des Systems zu erhöhen. Durch diese Methode streben wir an, eine stabilere Fingerprinting-Technik zu entwickeln, die besser mit Variabilitäten und Unsicherheiten in den Daten umgehen kann. Allerdings bedarf es noch weiterer Tests und Optimierungen, um die tatsächliche Effektivität in verschiedenen Szenarien zu validieren.

D. Implementierung der Mehrheitsabstimmung

Die Umsetzung der Mehrheitsabstimmung erfolgt durch ein serverseitiges Skript, das die Vorhersagen der sechs Modelle kombiniert. Dieses Skript empfängt Fingerprint-Daten, führt die Vorhersagen durch die Modelle aus und berechnet die Mehrheitsabstimmung. Die Modelle werden aus ihren jeweiligen Dateien geladen und auf die Fingerprint-Daten angewendet, dies findet im Backend statt. Anschließend werden die Vorhersagen der einzelnen Modelle gesammelt und eine

Mehrheitsabstimmung durchgeführt, um die endgültige Klassifikation zu bestimmen, dies wird dann für den Benutzer an das Frontend weitergeleitet.

E. Ergebnisse des bisherigen Ansatzes

Der bisherige Ansatz, der auf Basis des Praxissemesters entstanden ist, hat darauf abgezielt, die grundsätzliche Anwendbarkeit des Canvas Fingerprinting für IDentify zu prüfen. Die Technik wurde erfolgreich implementiert und mit dem begrenzten Datensatz getestet: 1900 Samples pro Nutzer bei insgesamt 5 Nutzern. Die Ergebnisse zeigen, dass die Methode prinzipiell funktioniert und Geräte anhand ihrer Konfigurationen unterscheiden kann. Die unterschiedlichen Leistungen der entwickelten Modelle weisen auf Optimierungspotenzial hin. Diese ersten Erkenntnisse bilden die Basis für die weitere Entwicklung des Canvas Fingerprinting-Ansatzes in dieser Arbeit. Die detaillierten Ergebnisse werden im Folgenden dargestellt. Im zweiten Ansatz wird hingegen geprüft, ob die Technik auch bei größeren und spezifischeren Datensätzen (20 Nutzer, 100.000 Samples) effektiv funktioniert. Diese erweiterte Untersuchung wird im Kapitel *Ansatz 2* ausführlich besprochen.

F. Begriffe und Konzepte

Um die folgenden Ergebnisse besser zu verstehen, werden in den nächsten Abschnitten kurz wichtige Begriffe erläutert.

1) Username

Ein **Username** ist ein im Rahmen unseres Projektes ein eindeutiger Name, der genutzt wird, um die Modelle und Fingerabdrücke der Nutzer zu identifizieren.

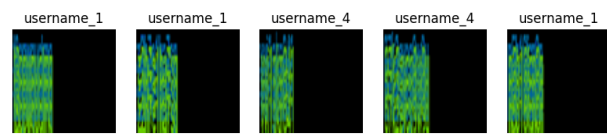
2) Fingerabdruck

Ein **Fingerabdruck** ist im Rahmen des Projektes ein Base64-codierter String, der das Canvas repräsentiert, das von einem Benutzer generiert wurde. Diese kodierten Fingerabdruckbilder werden in den folgenden Tests genutzt, um zu beschreiben, von welchem Benutzer der Fingerabdruck stammt. Diese werden im Rahmen der Arbeit auch Samples genannt.

3) Canvas

Ein **Canvas** ist, wie bereits beschrieben, das Bild, auf dem der Benutzer gezeichnet hat. Dies wird im Folgenden anhand von drei Beispielen illustriert.

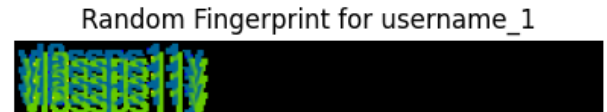
Wie in Abbildung 16a zu sehen ist, wurden hier viele Stichproben mit zufälligen Strings generiert. Diese werden verwendet, um beispielsweise ein Modell zu trainieren oder dessen Performance zu testen. Abbildung 16b zeigt einen einzelnen Canvas, der typischerweise mit einem zufälligen String vom Benutzer generiert und anschließend vom Server verifiziert wird. In Abbildung 16c kann man einen fertigen Canvas erkennen, welcher von einem Nutzer erstellt und mit einem zufälligen String beschrieben wurde. Obwohl diese Canvase oberflächlich fast identisch aussehen, weisen sie dennoch subtile Unterschiede auf, die mit einem ML-Modell erkannt werden können.



(a) Beispiel mehrerer Fingerabdrücke mit zufälligen Strings



(b) Beispiel mehrere Fingerabdruck-Canvase



(c) Beispiel eines Fingerabdruck-Canvases

Abbildung 16: Illustrationen von Fingerabdruck-Canvases

G. Leistung der Modelle

Zunächst wird der erste Testdurchlauf der Modelle beschrieben, bei dem zufällig ausgewählte Fingerabdrücke aus einer MongoDB-Datenbank verwendet wurden. Anschließend werden die Ergebnisse der Modelle grafisch dargestellt, um die Performance der verschiedenen Architekturen zu verdeutlichen und ihre Unterscheidungsfähigkeit zu bewerten.

1) Metriken zur Modellbewertung

Zur Bewertung der Leistung der Modelle werden folgende Kennzahlen herangezogen:

- **Accuracy:** Anteil der korrekt klassifizierten Fälle an der Gesamtzahl der Fälle.
- **Precision:** Anteil der korrekten positiven Vorhersagen an allen als positiv klassifizierten Fällen.
- **Recall:** Anteil der richtig erkannten positiven Fälle an der Gesamtzahl aller tatsächlichen positiven Fälle.
- **F1-Score:** Harmonischer Mittelwert von Precision und Recall, berechnet als

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Die **Konfusionsmatrix** ergänzt diese Metriken, indem sie die Verteilung von True Positives (TP), True Negatives (TN), False Positives (FP) und False Negatives (FN) darstellt. Dies ermöglicht eine detaillierte Analyse der Fehlklassifikationen und hilft dabei, die Unterscheidungsfähigkeit der Modelle, insbesondere hinsichtlich der Erkennung desselben Nutzers über verschiedene Sitzungen hinweg, zu bewerten.

H. Testdurchlauf der Modelle 1

Für den Testdurchlauf wurden die Fingerabdruckbilder der Benutzer 1 und Benutzer 4 verglichen. Zunächst wird eine Verbindung zur MongoDB-Datenbank aufgebaut, um auf die Fingerabdruckdaten zuzugreifen. Die Base64-kodierten Bilder werden anschließend dekodiert und für den Einsatz in

TensorFlow-Modellen vorbereitet. Danach werden die Modelle geladen und auf die Fingerabdrücke beider Benutzer angewendet, um Vorhersagen zu treffen. Abschließend erfolgt die Ausgabe der Ergebnisse, um zu evaluieren, wie gut die Modelle die Fingerabdrücke der jeweiligen Benutzer vorhersagen.

- 1) **Verbindung zur Datenbank:** Herstellung einer Verbindung zur MongoDB-Datenbank, um auf Fingerabdruckdaten zuzugreifen.
- 2) **Dekodieren der Bilder:** Base64-kodierte Fingerabdruckbilder werden dekodiert und in RGB-Bilder umgewandelt.
- 3) **Vorbereiten der Bilder:** Die Bilder werden auf eine einheitliche Größe skaliert.
- 4) **Laden der Modelle:** TensorFlow-Modelle werden aus einem Verzeichnis geladen und kompiliert.
- 5) **Abrufen der Fingerabdrücke:** Zufällige Fingerabdruckbilder von Benutzer 1 und Benutzer 4 werden aus der Datenbank abgerufen.
- 6) **Durchführen der Tests:** Die vorbereiteten Bilder werden durch die Modelle verarbeitet, um Vorhersagen zu treffen.
- 7) **Ausgabe der Ergebnisse:** Die Vorhersagen der Modelle werden ausgegeben und grafisch dargestellt, um die Leistung der Modelle zu analysieren.

In den Graphen tauchen oft die Bezeichnungen *username_x* auf. Dies ist gleichzusetzen mit dem Benutzer X und stellt lediglich eine Art der Speicherung in der Datenbank dar

1) Graphen zur Modellleistung

In den folgenden zwei Graphen wird die Leistung der sechs Modelle für je zwei Benutzer dargestellt: einmal für Benutzer 1 und einmal für Benutzer 4. Hierbei werden die Modelle gegen Fingerabdrücke aus der Datenbank getestet, um die Vorhersagen zu evaluieren und die Performance zu bewerten. Die Graphen zeigen, wie sicher die Modelle in ihrer Einschätzung sind, ob es sich bei den Samples tatsächlich um Benutzer 1 bzw. Benutzer 4 handelt. Diese Darstellung ermöglicht einen direkten Vergleich der Modellgenauigkeit für die beiden ausgewählten Benutzer.

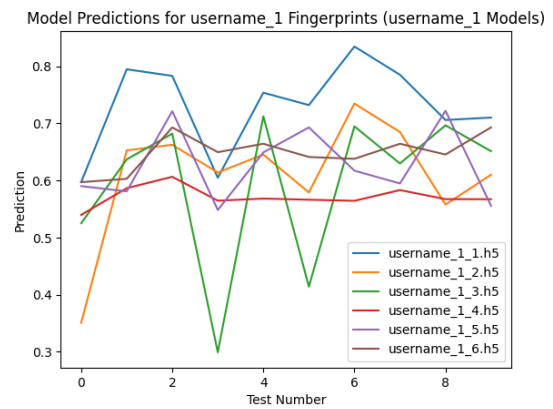


Abbildung 17: Leistung der sechs Modelle von *Benutzer 1* gegen Fingerabdrücke von *Benutzer 1*

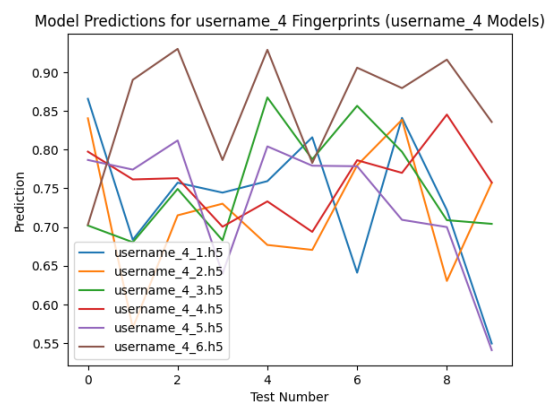


Abbildung 18: Leistung der sechs Modelle von *Benutzer 4* gegen Fingerabdrücke von *Benutzer 4*

Die Graphen 17 und 18 zeigen die Performance der verschiedenen Modellarchitekturen für Benutzer 1 und Benutzer 4. Wie man hier erkennen kann, sind die sechs Modelle, die wir beschrieben haben, jeweils durch eine nachgestellte Zahl im Benutzernamen gekennzeichnet. Die Genauigkeit der Modelle variiert dabei zwischen 0.6 und 0.8, mit einigen Ausreißern nach oben und unten. Für Benutzer 1 zeigt sich eine deutliche Varianz in den Vorhersagen:

- `username-1-1.h5` liefert Vorhersagen zwischen 0.532 und 0.819, was auf eine gewisse Stabilität hindeutet.
- `username-1-3.h5` zeigt eine größere Streuung, was eine inkonsistentere Leistung impliziert.

Bei Benutzer 4 fällt besonders `username-4-6.h5` auf, das konsistent hohe Vorhersagen zwischen 0.593 und 0.914 liefert und damit eine robustere Leistung aufweist. Wie deutlich zu erkennen ist, führt eine höhere Komplexität bei den Modellen nicht zwangsläufig zu besseren Ergebnissen. So schneidet beispielsweise Modell 6 bei Nutzer 4 signifikant besser ab als die übrigen Modelle, jedoch lässt sich insgesamt kein eindeutiger Trend festmachen. Die unterschiedlichen Ergebnisse können mehrere Gründe haben. Im Rahmen dieser Untersuchung ver-

muten wir, dass diese Ergebnisse auf die begrenzte Testdatengröße zurückzuführen sind, da die Performance-Unterschiede zwischen den Modellen äußerst gering sind und die Modellarchitekturen für beide Benutzer gleich sind. Insgesamt verdeutlichen die Graphen, dass die Modelle unterschiedlich gut bei der Klassifikation der Fingerabdrücke der beiden Benutzer performen. Einige Modelle liefern konsistentere und genauere Vorhersagen, während andere Modelle größere Schwankungen in der Leistung aufweisen. Im folgenden Abschnitt gehen wir nun auf die Performance bzw. die Ergebnisse der Modelle ein, die gegen unterschiedliche Nutzer getestet wurden.

2) Graphen zur Unterscheidungsfähigkeit der Modelle

Die folgenden zwei Graphen zeigen die Leistung der sechs Modelle von Benutzer 4 und Benutzer 1 bei der Klassifikation von zufälligen Fingerabdrücken von Benutzer 1 und Benutzer 4. Diese Graphen verdeutlichen, wie gut die Modelle zwischen den Fingerabdrücken der beiden Benutzer unterscheiden können.

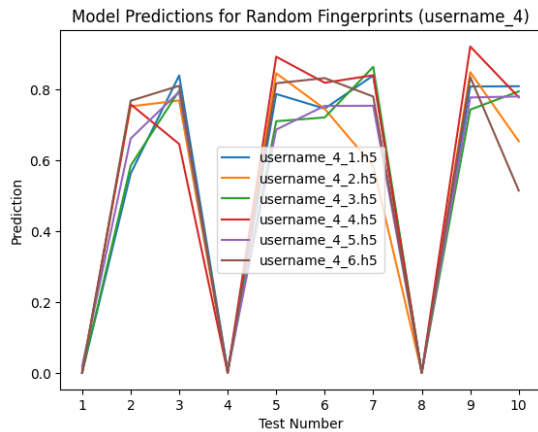


Abbildung 19: Leistung der sechs Modelle von Benutzer 4 gegen zufällige Fingerabdrücke von Benutzer 1 und Benutzer 4

Testdurchläufe für Graph 19:

Testdurchlauf	Benutzer
1	username_1
2	username_4
3	username_4
4	username_1
5	username_4
6	username_4
7	username_4
8	username_1
9	username_4
10	username_4

Tabelle II: Übersicht Testnummer und Benutzer

Testdurchläufe für Graph 20:

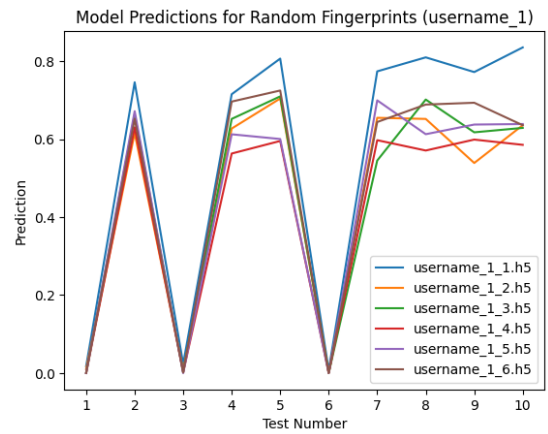


Abbildung 20: Leistung der sechs Modelle von Benutzer 1 gegen zufällige Fingerabdrücke von Benutzer 1 und Benutzer 4

Testnummer	Benutzer
1	username_4
2	username_1
3	username_4
4	username_1
5	username_1
6	username_4
7	username_1
8	username_1
9	username_1
10	username_1

Tabelle III: Übersicht der Testnummern und Benutzer

Die Graphen 19 und 20 zeigen die Reaktionen der Modelle auf verschiedene zufällige Samples von beiden Benutzern. Es wird deutlich, dass die Modelle zwischen den Fingerabdrücken der beiden Benutzer unterscheiden können. Obwohl die Performance hier schwankend ist, möchten wir den Fokus auf die Fähigkeit der Modelle legen, eindeutig zu erkennen, ob ein Fingerabdruck einem bestimmten Benutzer zugeordnet werden kann. Für die Benutzer 4 Modelle ist eine klare Trennung erkennbar. Das Modell Benutzer 4-1.h5 liefert beispielsweise niedrige Werte (0.005) für Samples von Benutzer 1 und höhere Werte (0.6186) für Samples von Benutzer 4. Dies zeigt, dass das Modell Fingerabdrücke von Benutzer 1 sicher als nicht zu Benutzer 4 gehörend erkennt. Ähnliches gilt für die Benutzer 1 Modelle. Das Modell Benutzer 1-1.h5 erzielt niedrige Werte (0.0104) für Samples von Benutzer 4 und höhere Werte (0.7738) für Samples von Benutzer 1.

Diese konsistenten Unterscheidungen über verschiedene zufällige Samples hinweg zeigen, dass das Konzept des Canvas Fingerprintings grundsätzlich funktioniert. Die Ergebnisse deuten darauf hin, dass unsere Modelle Fingerabdrücke effektiv klassifizieren können, trotz der begrenzten Datenmenge. Für eine Bestätigung der Leistungsfähigkeit in praktischen Anwendungen sind jedoch weitere Untersuchungen mit größeren Datensätzen unter realen Bedingungen notwendig.

3) Zusammenfassung der Ergebnisse 1

Die Evaluierung zeigt, dass die Modelle Genauigkeiten zwischen 0,6 und 0,8 erreichen, mit einigen Ausreißern. Sie können die Fingerabdrücke von Benutzer 1 und Benutzer 4 effektiv unterscheiden, wie die Klassifikation zufälliger Fingerabdrücke in den Abbildungen 20 und 19 belegt. Diese Ergebnisse demonstrieren die grundsätzliche Leistungsfähigkeit der Modelle, zeigen aber auch Verbesserungspotenzial. Eine Erweiterung des Datensatzes könnte die Genauigkeit und Zuverlässigkeit weiter steigern, insbesondere bei der Erkennung neuer Fingerabdrücke.

1. Testdurchlauf der Modelle 2

Für den Testdurchlauf 2 wurden zufällig 500 Fingerabdruckbilder aus der MongoDB-Datenbank ausgewählt. Die Bilder wurden dekodiert und für die Modellvorhersage vorbereitet. Die Modelle sollten vorhersagen, ob ein Fingerabdruck zu Benutzer 1 oder Benutzer 4 gehört. Die Labels wurden entsprechend der Benutzernamen erstellt: 1 für Benutzer 1 und 0 für Benutzer 4.

Die Leistung der Modelle wurde anhand der bereits erläuterten Metriken bewertet: Accuracy, Precision, Recall, F1-Score und Konfusionsmatrix.

1) Ergebnisse

Model 1: username_1_1.h5

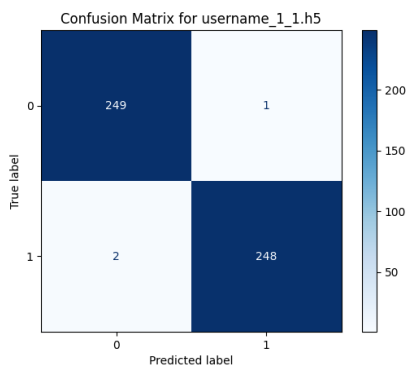


Abbildung 21: Konfusionsmatrix für Model 1

- Accuracy: 0.9820
- Precision: 0.9776
- Recall: 0.9887
- F1 Score: 0.9831

Model 2: username_1_2.h5

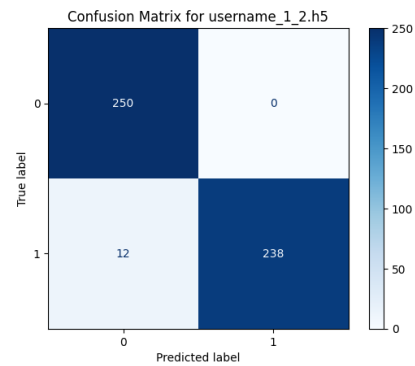


Abbildung 22: Konfusionsmatrix für Model 2

- Accuracy: 0.9680
- Precision: 1.0000
- Recall: 0.9396
- F1 Score: 0.9689

Model 3: username_1_3.h5

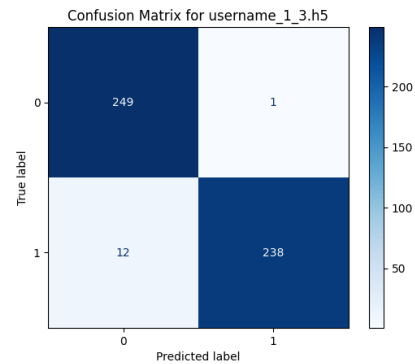


Abbildung 23: Konfusionsmatrix für Model 3

- Accuracy: 0.9660
- Precision: 0.9960
- Recall: 0.9396
- F1 Score: 0.9670

Model 4: username_1_4.h5

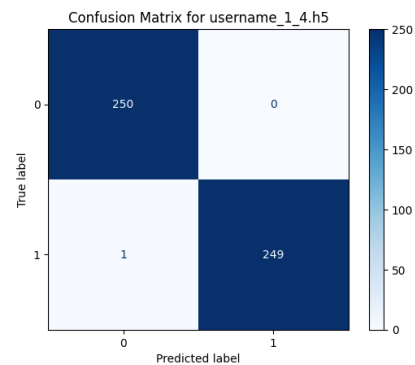


Abbildung 24: Konfusionsmatrix für Model 4

- Accuracy: 0.9900

- Precision: 1.0000
- Recall: 0.9811
- F1 Score: 0.9905

Model 5: username_1_5.h5

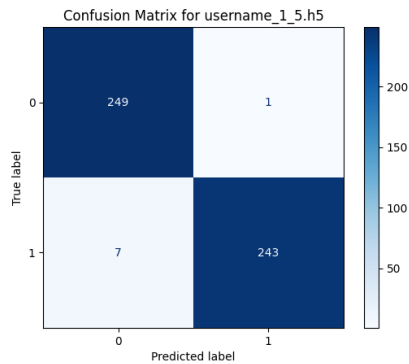


Abbildung 25: Konfusionsmatrix für Model 5

- Accuracy: 0.9820
- Precision: 0.9961
- Recall: 0.9698
- F1 Score: 0.9828

Model 6: username_1_6.h5

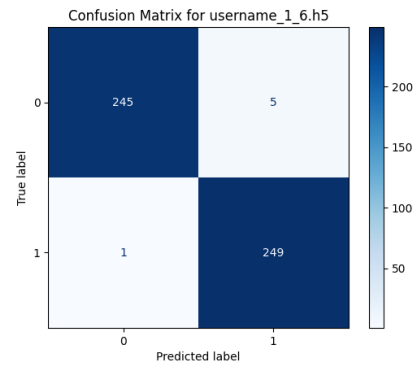


Abbildung 26: Konfusionsmatrix für Model 6

- Accuracy: 0.9980
- Precision: 0.9962
- Recall: 1.0000
- F1 Score: 0.9981

Die Ergebnisse zeigen, dass die Modelle eine hohe Genauigkeit und Präzision bei der Unterscheidung zwischen verschiedenen Nutzern aufweisen. Die besten Ergebnisse wurden mit dem Modell username_1_6.h5 erzielt, das eine Accuracy von 0.9980 und eine F1 Score von 0.9981 aufweist.

2) Grafische Darstellung der Ergebnisse

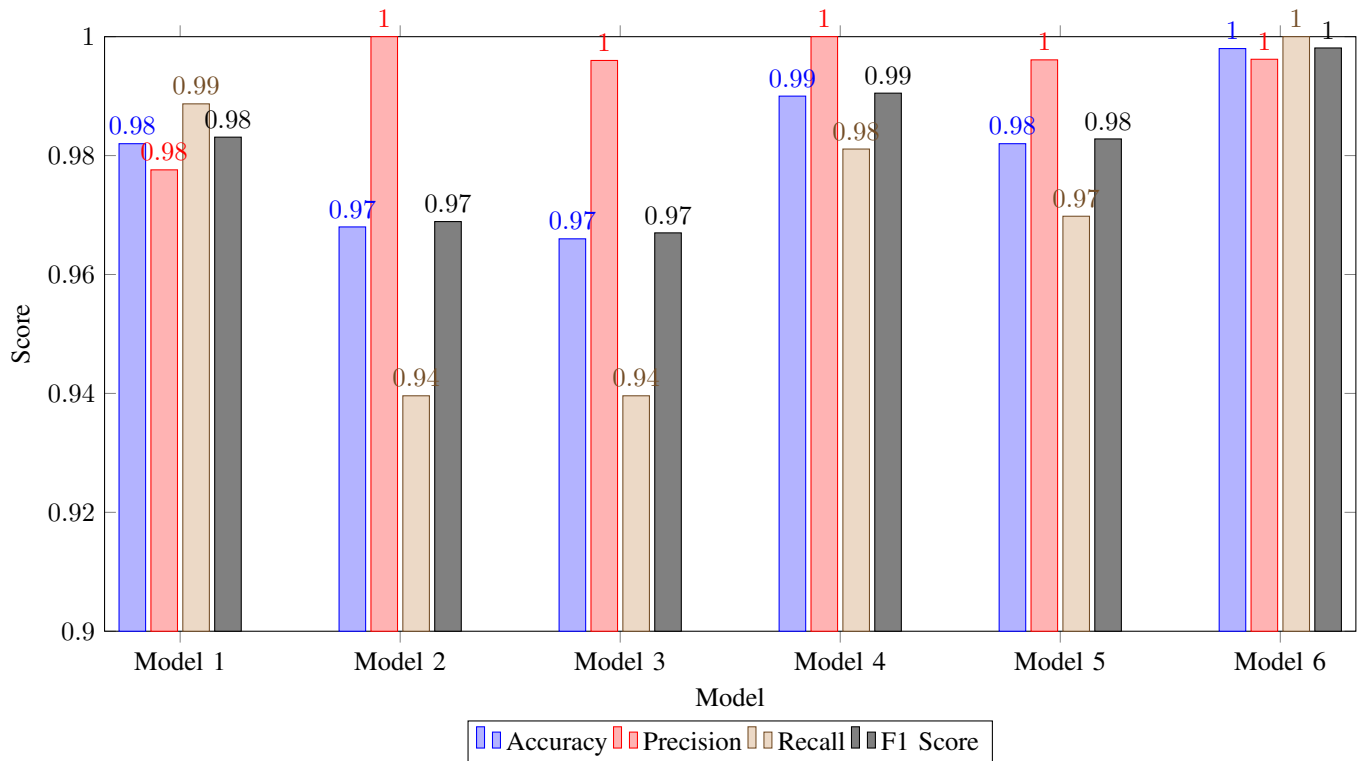


Abbildung 27: Vergleich der Metriken für die Modelle

3) Zusammenfassung der Ergebnisse 2

In den Testdurchläufen der Modelle konnte festgestellt werden, dass diese effektiv die aus den Testdaten erstellten Fingerprints erkennen. Das Modell `username_1_6.h5` erzielte die besten Ergebnisse mit einer Accuracy von 0.9980 und einem F1-Score von 0.9981. Dies zeigt, dass die in den vorherigen Kapiteln beschriebene Architektur 6 durch ihre Tiefe und mehrere Schichten besser verschiedene Muster erkennen kann. Dieses Wissen haben wir in Ansatz 2 genutzt, um ähnliche Ergebnisse zu erzielen. Insgesamt zeigten die Modelle eine hohe Genauigkeit und Präzision bei der Unterscheidung zwischen verschiedenen Nutzern. Die grafische Darstellung der Ergebnisse zeigt die Unterschiede in der Performance der Modelle. Einige Modelle lieferten konsistentere und genauere Vorhersagen, während andere größere Schwankungen aufwiesen.

Zusammenfassend zeigen die Ergebnisse, dass die Modelle bereits eine hohe Leistungsfähigkeit bei der Fingerabdruckerkennung aufweisen, wie bereits in den vorherigen Abschnitten hervorgehoben wurde.

J. Gesamtbewertung

Die Ergebnisse zeigen, dass für die Modelle Optimierungspotenzial besteht. Trotz der begrenzten Daten war es trotzdem möglich, klar zwischen verschiedenen Benutzern zu unterscheiden. In dem folgenden Kapitel wird der zweite Teil der praktischen Implementierung besprochen und dargestellt. Dieser Abschnitt baut auf den bisherigen Erkenntnissen auf und erweitert die Untersuchung auf eine größere Datenbasis, um die Leistungsfähigkeit und Skalierbarkeit der entwickelten Modelle unter realistischeren Bedingungen zu evaluieren.

VII. ANSATZ II

In diesem Abschnitt der praktischen Umsetzung des Projekts liegt der Fokus darauf, die im ersten Teil erzielten Ergebnisse zu validieren und deren Reproduzierbarkeit mit einer größeren Datenbasis zu überprüfen. In Zusammenarbeit mit Herrn Dr. Hossein Fereidooni[26] wurden neue Konzepte entwickelt, um die Robustheit und Skalierbarkeit der maschinellen Lernmethoden zur Benutzererkennung anhand von Fingerabdruckdaten weiter zu verbessern.

Ein zentraler Aspekt ist die Verifizierung, ob die erzielten Ergebnisse auch mit einer größeren Datenmenge repliziert werden können. Dies dient dazu, die Stabilität und Zuverlässigkeit der Modelle unter realistischen Bedingungen zu bewerten. Hierbei wurden verschiedene Modellarchitekturen getestet, darunter klassische Ansätze wie *Isolation Forest*[27] sowie moderne neuronale Netzwerke wie *Convolutional Neural Networks (CNNs)*[30].

1) Ziele und Herangehensweisen

Die Hauptziele dieses Abschnitts umfassen:

- **Analyse verschiedener Ansätze:** Analyse unterschiedlicher Ansätze des maschinellen Lernens für Canvas Fingerprinting im Rahmen des Projekts.
- **Reproduzierbarkeit:** Sicherstellen, dass die Modelle auch bei größeren und diverseren Datenmengen zuverlässig arbeiten.
- **Optimierung der Modelle:** Verbesserung durch Anpassung von Hyperparametern und Integration moderner Techniken.

2) Einleitung Modelle und Evaluierungsmethoden

In diesem Kapitel werden die eingesetzten Modelle sowie die Gründe für ihre Auswahl beschrieben. Die Entwicklung der Modelle, ihre Implementierung und die Bewertung ihrer Leistung stehen im Fokus. Dabei wird auf spezifische Metriken wie Genauigkeit, Präzision, Recall und F1-Score eingegangen[1].

Die Auswahl der Modelle erfolgte in enger Zusammenarbeit mit Dr. Hossein Fereidooni[25]. Basierend auf seiner umfangreichen Erfahrung und seinen zahlreichen Publikationen[26] in diesem Forschungsgebiet wurde ein fundierter, wissenschaftlich solider Ansatz entwickelt. Der Ansatz sah vor, zunächst einfache maschinelle Lernmethoden zu testen, um zu überprüfen, ob gängige Ansätze den Task der Benutzererkennung auf Basis von Fingerabdruckdaten erfüllen können. Dies umfasst klassische Methoden wie Isolation Forests. Anschließend wurden zunehmend komplexere Modelle wie Autoencoder und Convolutional Neural Networks (CNNs) implementiert, um sicherzustellen, dass keine simpleren Methoden das Konzept effizienter umsetzen können.

Die schrittweise Entwicklung der Modelle ermöglichte es, die Leistungsfähigkeit einfacher und komplexerer Ansätze systematisch zu vergleichen. Dabei wurden die Stärken und Schwächen der verschiedenen Methoden herausgearbeitet. Die Ergebnisse der Evaluierung werden grafisch dargestellt, um Unterschiede in der Modellleistung zu verdeutlichen und die effektivsten Methoden für die Benutzererkennung zu identifizieren.

3) Zusammenfassung der Methodik

Die Methodik umfasst:

- Nutzung der erweiterten Datenbasis mit 100.000 Stichproben von 20 verschiedenen Geräten.
- Entwicklung von Modellen unterschiedlicher Komplexität, um sicherzustellen, dass keine einfacheren Ansätze das Konzept erfüllen könnten.
- Evaluierung der Modelle anhand standardisierter Metriken wie Accuracy, Precision, Recall und F1-Score.
- Analyse von Fehlklassifikationen durch Konfusionsmatrizen zur Identifikation von Schwächen in den Modellen.

Diese strukturierte Vorgehensweise stellt sicher, dass sowohl einfache als auch komplexe Methoden getestet werden, um das Konzept umfassend zu validieren.

Diese systematische Herangehensweise ermöglicht es, die Eignung der verschiedenen Modelle für die Benutzererkennung fundiert zu beurteilen. Die gewonnenen Erkenntnisse sollen zur Weiterentwicklung des Systems beitragen und sollen eine Grundlage für zukünftige Arbeiten in diesem Bereich bilden. Bevor wir auf die Modelle und ihre Leistung eingehen, möchten wir die verwendeten Daten, deren Qualität sowie die statistischen Werte näher betrachten, mit denen die Modelle trainiert wurden.

A. Beschreibung der Daten und Ergebnisse

1) Einführung

In diesem Abschnitt wurden wie in dem vorherigen Ansatz die Bilder aus einer MongoDB-Datenbank geladen und analysiert. Die Datenbank wurde in einem Docker-Container betrieben und leicht modifiziert, um die größeren Datenmengen korrekt speichern zu können. Ein Ziel war es, die Bilddaten zu dekodieren und eine statistische Analyse durchzuführen, um die Qualität der Daten zu ermitteln, zum Beispiel, um Ähnlichkeit zwischen den Bildern verschiedener Benutzer zu bewerten. Dies würde ein Indiz für die Qualität der gesammelten Bilder sein.

2) Datenbankverwaltung und Docker-Setup

Die MongoDB-Datenbank wurde in einem Docker-Container bereitgestellt. Das Docker-Setup bestand aus den folgenden Hauptkomponenten:

- **docker-compose.yml:** Definiert die Services für MongoDB und Jupyter-Notebook.
- **Dockerfile.mongo:** Erstellt ein benutzerdefiniertes MongoDB-Image mit zusätzlichen Tools.
- **Initialisierungsskripte:** Importieren der JSON-Daten (`fingerprintDB.fingerprints.json[10]` und `fingerprintDB.canvassamples.json[10]`) in die Datenbank.

Dieses Setup ist ähnlich zu dem vorherigen Ansatz, aber ist leicht abgeändert.

Die Datenbank enthält zwei Hauptsammlungen:

- **Canvassamples:** Enthält Base64-kodierte Bilddaten sowie Metadaten wie Fingerprint-ID und Erstellungsdatum.
- **Fingerprints:** Speichert Benutzerinformationen wie Fingerprint-Hash, Betriebssystem und Benutzernamen.

Der Fingerprint-Hash stellt in diesem Fall den eindeutigen Schlüssel dar, der die verschiedenen Nutzer voneinander unterscheidet. Dadurch wird es ermöglicht, die Abdrücke den jeweiligen Nutzern zuzuordnen. Eine genauere Erklärung dazu erfolgt in einem späteren Kapitel.

3) Beispiele für Canvas Fingerprints

Zur Veranschaulichung der gesammelten Daten zeigen die folgenden Abbildungen Beispiele für Canvas Fingerprints von den Geräten der verschiedenen Benutzer:



Abbildung 28: Canvas Fingerprint Beispiel für Benutzer 1



Abbildung 29: Canvas Fingerprint Beispiel für Benutzer 2



Abbildung 30: Canvas Fingerprint Beispiel für Benutzer 3

Diese Fingerabdrücke stammen von verschiedenen Nutzern und sind Teil des zweiten Ansatzes, bei dem 100.000 Abdrücke pro Nutzer verwendet wurden. Diese sehen zwar unterschiedlich zu den Abdrücken aus dem ersten Ansatz aus, sind aber mit einer ähnlichen Methode erhoben worden.

B. Verwendete Geräte

Im Rahmen des zweiten Ansatzes wurden Fingerabdruckdaten von 20 verschiedenen Geräten gesammelt und analysiert. Die folgende Liste zeigt die verwendeten Geräte, deren Betriebssysteme, die Anzahl der gesammelten Samples sowie die Browser-Informationen (UserAgent).

Diese Übersicht soll zeigen, wie wir eine Vielzahl unterschiedlicher Geräte für unsere Analyse genutzt haben. Durch die Einbeziehung dieser diversen Gerätetypen konnten wir die Datenmenge und Vielfalt steigern, was zu einer Verbesserung der Datenqualität für unsere Untersuchung führte. Ein Großteil der gesammelten Daten sind von Geräten, die innerhalb von KOBIL[58] bereitgestellt wurden.

• benutzername_1:

- Gerät: ThinkPad E14 Gen 5
- Betriebssystem: Windows 11
- Browser/User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/116.0.5845.188 Safari/537.36
- Anzahl der Samples: 100.000

• benutzername_3:

- Gerät: iPhone 11

- Betriebssystem: iOS 17.5.1
- Browser/User-Agent: Mozilla/5.0 (iPhone; CPU iPhone OS 17_5_1 like Mac OS X) AppleWebKit/605.1.15 (KHTML, like Gecko) Mobile/15E148
- Anzahl der Samples: 100.000

• benutzername_4:

- Gerät: Motorola 60
- Betriebssystem: Android 11
- Browser/User-Agent: Mozilla/5.0 (Linux; Android 11; Moto G60) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/96.0.4664.45 Mobile Safari/537.36
- Anzahl der Samples: 100.000

• benutzername_5:

- Gerät: iPhone 7
- Betriebssystem: iOS 15.7
- Browser/User-Agent: Mozilla/5.0 (iPhone; CPU iPhone OS 15_7 like Mac OS X) AppleWebKit/605.1.15 (KHTML, like Gecko) Version/15.6 Mobile/15E148 Safari/604.1
- Anzahl der Samples: 100.000

• benutzername_6:

- Gerät: S24 Samsung
- Betriebssystem: Android 14
- Browser/User-Agent: Mozilla/5.0 (Linux; Android 14; SM-S24) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/117.0.5938.62 Mobile Safari/537.36
- Anzahl der Samples: 100.000

• benutzername_7:

- Gerät: iPhone SE
- Betriebssystem: iOS 15.4.1
- Browser/User-Agent: Mozilla/5.0 (iPhone; CPU iPhone OS 15_4_1 like Mac OS X) AppleWebKit/605.1.15 (KHTML, like Gecko) Mobile/15E148
- Anzahl der Samples: 100.000

• benutzername_8:

- Gerät: Tab S6 Lite
- Betriebssystem: Android 13
- Browser/User-Agent: Mozilla/5.0 (Linux; Android 13; SM-P610) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/117.0 Safari/537.36
- Anzahl der Samples: 100.000

• benutzername_9:

- Gerät: iPhone SE (2016)
- Betriebssystem: iOS 14.4.1
- Browser/User-Agent: Mozilla/5.0 (iPhone; CPU iPhone OS 14_4_1 like Mac OS X) AppleWebKit/605.1 Mobile Safari
- Anzahl der Samples: 100.000

• benutzername_10:

- Gerät: iPhone 8
- Betriebssystem: iOS 14.4.1
- Browser/User-Agent: Mozilla/5.0 (iPhone; CPU iPhone OS 14_4_1 like Mac OS X) AppleWebKit/605.1 Mobile Safari

- Anzahl der Samples: 100.000
- **benutzername_11:**
 - Gerät: Home PC
 - Betriebssystem: Windows 11
 - Browser/User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit Chrome Version Safari
 - Anzahl der Samples: 100.000
- **benutzername_13:**
 - Gerät: Aspire A515-56
 - Betriebssystem: Windows 11
 - Browser/User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit Chrome Version Safari
 - Anzahl der Samples: 100.000
- **benutzername_14:**
 - Gerät: MacBook Air M2
 - Betriebssystem: macOS Sonoma 14.6.1
 - Browser/User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X Sonoma) AppleWebKit Safari
 - Anzahl der Samples: 100.000
- **benutzername_15:**
 - Gerät: Lenovo ThinkPad E330
 - Betriebssystem: Ubuntu 22.04.5 LTS
 - Browser/User-Agent: Mozilla/5.0 (X11; Linux x86_64) Gecko Firefox/Linux
 - Anzahl der Samples: 100.000
- **benutzername_16:**
 - Gerät: iPhone 12 Mini
 - Betriebssystem: iOS 18.0.1
 - Browser/User-Agent: Mozilla/5.0 (iPhone; CPU iPhone OS 18_0_1 like Mac OS X) AppleWebKit Safari/Mobile
 - Anzahl der Samples: 100.000
- **benutzername_17:**
 - Gerät: HP ZBook Power G8
 - Betriebssystem: Ubuntu 24.04.1 LTS
 - Browser/User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64) Gecko Firefox/Linux
 - Anzahl der Samples: 100.000
- **benutzername_18:**
 - Gerät: MacBook Pro
 - Betriebssystem: macOS Ventura 13.6
 - Browser/User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X) AppleWebKit Safari
 - Anzahl der Samples: 100.000
- **benutzername_19:**
 - Gerät: Lenovo 21N10007GE
 - Betriebssystem: Windows 11 Pro Arm
 - Browser/User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; ARM64) AppleWebKit Chrome Edge
 - Anzahl der Samples: 100.000
- **benutzername_20:**

- Gerät: Mac Mini (2016)
- Betriebssystem: macOS Sonoma 14.3.1
- Browser/User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X) AppleWebKit Safari
- Anzahl der Samples: 100.000

- **benutzername_21:**
 - Gerät: Mac Mini M1 (2020)
 - Betriebssystem: macOS Ventura 13.2
 - Browser/User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X) AppleWebKit Safari
 - Anzahl der Samples: 100.000

- **benutzername_22:**
 - Gerät: M2 Mac Mini
 - Betriebssystem: macOS Sequoia 15.0
 - Browser/User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X) AppleWebKit Safari
 - Anzahl der Samples: 100.000

Die Nummerierung der Benutzer reicht zwar bis 22, jedoch wurden zwei Geräte aufgrund von Fehlern bei der Datensammlung aus der Datenbank entfernt. An ihrer Stelle wurden zwei neue Geräte als Benutzer 21 und 22 in die Datenbank aufgenommen. Somit bleibt die Gesamtzahl der analysierten Geräte bei 20, wie ursprünglich geplant.

1) Verteilung der Geräte, Betriebssysteme, Hersteller und Browser

Die folgende Analyse zeigt die Verteilung der erfassten Nutzerdaten, basierend auf einer Stichprobe von **2.000.000 Datensätzen**, welche die Verwendung von Geräten, Betriebssystemen, Herstellern und Browsern zeigt. Die meisten der gesammelten Daten stammen von Geräten aus dem Unternehmen und innerhalb des Unternehmens.

2) 1. Geräteverteilung

Die Mehrheit der gesammelten Daten zeigt, dass **40%** der erfassten Geräte Smartphones sind, gefolgt von **35%** Laptops/PCs. Tablets und Desktop-Macs machen mit **5%** bzw. **20%** nur einen kleineren Anteil aus. Dies wird in dem folgenden Graphen illustriert.

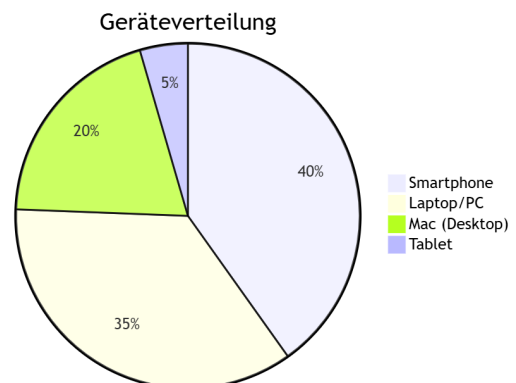


Abbildung 31: Geräteverteilung in der Stichprobe

3) 2. Betriebssysteme

Mit einem Anteil von **30%** ist iOS das am häufigsten erfasste Betriebssystem in den Daten, gefolgt von macOS mit **25%** Anteil. Windows ist direkt dahinter mit **20%** danach kommt Android mit **15%** und Abschließend kommt Ubuntu/Linux mit **10%**.

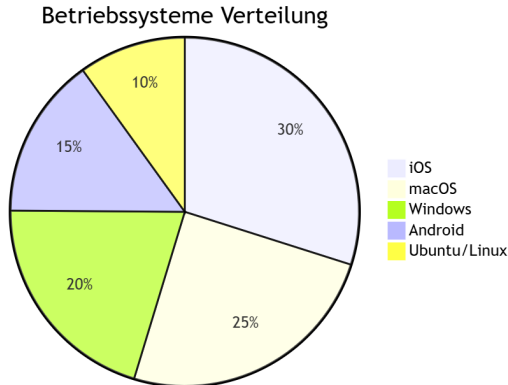


Abbildung 32: Betriebssystemverteilung in der Stichprobe

4) 3. Herstellerverteilung

Die Hersteller verteilen sich von Apple bis hin zu Lenovo und Samsung sowie weiteren kleineren Marken wie Motorola und Acer.

Basierend auf den gesammelten Daten ergibt sich folgende Verteilung: Apple dominiert mit einem Anteil von **55%**, was auf die iPhones und Macs zurückzuführen ist. Lenovo folgt mit einem Anteil von **15%**, während Samsung mit einem Anteil von **10%** mit 2 Geräten dahinter liegt. Motorola, HP und Acer teilen sich jeweils kleinere Anteile von je **5%** bzw 1 Gerät pro Hersteller. Zusätzlich gibt es eine Kategorie für generische oder unbekannte Geräte, die ebenfalls einen Anteil von **5%** ausmacht, dies ist mein Heim Pc welcher keiner Marke angehört.

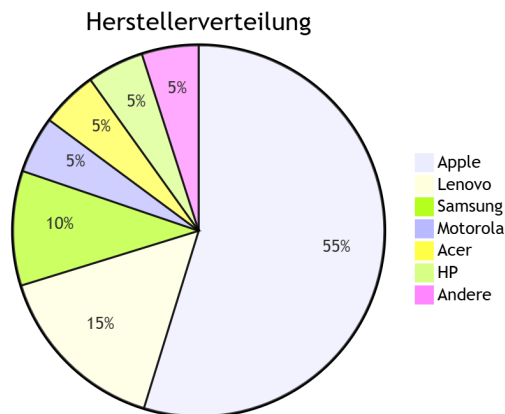


Abbildung 33: Herstellerverteilung in der Stichprobe

5) 4. Browserverteilung

Google Chrome ist mit einem Anteil von **47%** der am häufigsten genutzte Browser, gefolgt von Safari mit **35%**, was die Apple-Geräten verursachen. Firefox und Edge liegen deutlich dahinter mit Anteilen von **12%** bzw. **6%**.

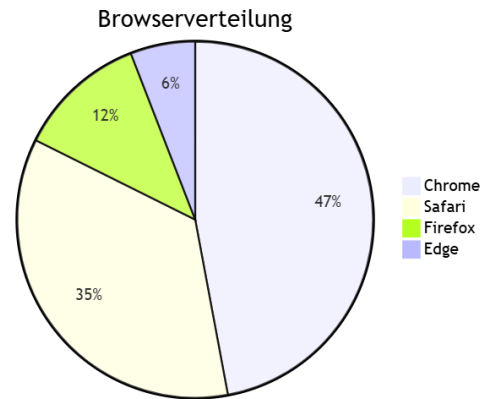


Abbildung 34: Browserverteilung in der Stichprobe

Da wir nun die Nutzerverteilung analysiert haben gehen wir nun auf die Qualität der Daten ein was durch eine statische Analyse umgesetzt wird.

6) Statistische Analyse

In diesem Abschnitt befassen wir uns mit der statistischen Analyse der Daten, um deren Qualität zu verifizieren. Es wurden verschiedene Tests durchgeführt, deren genaue Methoden und Durchführung auf der GitHub-Seite[10] des Projekts dokumentiert sind.

Als eine der ersten Analysen haben wir die Länge, die Abweichung sowie den Mittelwert der Daten untersucht. Diese grundlegenden statistischen Kennzahlen liefern Einblicke in die Struktur und Verteilung unserer Datensätze. Hier ein paar Beispiele:

- Benutzername 1: Mittelwert 8371.52, Standardabweichung 898.78.
- Benutzername 3: Mittelwert 8810.50, Standardabweichung 976.08.
- Benutzername 4: Mittelwert 8611.56, Standardabweichung 872.02.

Die folgende Abbildung zeigt die Verteilung der Stringlängen:

Diese Tests wurden durchgeführt, um fehlerhafte Datensammlungen zu identifizieren und zu beheben. Zudem ermöglichten sie uns, erste Einschätzungen hinsichtlich der Heterogenität der Daten zu gewinnen. Die Analyse der statistischen Kennzahlen bestätigt, dass die Datenerhebung ordnungsgemäß verlief und dass die erfassten Daten ausreichende Unterschiede aufweisen, was ein entscheidender Faktor für ein effektives Modell Training ist.

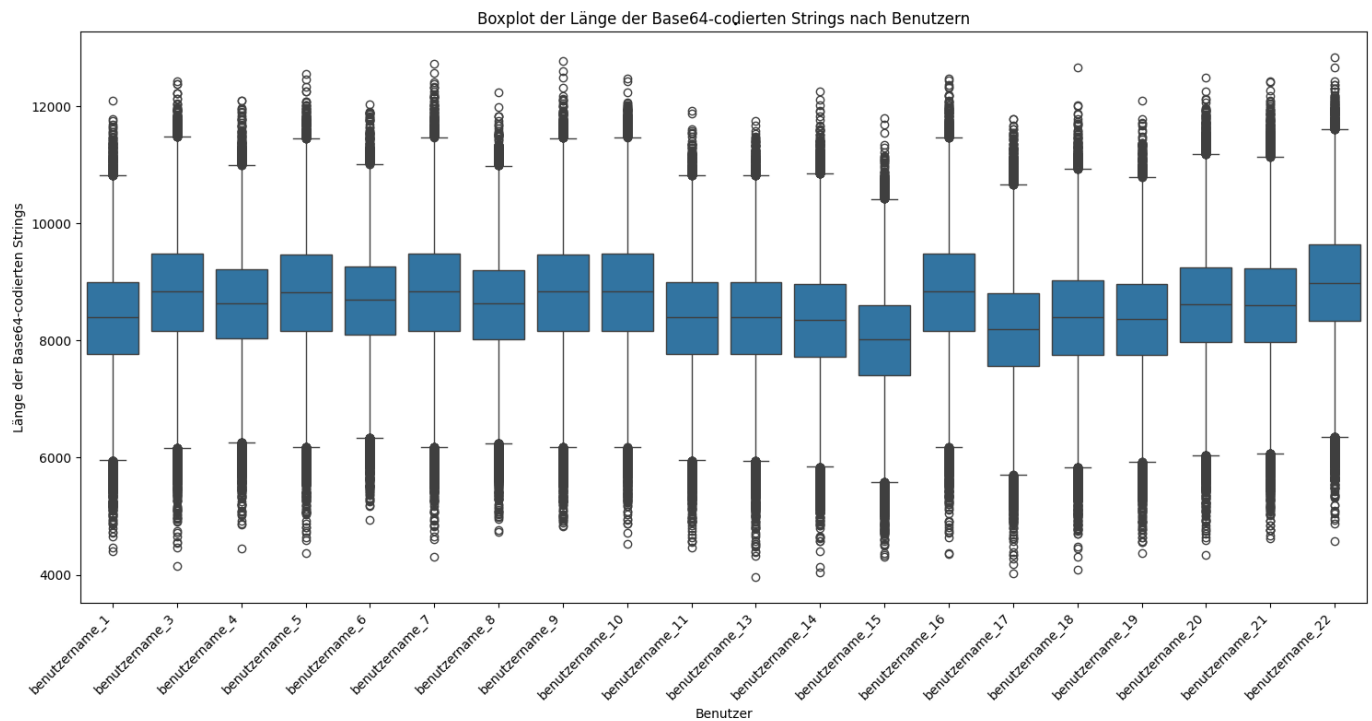


Abbildung 35: Boxplot der Länge der Base64-codierten Strings nach Benutzern

7) Ähnlichkeitsanalyse mit SSIM

Die strukturelle Ähnlichkeit (Structural Similarity Index Measure, SSIM[63]) wurde berechnet, um die Unterschiede zwischen den Bildern verschiedener Benutzer zu bewerten. SSIM[63] ist eine Methode zur Messung der wahrgenommenen Ähnlichkeit zwischen zwei Bildern, die Aspekte wie Helligkeit, Kontrast und Struktur berücksichtigt. Der SSIM-Wert liegt zwischen -1 und 1, wobei 1 eine perfekte Übereinstimmung bedeutet, 0 keine strukturelle Ähnlichkeit anzeigt und negative Werte auf eine inverse Struktur hindeuten. In der Praxis werden meist Werte zwischen 0 und 1 beobachtet.

Die folgende Heatmap zeigt die SSIM-Werte zwischen allen Benutzern: auch in den entsprechenden Tests bestätigt.

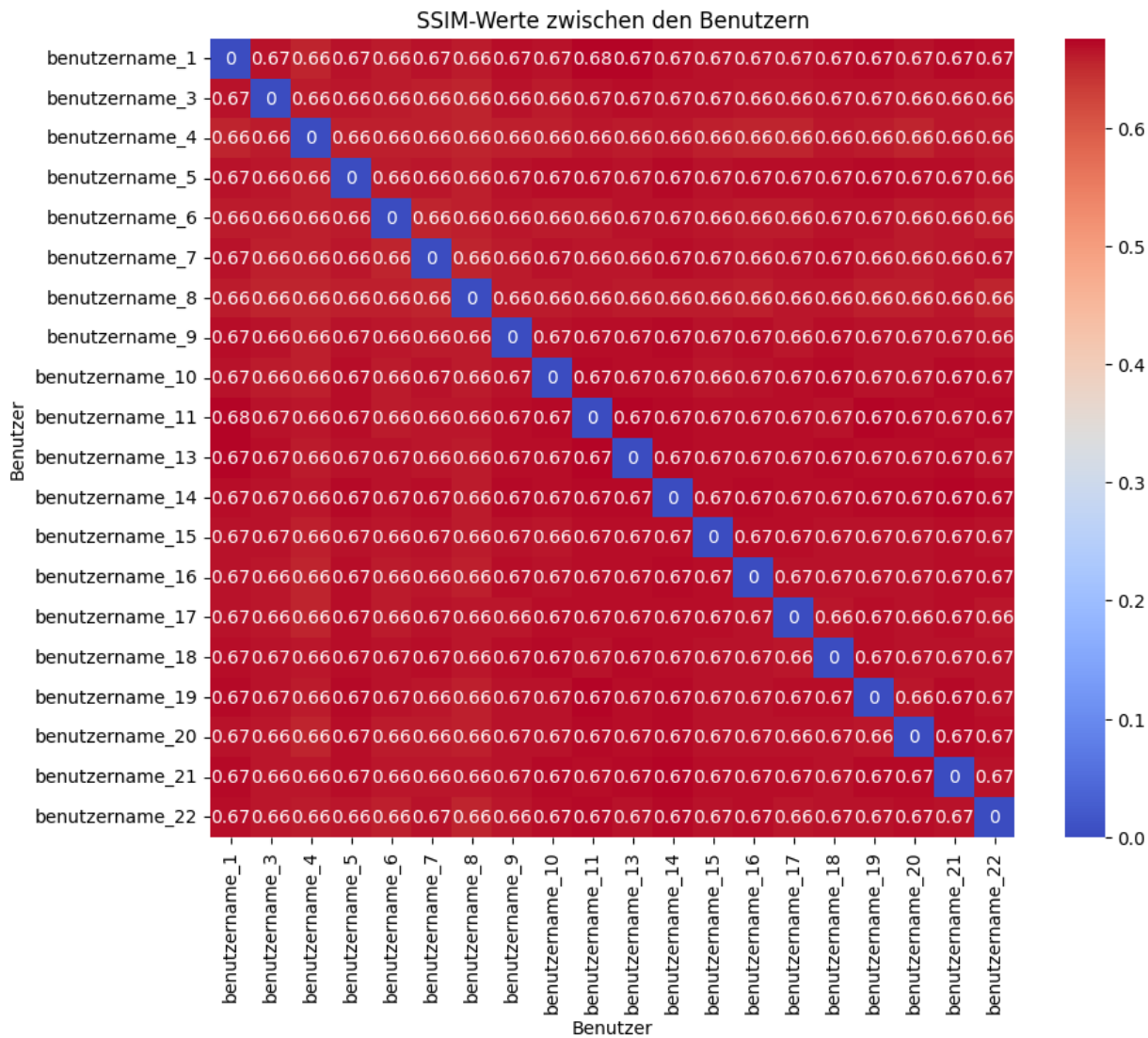


Abbildung 36: Heatmap der SSIM-Werte zwischen den Benutzern

Die SSIM-Werte zeigen moderate Ähnlichkeiten zwischen den Benutzern, wie in der Heatmap zu erkennen ist. Die Werte bewegen sich hauptsächlich im Bereich von 0.66 bis 0.681, was auf eine konsistente, aber nicht zu hohe Ähnlichkeit hinweist.

Obwohl die Daten eine gewisse Ähnlichkeit aufweisen, sind sie nicht identisch, was uns dabei unterstützt, zu entscheiden, ob sie für maschinelles Lernen geeignet sind. Da alle Daten mit demselben Verfahren erfasst wurden, wird ein gewisser Übereinstimmungsgrad erwartet. Die beobachteten SSIM-Werte von etwa 0.671 zwischen den meisten Benutzern bestätigen diese moderat erwartete Ähnlichkeit, während sie dennoch ausreichend variieren, um die Unterscheidbarkeit der Fingerabdrücke zu gewährleisten. Diese Ergebnisse wurden

C. Datenstruktur

Im zweiten Ansatz wurde die Datenstruktur überarbeitet, um größere Datenmengen effizient speichern zu können. Während im ersten Ansatz die Fingerabdrücke direkt in einem Array innerhalb derselben Tabelle gespeichert wurden, stieß diese Methode bei größeren Datenmengen schnell an ihre Grenzen (maximal ca. 2000 Werte). Um dieses Problem zu lösen, haben wir eine neue Struktur implementiert, die eine Trennung der Fingerabdrücke und ihrer zugehörigen Samples vorsieht.

Die neue Datenstruktur besteht aus zwei Tabellen: `fingerprints` und `canvassamples`. Die Tabelle `fingerprints` speichert allgemeine Informationen zu den Fingerabdrücken, wie den Benutzernamen, das Betriebssystem und die Geräteinformationen. Die tatsächlichen Canvas-Samples werden in der Tabelle `canvassamples` gespeichert, wobei jede Probe über einen Fremdschlüssel (`fingerprintId`) mit einem Fingerabdruck verknüpft ist.

Dieser Ansatz ermöglicht eine flexible Speicherung von beliebig vielen Samples pro Benutzer und verbessert die Skalierbarkeit der Datenbank. Die folgende Abbildung³⁷ zeigt die aktuelle Datenstruktur:

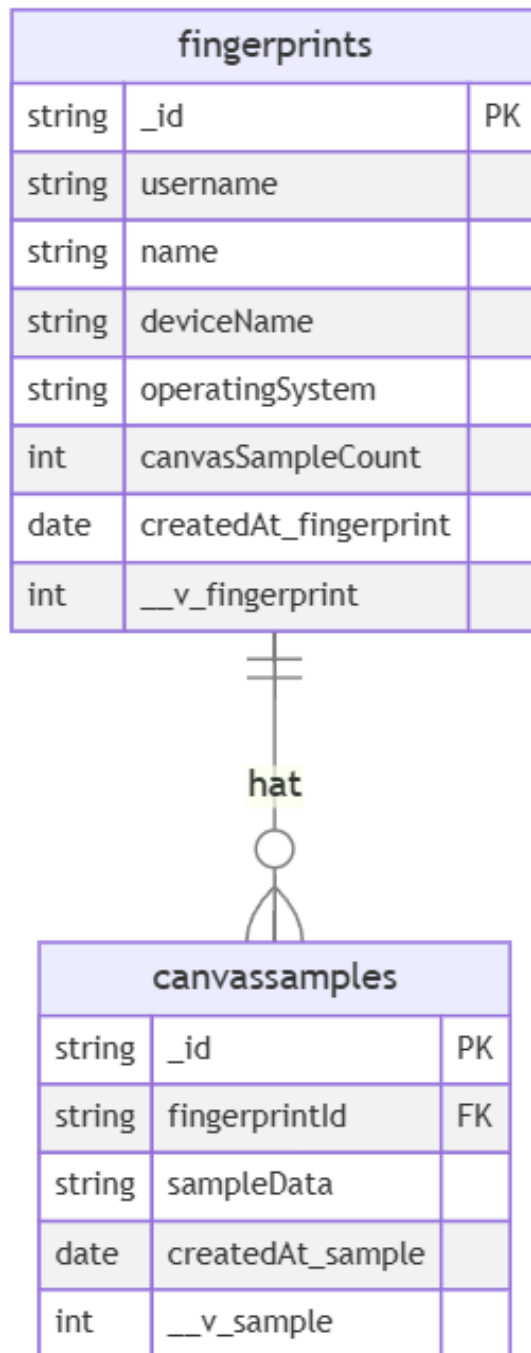


Abbildung 37: Datenstruktur des zweiten Ansatzes

In den folgenden Kapiteln werden wir auf die verschiedenen Machine Learning Methoden eingehen und diese erläutern.

D. Isolation Forest

Der Isolation Forest^[27] ist ein Algorithmus zur Anomalieerkennung, der auf dem Prinzip basiert, dass Anomalien

einfacher zu isolieren sind als normale Datenpunkte. Der Algorithmus erstellt sogenannte Isolation Trees, die durch zufällige Partitionierung der Daten entstehen. Anomalien, die seltener und weiter entfernt vom Rest der Daten liegen, werden mit weniger Partitionen isoliert, was zu kürzeren Pfadlängen führt. Die Hauptvorteile des Isolation Forests sind seine Effizienz bei großen Datensätzen und die Fähigkeit, ohne vorherige Klassifizierung der Daten zu arbeiten. Er wurde im Rahmen dieser Arbeit genutzt, um zu testen, ob der Algorithmus komplexe Strukturen wie Canvas Fingerprints analysieren und unterscheiden kann.

1) Isolation Forest: Erste Modellimplementierung

In diesem Abschnitt wird die Implementierung und Evaluierung des ersten getesteten Modells, des Isolation Forests, beschrieben. Dieses Modell wurde entwickelt, um Anomalien in den Bilddaten zu erkennen und zu klassifizieren. Im Folgenden werden die wichtigsten Schritte, Ergebnisse und Erkenntnisse des Modells dargestellt.

2) Zusammenfassung des Codes

Der Code für das Isolation-Forest-Modell umfasst mehrere Schritte, die von der Datenvorbereitung bis zur Bewertung der Modellleistung reichen:

- **Daten laden und zusammenführen:** Daten wurden aus einer MongoDB-Datenbank geladen und basierend auf den Feldern `fingerprintId` und `_id` zusammengeführt.
- **Datenvorverarbeitung:** Funktionen wie `process_image_rgb` wurden definiert, um Bilder aus Base64-Strings zu dekodieren.
- **Datenaufteilung:** Die Daten wurden in Trainings-, Validierungs- und Testsets aufgeteilt (70/20/10).
- **Modelltraining:** Ein Isolation-Forest-Modell wurde mit den Trainingsdaten trainiert. Vorhersagen wurden für Validierungs- und Testsets gemacht.
- **Modellbewertung:** Die Leistung wurde anhand von Metriken wie F1-Score, Precision, Recall und Accuracy bewertet. Zusätzlich wurden eine Konfusionsmatrix erstellt.

3) Ergebnisse des Modells

Die Ergebnisse des ersten Isolation-Forest-Modells zeigen die Herausforderungen bei der Benutzererkennung:

- **Gesamtdatensatz:** Der Datensatz umfasste 2.000.000 Fingerabdrücke.
- **Validierungsset:** - F1 Score: 0.1569 - Precision: 0.4598 - Recall: 0.0946 - Accuracy: 0.5006
- **Testset:** - F1 Score: 0.1555 - Precision: 0.4684 - Recall: 0.0932 - Accuracy: 0.4975

Die niedrigen Werte für F1-Score, Precision und Recall deuten darauf hin, dass das Modell Schwierigkeiten hatte, die Benutzer korrekt zu identifizieren. Dies führt zu der Erkenntnis, dass das erste Isolation-Forest-Modell nicht in der Lage ist, Unterschiede zwischen den verschiedenen Nutzern und deren Fingerabdrücken zu erkennen.

4) Grafische Darstellung der Ergebnisse

Zur Veranschaulichung der Modellleistung wurden folgende Visualisierungen erstellt:

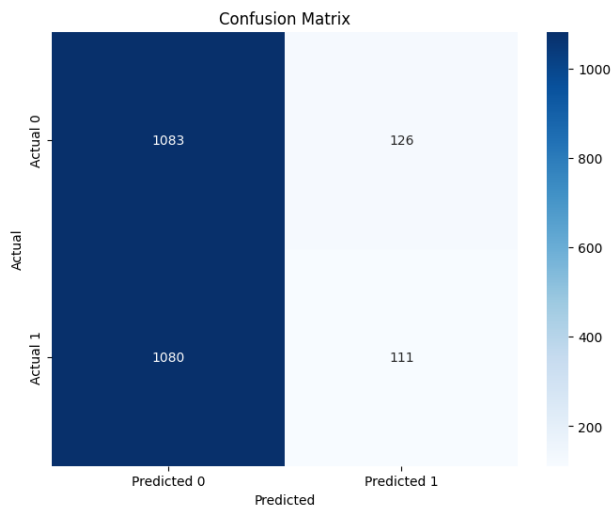


Abbildung 38: Konfusionsmatrix: Zeigt die Anzahl der True Positives, True Negatives, False Positives und False Negatives.

Wie hier zu erkennen ist, hat das Modell leider alle Fingerabdrücke, auch die korrekten, als nicht zu Nutzer 1 zugehörig erkannt. Dies erklärt auch die schlechten Werte im F1-Score und in den anderen Metriken.

5) Isolation Forest: Zweite Modellimplementierung

Da das erste Modell Schwierigkeiten hatte zu unterscheiden, welche Fingerabdrücke zu Nutzer 1 gehören und welche nicht, wurde ein zweites Modell entwickelt. Dieses Modell wurde mit angepassten Hyperparametern trainiert und verwendet modifizierte Trainingsmethoden, insbesondere in der Vorverarbeitung, um die Erkennungsgenauigkeit zu verbessern.

6) Zusammenfassung des Codes

Die zweite Version des Isolation-Forest-Modells folgt im Wesentlichen denselben Schritten wie die erste Implementierung, unterscheidet sich jedoch grundlegend in der Methode der Feature-Extraktion:

Anstatt, wie im ersten Modell, direkt mit den rohen RGB-Pixelwerten der Bilddaten zu arbeiten, wobei hierzu die Funktion `process_image_rgb` genutzt wird, werden in dieser Version statistische Kennzahlen und Histogramme aus den Bildern extrahiert. Diese komprimierten Merkmale sollen die relevanten visuellen Informationen übersichtlicher darstellen. Mit den neu extrahierten Features wird anschließend das Isolation-Forest-Modell trainiert und evaluiert.

7) Ergebnisse des Modells

Die Ergebnisse der zweiten Isolation-Forest-Implementierung zeigen eine ähnliche Leistung wie die erste Version trotz Änderungen in der Architektur:

- Gesamtdatensatz: Der Datensatz enthält 2.000.000 Fingerabdrücke.
- Validierungsset:
 - F1 Score: 0.1823
 - Precision: 1.0
 - Recall: 0.1003
 - Accuracy: 0.10031
- Testset:

- F1 Score: 0.18617
- Precision: 1.0
- Recall: 0.1026
- Accuracy: 0.1026

Die Ergebnisse, obwohl diese auf den ersten Blick leicht besser erscheinen, deuten darauf hin, dass das Modell weiterhin Schwierigkeiten hat, Benutzer zuverlässig zu identifizieren. Insbesondere die niedrigen Werte für F1-Score, Precision und Recall zeigen, dass diese Art von Architektur sich nicht für unsere Nutzungszwecke eignet

8) Grafische Darstellung der Ergebnisse

Zur Veranschaulichung der Modellleistung wurden folgende Visualisierungen erstellt:

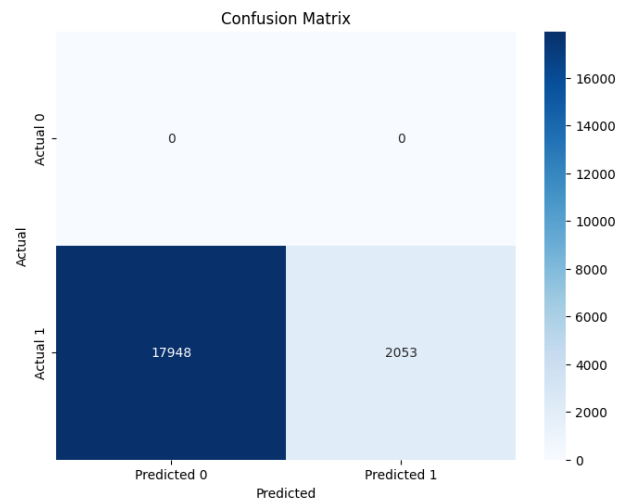


Abbildung 39: Konfusionsmatrix: Zeigt die Anzahl der True Positives, True Negatives, False Positives und False Negatives.

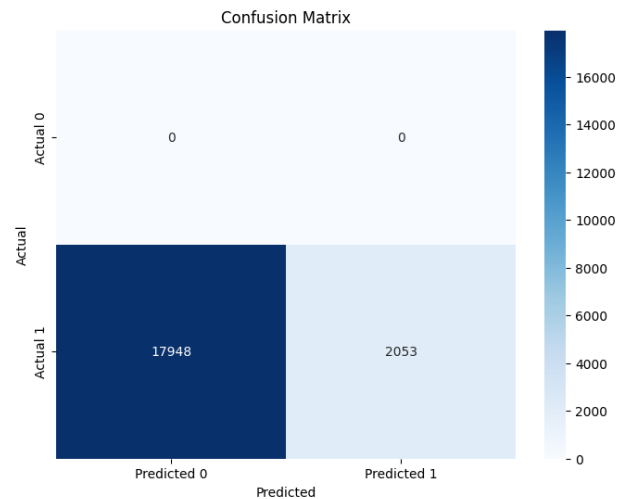


Abbildung 40: Konfusionsmatrix: Zeigt die Anzahl der True Positives, True Negatives, False Positives und False Negatives.

Die Ergebnisse deuten darauf hin, dass das Modell weiterhin Schwierigkeiten hat, Benutzer zuverlässig zu identifizieren.

Insbesondere die niedrigen Werte für F1-Score, Precision und Recall zeigen, dass diese Art von Architektur nicht für unsere Nutzungszwecke geeignet ist. Beide Modelle die erstellt worden sind nicht in der Lage gewesen die Benutzer zu unterscheiden.

E. Autoencoder: Modellimplementierung und Evaluierung

Im Rahmen der Evaluierung verschiedener maschineller Lernansätze wurde ein Autoencoder-Modell implementiert, um dessen Eignung für die Benutzererkennung zu untersuchen. Nachdem wir die einfachen Machine Learning Methoden abgeschlossen haben, wenden wir uns nun den komplexeren Ansätzen zu wie dem Autoencoder.

1) Was ist ein Autoencoder?

Ein Autoencoder ist ein neuronales Netzwerk, das darauf trainiert wird, Eingabedaten durch eine niedrigdimensionale Darstellung zu rekonstruieren. Es besteht aus zwei Hauptkomponenten:

- **Encoder:** Reduziert die Dimension der Eingabedaten und extrahiert wesentliche Merkmale.
- **Decoder:** Rekonstruiert die ursprünglichen Daten aus der komprimierten Darstellung.

In diesem Projekt wird der Autoencoder verwendet, um relevante Merkmale aus den Fingerabdruckdaten zu extrahieren und Unterschiede zwischen Nutzern zu erkennen.

2) Zusammenfassung des Codes

Die Implementierung des Autoencoder-Modells umfasst mehrere Schritte:

- **Datenbankzugriff:** Verbindung zur MongoDB wurde hergestellt und relevante Daten in DataFrames geladen.
- **Datenaufteilung:** Die Daten wurden in Trainings-, Validierungs- und Testsets unterteilt (70/20/10).
- **Bildverarbeitung:** Funktionen zum Dekodieren und Verarbeiten der Bilder wurden implementiert, einschließlich eines DataGenerators für Batch-Verarbeitung.
- **Autoencoder-Training:** Ein Autoencoder wurde trainiert und der Rekonstruktionsfehler berechnet.
- **Evaluation:** Die Modellleistung wurde anhand von Rekonstruktionsfehlern und einem definierten Schwellenwert bewertet.

3) Ergebnisse des Modells

Das Modell, das trainiert wurde, zeigte die folgenden Ergebnisse

- Durchschnittlicher Rekonstruktionsfehler (Nutzer 1): 0.003645
- Durchschnittlicher Rekonstruktionsfehler (zufällige Nutzer): 0.003870
- True Positives: 1058
- True Negatives: 13024
- False Positives: 5918
- False Negatives: 0
- Genauigkeit: 0.7041

Die Ergebnisse zeigen, dass das Modell nicht in der Lage ist Nutzer 1 zu erkennen bzw. eher durch Zufall diesen erkennen kann.

4) Grafische Darstellung der Ergebnisse

Zur Veranschaulichung der Modellleistung wurden folgende Visualisierungen erstellt:

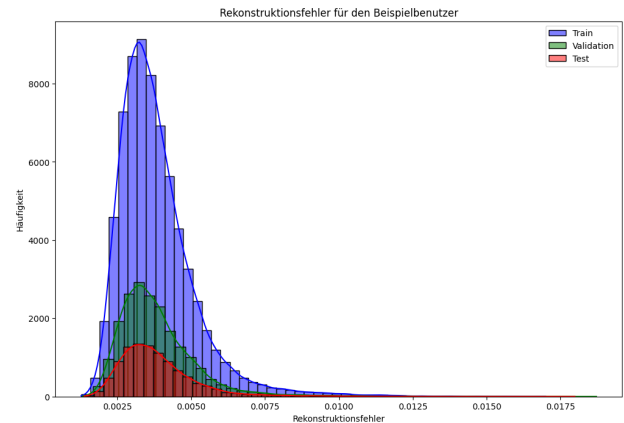


Abbildung 41: Rekonstruktionsfehler für den Beispielnutzer (Train, Validation, Test)

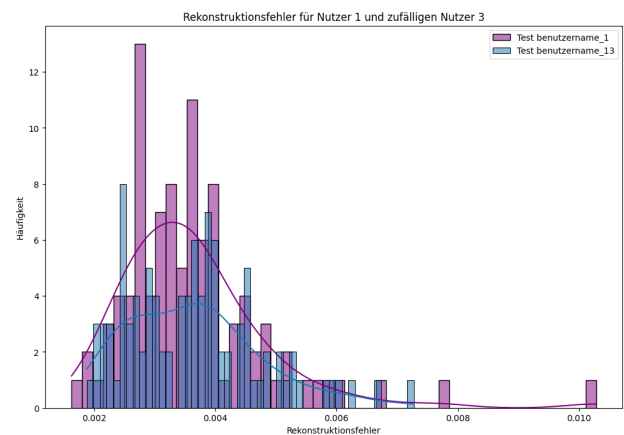


Abbildung 42: Rekonstruktionsfehler für Nutzer 1 und zufälligen Nutzer 3

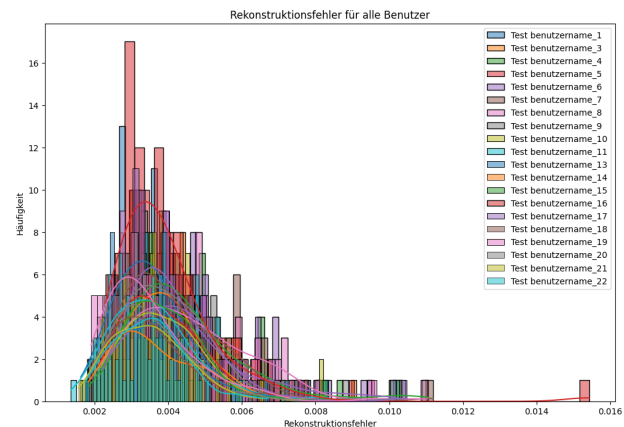


Abbildung 43: Rekonstruktionsfehler für alle Nutzer

- **Fehleranalyse:** In Abbildung 41 sehen wir, wie gut unser Autoencoder die Fingerabdrücke eines einzelnen Nutzers, in diesem Fall Benutzer 1 nachbilden kann. Wir vergleichen dabei die Fehler für Trainings-, Test- und Validierungsdaten. Das gibt uns einen Einblick, wie zuverlässig das Modell für diesen Nutzer funktioniert.
- **Nutzer-Vergleich:** Abbildung 42 stellt die Fehler von Nutzer 1 denen eines zufällig gewählten Nutzers (hier Nutzer 3) gegenüber. So können wir sehen, ob unser Modell tatsächlich zwischen verschiedenen Nutzern unterscheiden kann. Das ist wichtig, um die Effektivität unseres Fingerprinting-Ansatzes zu überprüfen.
- **Gesamtübersicht:** In Abbildung 43 betrachten wir die Fehlerverteilung über alle Nutzer hinweg. Diese Darstellung hilft uns zu verstehen, wie gut unser Modell insgesamt funktioniert und ob es konsistent zwischen verschiedenen Nutzern unterscheiden kann.

Die Analyse der Graphen zeigt, dass die Rekonstruktionsfehler des Autoencoders für verschiedene Nutzer sehr ähnlich sind. Obwohl das Modell in einigen Fällen zwischen Nutzern unterscheiden kann, scheint dies eher zufällig zu sein. Besonders bei der Betrachtung mehrerer Nutzer, wie in Graph 43 ersichtlich, wird deutlich, dass die Methode für eine zuverlässige Unterscheidung ungeeignet ist. In diesem kann man erkennen, dass der Rekonstruktionsfehler für die verschiedene Nutzer sehr nah beieinander liegt und somit keine Differenzierung ermöglicht. Zusammenfassend lässt sich sagen, dass dieses Modell keine ausreichend robuste und zuverlässige Methode für die Benutzererkennung darstellt.

F. Autoencoder: Zweite Modellimplementierung

Nach der Evaluierung des ersten Autoencoder-Modells wurde eine zweite Version entwickelt, um die Datenverarbeitung zu vereinfachen und die Modellleistung weiter zu optimieren. Im Folgenden werden die Implementierungsschritte, Ergebnisse und Unterschiede zur ersten Version beschrieben.

1) Unterschiede zur ersten Version

Die zweite Version unterscheidet sich von der ersten Version in wenigen Änderungen an dem Code und der Architektur sowie Änderungen an der Datenverarbeitung.

2) Ergebnisse des Modells

Die Ergebnisse zeigen Verbesserung gegenüber der ersten Version:

- Durchschnittlicher Rekonstruktionsfehler (Nutzer 1): 0.0558
- Durchschnittlicher Rekonstruktionsfehler (zufällige Nutzer): 0.0576

Die Ergebnisse deuten darauf hin, dass das Modell nicht in der Lage ist, Unterschiede zwischen Nutzern zu erkennen, da die Rekonstruktionsfehler der unterschiedlichen Benutzer erneut keine deutlichen Unterschiede aufweisen.

3) Grafische Darstellung der Ergebnisse für das zweite Modell

Zur Veranschaulichung der Leistung des zweiten Modells wurden folgende Visualisierungen erstellt:

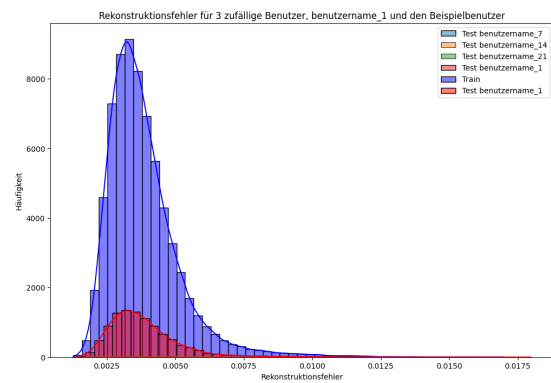


Abbildung 44: Rekonstruktionsfehler für den Beispielnutzer und andere Nutzer

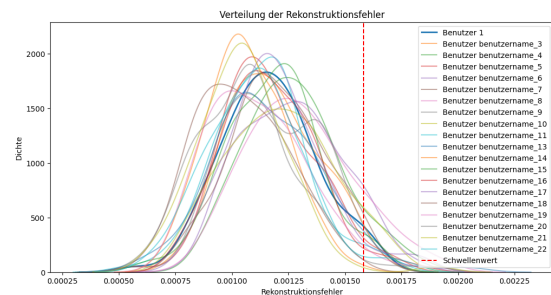


Abbildung 45: Rekonstruktionsfehler für Nutzer 1 und zufälligen Nutzer 3 – Zweites Modell

- **Rekonstruktionsfehler für den Beispielnutzer:** Die Verteilung der Rekonstruktionsfehler für die Trainings-, Validierungs- und Testdaten sowie für zufällig ausgewählte Nutzer gegen den Benutzer 1 ist in Abbildung 44 dargestellt. Diese Abbildung zeigt, wie gut das zweite Modell die Daten des Nutzers rekonstruieren kann
- **Rekonstruktionsfehler für alle Nutzer:** Die Verteilung der Rekonstruktionsfehler über alle Nutzer hinweg wird in Abbildung 45 dargestellt. Diese zeigt die allgemeine Leistungsfähigkeit des zweiten Modells bei der Unterscheidung zwischen allen Nutzern.

Mann kann in den beiden Graphen erkennen, dass der Rekonstruktionsfehler der beiden Modelle für alle Benutzer sehr ähnlich ist. Dies zeigt, dass diese Art von Autoencoder nicht für unsere Testzwecke geeignet ist. Da auch die zweite Version des Autoencoders nur begrenzte Ergebnisse lieferte, wurde eine weitere Version der Autoencoder Struktur getestet, diese wird nun im nächsten Kapitel besprochen.

4) CNN Autoencoder

Als dritte Methode wurde ein CNN Autoencoder[64] implementiert.

Im Gegensatz zu herkömmlichen Autoencodern, die vollständig auf Dense-Layer-Architekturen basieren, verwendet der CNN-Autoencoder Convolutional Neural Networks (CNNs), um die räumlichen Merkmale der Eingabebilder besser zu erfassen. Diese Architektur ist besonders vorteilhaft für unseren Verwendungszweck, da sie lokale Muster

und Strukturen erkennt, die in Dense-Layern verloren gehen könnten.

Die Architektur des CNN-Autoencoders unterscheidet sich deutlich von dichten Autoencodern, da sie die räumlichen Zusammenhänge in Bilddaten bewahrt. Im Encoder werden Convolutional-Schichten (z. B. Conv2D mit 32 Filtern, einer Kernelgröße von 3x3 und ReLU-Aktivierung) verwendet, um lokale Muster wie Kanten, Texturen und Formen direkt aus den Bildern zu extrahieren. Dabei bleibt die natürliche 2D-Struktur der Bilder erhalten. Zur Effizienzsteigerung wird MaxPooling2D eingesetzt, um die räumliche Dimensionalität zu reduzieren und gleichzeitig die wichtigsten Merkmale beizubehalten. Im Decoder ermöglichen UpSampling2D-Schichten in Kombination mit zusätzlichen Convolutional-Schichten eine schrittweise Rekonstruktion der ursprünglichen Bildgröße. Dadurch werden die räumlichen Beziehungen der Pixel präzise wiederhergestellt.

5) Modellbewertung

Die Leistung des Modells wurde anhand des Rekonstruktionsfehlers bewertet. Ein Schwellenwert wurde berechnet, um zwischen normalen Daten und Anomalien zu unterscheiden. Dies wurde genutzt, um festzustellen, wie das Modell auf die Samples reagiert und dies zu visualisieren.

6) Grafische Darstellung der Ergebnisse

Zur Veranschaulichung der Modelleistung wurden folgende Visualisierungen erstellt:

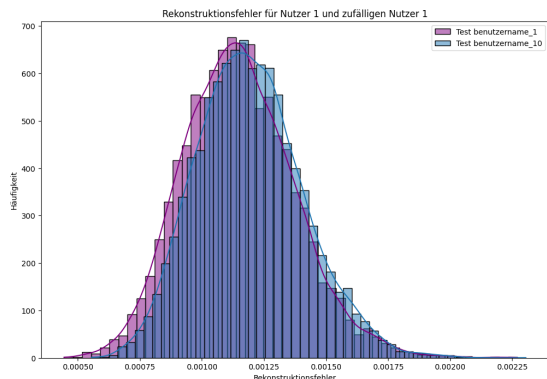


Abbildung 46: Rekonstruktionsfehler für den Beispielnutzer (Train, Validation, Test)

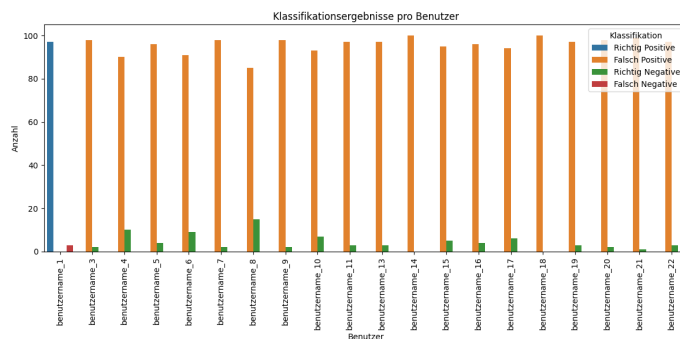


Abbildung 47: Rekonstruktionsfehler für Nutzer 1 und zufälligen Nutzer 3

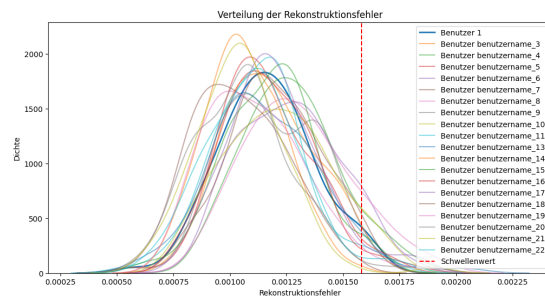


Abbildung 48: Rekonstruktionsfehler für alle Nutzer

- **Rekonstruktionsfehler für den Beispielnutzer:** Abbildung 46 zeigt die Verteilung der Rekonstruktionsfehler für den Benutzer 1 und einem zufälligen Benutzer in diesem Fall Benutzer 10.
- **Bewertung der Performance:** Abbildung 42 zeigt die Bewertung des Modells, wobei hier in Farben dargestellt wird, welche Proben korrekt eingeordnet wurden).
- **Rekonstruktionsfehler für alle Nutzer:** Abbildung 43 zeigt die Verteilung der Rekonstruktionsfehler über alle Nutzer hinweg.

7) Zusammenfassung der Ergebnisse

Leider war auch der CNN-Autoencoder nicht in der Lage, zuverlässige Unterschiede zwischen den Daten zu erkennen. Die Ergebnisse waren ähnlich wie bei den vorherigen Modellen. In einem zusätzlichen Test, wie in Graph42 zu erkennen ist, wurde der Autoencoder mit verschiedenen Samples konfrontiert. Dabei konnte er nur wenige Ergebnisse richtig interpretieren. Insgesamt sind die Ergebnisse, wie schon zuvor, für eine praktische Anwendung dieser Technik nicht brauchbar. Es scheint, dass Autoencoder-basierte Ansätze für unsere spezifische Aufgabe der Fingerabdruckerkennung nicht geeignet sind und wir alternative Methoden in Betracht ziehen müssen.

G. Übersicht der CNN-Modelle

Als Nächstes gehen wir auf die Convolutional Neural Networks (CNNs) Modelle ein, wie sie bereits in Ansatz 1 entwickelt wurden. Diese wurden neu entwickelt und getestet, um die Benutzererkennung auf Basis von Fingerabdruckdaten

zu optimieren. Dieser Abschnitt bietet eine detaillierte Übersicht über die implementierten Modelle, einschließlich ihrer Architektur, Datenverarbeitung und Trainingsmethoden.

1) Zusammenfassung der Entwicklungsphasen und Modelle

Bei der Entwicklung der CNN-Modelle wurden mehrere Ansätze verfolgt, um die optimale Konfiguration für das Canvas-Fingerprinting zu finden:

- **Vielfältige Architekturen:** Es wurden mehrere Modelle (Modell 1 bis Modell 5) mit unterschiedlichen Tiefen entwickelt, von einfachen bis zu sehr tiefen Strukturen.
- **Variation der Hyperparameter:** Verschiedene Optimierungsmethoden, Batch-Normalisierung und unterschiedlich große Faltungsschichten wurden getestet.
- **Sample-Größen:** Die Modelle wurden mit verschiedenen Datenmengen (z.B. 100, 1.000, 5.000, 10.000 Samples) trainiert, um die Skalierbarkeit zu untersuchen.
- **Negative Ratios:** Der Anteil negativer Beispiele wurde variiert (z.B. 0.5, 1, 2).

Die systematische Evaluation umfasste:

- Protokollierung von Trainings- und Validierungsergebnissen
- Analyse von Overfitting, Stabilität und Klassifikationsleistung
- Untersuchung des Einflusses von Schichttiefe, Dropout-Rate und Batch-Größe auf die Modellgüte

Durch dieses iterative Vorgehen konnten die Modelle schrittweise verbessert und der Einfluss verschiedener Parameter auf die Klassifikations-Performance genau nachvollzogen werden.

2) CNNV7: Evaluierung und Optimierung

Nach der iterativen Entwicklung und kontinuierlichen Verbesserung mehrerer Modelle wurde schließlich die Architektur CNNV7 entwickelt. Diese umfasst, wie bereits in Ansatz 1, mehrere Modelle, die mit unterschiedlichen Hyperparametern trainiert wurden. Durch diesen Ansatz war es möglich, präzise zu analysieren, wie die Architektur, die Anzahl der Samples für das Training und das Verhältnis negativer Beispiele die Performance beeinflussen. Insgesamt wurden 33 Modelle erstellt, die wertvolle Einblicke in die Auswirkungen der verschiedenen Parameter lieferten.

Die Architekturen wurden, ähnlich wie in Ansatz 1 beschrieben, genutzt und weiterentwickelt.

- **Modell 1 – Einfaches CNN:** Eine Convolutional-Schicht (32 Filter, 3×3, ReLU), MaxPooling2D, Flatten und zwei Dense-Layer mit Dropout.
- **Modell 2 – CNN mit Batch Normalization:** Aufbau wie Modell 1, ergänzt durch BatchNormalization und eine zusätzliche Convolutional-Schicht (64 Filter).
- **Modell 3 – Tieferes CNN:** Drei Convolutional-Schichten (32, 64, 128 Filter) mit BatchNormalization und MaxPooling.
- **Modell 4 – Noch tieferes CNN:** Vier Convolutional-Schichten (64 bis 512 Filter) mit BatchNormalization und MaxPooling.
- **Modell 5 – Sehr tiefes CNN:** Fünf Convolutional-Schichten (32 bis 512 Filter) mit BatchNormalization,

MaxPooling und abschließendem Dense-Layer mit Dropout.

Für jede Architektur wurden Experimente mit unterschiedlichen Sample-Größen durchgeführt. Dabei wurden wichtige Hyperparameter wie Lernrate, Batchgröße und Anzahl der Epochen iterativ angepasst, um die besten Ergebnisse zu erzielen. Diese systematische Variation lieferte wertvolle Erkenntnisse darüber, welche Kombination von Architektur und Parametern die beste Performance ermöglicht.

3) Ergebnisse

Die Ergebnisse der unterschiedlichen Modelle zeigen, dass bestimmte Kombinationen von Parametern eine deutlich bessere Performance aufweisen als andere. Dies lässt sich damit begründen, dass Modelle, die beispielsweise mit extrem wenigen Stichproben (wie 100 Stichproben) trainiert wurden oder nur über eine geringe Anzahl an Epochen trainiert wurden, keine zufriedenstellenden Ergebnisse erzielen konnten. Der Grund dafür liegt in der unzureichenden Menge an Trainingsdaten, was das Erlernen individueller Nutzerprofile erheblich erschwert. Des Weiteren können durch Überanpassung bei zu langen Iterationen ebenfalls schlechte Ergebnisse erzielt werden. Daher ist es relevant, welche Kombinationen von Parametern wir wählen. Zusätzlich spielen auch die negativen Samples eine große Rolle, da eine zu geringe Anzahl dieser dem Modell möglicherweise nicht ausreicht, um gute Ergebnisse zu erzielen. In den folgenden grafischen Darstellungen werden verschiedene Metriken, darunter auch der F1-Score, analysiert, welcher in dieser Untersuchung als Hauptmetrik dient. Die Diagramme verdeutlichen die unterschiedlichen Ergebnisse und heben ein Problem hervor, das im weiteren Verlauf detaillierter erläutert wird.

4) Grafische Darstellung der Ergebnisse des CNN-Modells

Die Darstellung zeigt die Performance der Modelle anhand von drei verschiedenen Visualisierungen pro Modell. Im ersten Graphen⁴⁹ wird dargestellt, wie das Modell auf verschiedene Samples von unterschiedlichen Benutzern reagiert. Pro Benutzer werden fünf verschiedene Samples geladen und dem Modell zur Evaluierung vorgelegt. Die Linie in unterschiedlichen Farben repräsentiert dabei die Vorhersagesicherheit des Modells (0–100%). Jede Farbe entspricht einem Benutzer, sodass erkennbar ist, wie stabil das Modell auf die einzelnen Benutzer reagiert. Die zweite Abbildung⁵⁰ zeigt ein Kreisdiagramm, das veranschaulicht, welche Benutzer fälschlicherweise als Benutzer 1 erkannt wurden. Dies dient dazu, problematische Nutzer im gesamten Evaluationsprozess zu identifizieren. Die dritte Abbildung⁵¹ präsentiert eine Tabelle mit wichtigen Metriken wie Accuracy, Precision, Recall und F1-Score, um die Gesamtleistung des Modells zu bewerten. Abschließend wird für jedes Modell eine Konfusionsmatrix^{??} gezeigt, die die Verteilung der korrekten und falschen Vorhersagen darstellt. Wir beginnen nun mit der Analyse des am besten performenden Modells, ausgewählt aus den 33 Modellen, basierend auf dem F1-Score: Modell Architektur 1 mit 1000 Samples und 1000 negativen Samples.

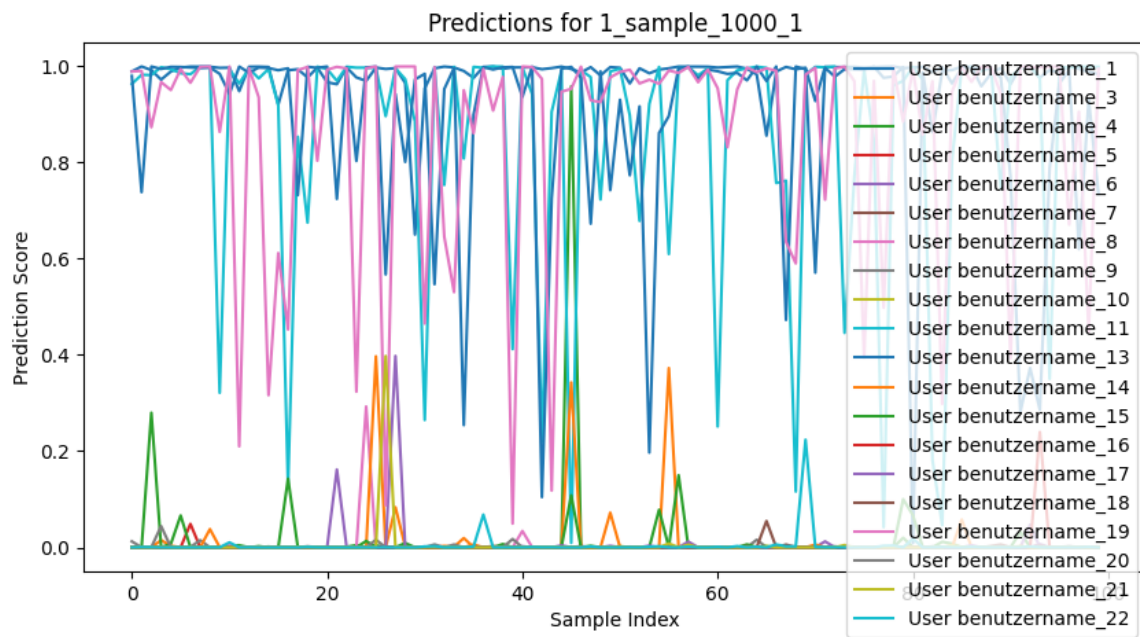


Abbildung 49: Bewertung verschiedener Nutzer

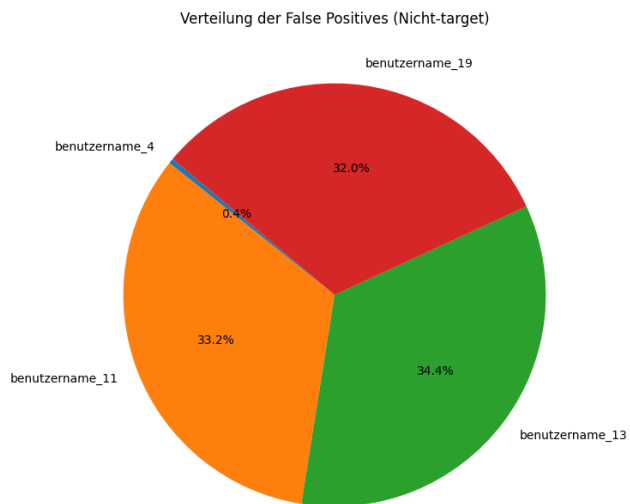


Abbildung 50: Kreisdiagramm False Positives

Metriken - 1_sample_1000_1

Model	1_sample_1000_1
Accuracy	0.87
Precision	0.28
Recall	0.99
F1 Score	0.44
Total Predictions	2000
Recognized as benutzername_1	99 von 100
Total False Positives	253

von 5000 Samples sowie einer Ratio von 0,5 für die negativen Beispiele trainiert.

Im weiteren Verlauf wird das beste Modell aller Architekturen vorgestellt ausgeschlossen Architektur 1. Dabei handelt es sich um ein Modell mit Architektur Typ 3, das eine komplexere Struktur aufweist. Dieses Modell wurde mit einem Trainingsvolumen von 5000 Samples und einer Ratio von 0,5 für die negativen Beispiele trainiert.

Abbildung 51: Metrische Tabelle

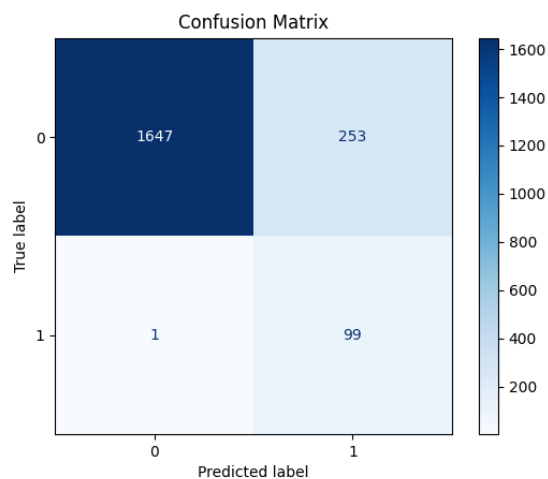


Abbildung 52: Konfusionsmatrix

Wie in den Abbildungen zu erkennen ist, scheint dieses Modell nicht sonderlich stabil zu sein. Dies lässt sich am Graphen 49 erkennen, da er viele Schwankungen aufweist, insbesondere bei anderen Nutzern. Des Weiteren zeigt die folgende Abbildung 50, dass drei Nutzer vom Modell falsch erkannt und als False Positives klassifiziert werden. Dies führt zu einem schlechten Precision-Score. Dennoch hat dieses Modell von allen Modellen, die wir erstellt haben, den besten F1-Score erreicht, wie in der Abbildung 51 zu sehen ist. Da das Modell jedoch sehr instabil wirkt und mit der Architektur Typ 1 die simpelste Architektur besitzt, ist dies nicht überraschend. Trotzdem konnte durch die Ergebnisse der beste F1-Score erzielt werden, wie auch in der Konfusionsmatrix 52 dargestellt wird. Im Folgenden wird nun das beste Modell aller Architekturen vorgestellt: Architektur 3. Dieses Modell besitzt eine komplexere Struktur und wurde mit einem Trainingsvolumen

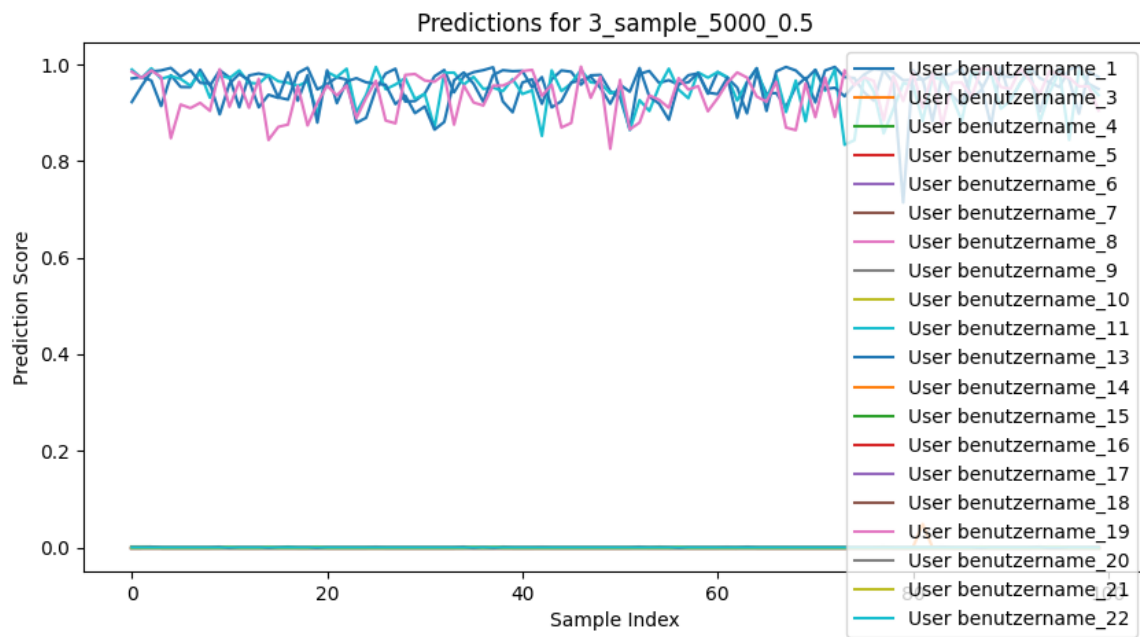


Abbildung 53: Bewertung verschiedener Nutzer

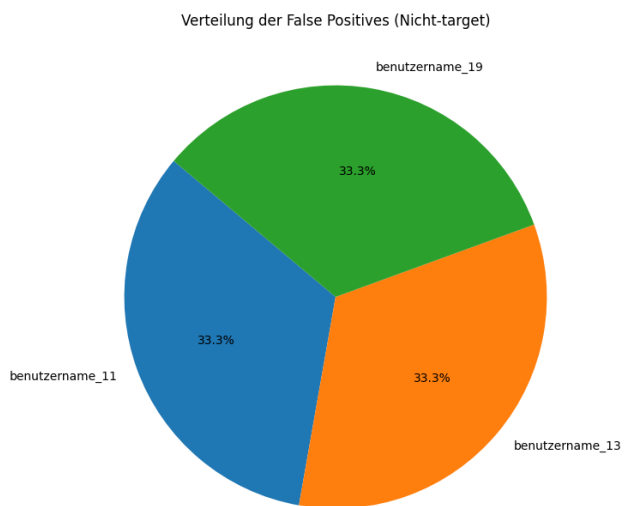


Abbildung 54: Kreisdiagramm False Positives

Metriken - 3_sample_5000_0.5

Model	3_sample_5000_0.5
Accuracy	0.85
Precision	0.25
Recall	1.0
F1 Score	0.4
Total Predictions	2000
Recognized as benutzername_1	100 von 100
Total False Positives	300

Abbildung 55: Metrische Tabelle

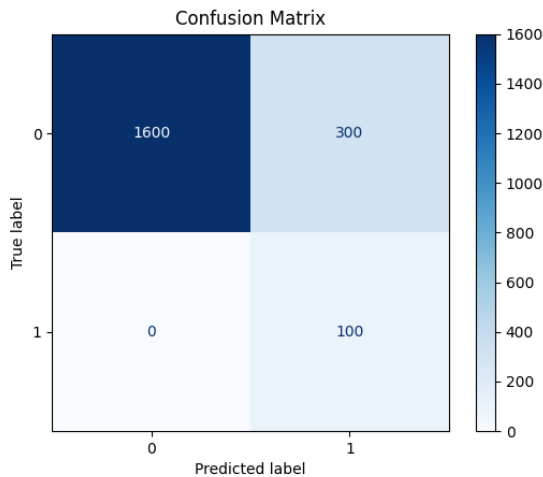


Abbildung 56: Konfusionsmatrix

In den Graphen kann man erkennen, dass das Modell der Architektur 3 deutlich stabiler bei der Bewertung der verschiedenen Samples ist, wie es anhand des Graphen 53 zu sehen ist. Dennoch ist hier zu erkennen, dass drei Nutzer, identisch wie bei dem ersten Modell, konstant als Nutzer 1 erkannt werden, was inkorrekt ist.

Dies spiegelt sich auch in unserem Kreisdiagramm 54 wider, welches dieselben drei Nutzer auflistet, die falsch klassifiziert wurden. Des Weiteren lässt sich anhand des F1-Scores und der Konfusionsmatrix erkennen, dass diese drei Nutzer die Metriken negativ beeinflusst haben. Obwohl das Modell recht deutlich und sehr performant sowie sicher Nutzer 1 erkennt, gelingt es ihm hingegen nicht, andere Nutzer korrekt zu identifizieren – abgesehen von den bereits genannten drei Nutzern. Dies zeigt, dass dieses Modell besser performt als die Modelle aus Ansatz 1, jedoch ein anderes Problem vorliegt.

Im Folgenden werden die globalen Metriken vorgestellt. Der nächste Graph (57) zeigt die erreichte Accuracy der verschiedenen Modelle. Die Linie im Graphen repräsentiert den Mittelwert, während die überlappenden Farben die höchsten und niedrigsten Ergebnisse aus den durchgeführten Tests darstellen. Die unterschiedlichen Farben stehen für die Größe der negativen Samples, die während des Trainings verwendet wurden. Aus den Modellnamen lässt sich sowohl der positive Anteil der Samples für das Training als auch die verwendete Architektur (z. B. Architektur 1) ablesen.

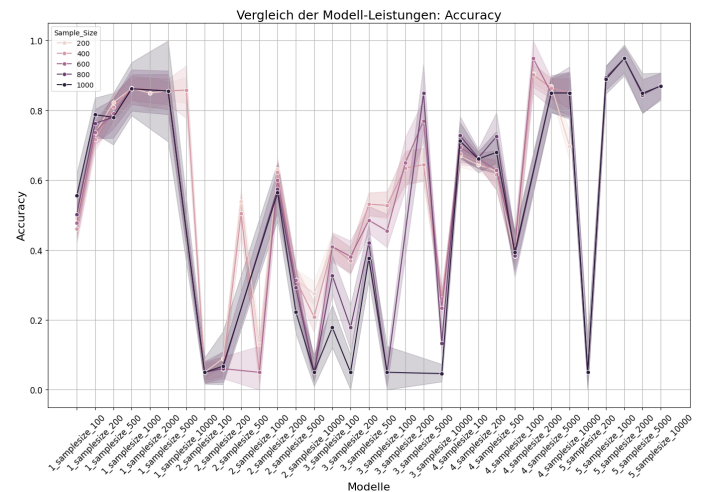


Abbildung 57: Gesamte Genauigkeit (Accuracy) des CNN-Modells

Wie man in dem Graphen erkennen kann, zeigt die Architektur 1 eine deutliche Leistungssteigerung, insbesondere in Relation zur Sample-Größe, was in diesem Fall zu erwarten war. Dass die Modelle 2, 3 und 4 etwas weniger zufriedenstellende Ergebnisse erzielen, liegt mit größter Wahrscheinlichkeit an dem bereits angesprochenen Problem mit den Fehlklassifizierungen der Benutzer 19, 11 und 13. Dieses Thema werde ich in den folgenden Kapiteln genauer erläutern.

Dennoch ist zu erkennen, dass insbesondere die Modelle mit der Architektur 5 die höchsten Werte erzielen, was ebenfalls zu erwarten war. Da die Modelle im ersten Ansatz ähnliche Ergebnisse erzielen konnten, bestätigt dies die Annahme, dass sowohl die Komplexität als auch die Sample-Größe für das Training relevant sind und einen erheblichen Einfluss auf die Performance des Modells haben.

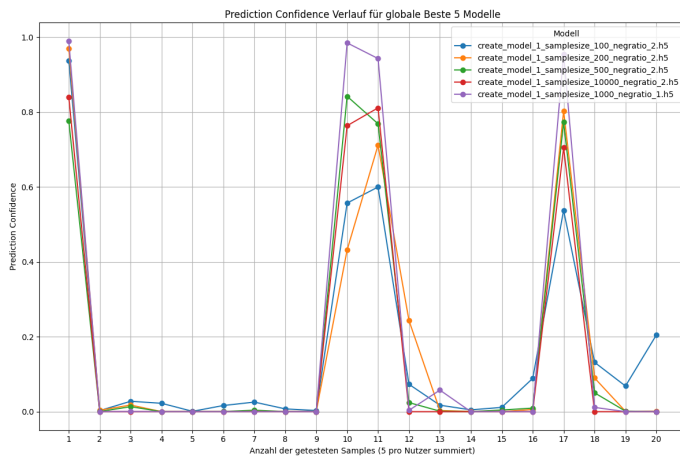


Abbildung 58: Genauigkeit der besten 5 Modelle

In diesem Graphen 58 wird ein weiterer Test dargestellt, der die fünf besten Modelle basierend auf dem F1-Score zeigt und diese erneut gegen einzelne Stichproben testet, um deren Performance deutlicher darzustellen. Zu den Daten: Von jedem der 20 Benutzer wurden fünf Samples geladen und gegen die Modelle getestet. Anschließend wurde der Mittelwert der Ergebnisse berechnet und als Datenpunkt in den Graphen eingetragen. Die X-Achse repräsentiert die Benutzer, während die Y-Achse die Zuverlässigkeit des Modells anzeigt, mit der es vorhersagen kann, dass ein bestimmtes Sample Benutzer 1 zugeordnet werden kann. Wie im Graphen zu erkennen ist, klassifizieren die fünf Modelle (alle aus der Architekturklasse 1 mit unterschiedlichen Sample-Größen und negativen Sample-Größen) Benutzer 1 erwartungsgemäß korrekt. Leider wurden jedoch erneut Benutzer 11, 13 und 19 fehl klassifiziert. Die Unterschiede zwischen den Benutzernamen im Graphen und deren Bezeichnung können dabei auf den ersten Blick verwirrend wirken. Zwar werden im Graphen Benutzer 1 bis 20 dargestellt, jedoch waren ursprünglich Benutzernamen bis 22 in der Datenbank vorhanden. Aufgrund fehlerhafter Einträge wurden zwei Benutzer aus der Datenbank entfernt wie bereits angesprochen. Somit stimmen die Bezeichnungen dennoch überein, da es sich um eine reduzierte Menge von 20 Benutzern handelt. Die Zuordnung der verbleibenden Benutzer bleibt konsistent und korreliert mit den in der Legende angegebenen Nummerierungen. Eine vollständige Auflistung der Geräte und deren Zuordnung kann in den vorherigen Kapiteln eingesehen werden. Der Graph bestätigt erneut, dass das Modell mit der Architekturklasse 1 und einer Sample-Größe von 1000 am besten abschneidet. Dies ist an der lila Kurve im Graphen deutlich zu erkennen.

In der Abbildung 59 ist ein ähnliches Test-Setup wie der vorherige Graph zu erkennen, jedoch wurden hier anstelle der fünf besten Modelle insgesamt, basierend auf dem F1-Score, die besten Modelle jeder Architektur dargestellt. Hier ist zu erkennen, dass die Modelle der Architektur 2 Probleme bei der Klassifizierung hatten, weswegen diese in unseren Ergebnissen die niedrigsten F1-Werte aufweisen. Es ist jedoch auch zu

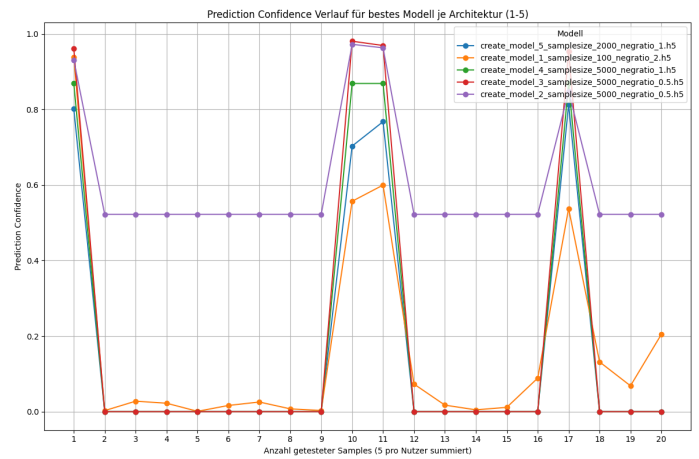


Abbildung 59: Genauigkeit der besten Modelle pro Architektur

sehen, dass die Modelle der Architekturen 1, 3, 4 und 5 eine ähnliche Performance bieten. Dies unterstreicht, dass die Modelle in der Lage sind, sicher zwischen den Benutzern zu unterscheiden. Besonders die Architekturen 3, 4 und 5 zeigen hierbei eine herausragende Leistung, wie anhand der Tests für Nutzer, die nicht Benutzer 1 sind, deutlich wird. Es ist klar zu erkennen, dass sich diese drei Architekturen sehr sicher sind, dass diese Samples nicht zu Benutzer 1 gehören – was korrekt ist. Nicht zu übersehen ist jedoch, dass die Modelle der Architektur 3, gefolgt von 4 und 5, deutlich besser und sicherer agieren als die Modelle der Architektur 1. Dies stützt die Vermutung, dass sowohl die Architektur als auch die Tiefe des Netzwerks ausschlaggebend für die Performance sind. Zudem ist anhand der Legende erkennbar, dass auch die Sample-Größe eine wichtige Rolle spielt. Alle Modelle außer denen der Architektur 1 haben sehr große Sample-Größen genutzt, um bessere Performance-Ergebnisse zu erzielen. Außerdem scheint eine negative Ratio von 0,5 ebenfalls einen Einfluss auf die Performance zu haben. Es zeigt sich, dass auch kleinere Mengen an negativen Samples effizient sein können. Das beste Modell in diesem Fall Architektur 3 welches 5000 Samples mit einer negativen Ratio von 0,5 nutzt, welches durch die rote Linie repräsentiert wird. Dennoch ist erneut nicht zu übersehen, dass wieder dieselben Nutzer (11, 13 und 19) fehl klassifiziert wurden, was die Performance und die Ergebnisse verschlechtert.

Kommen wir zu dem nächsten Graphen 60, welcher die Metriken Accuracy, Precision, Recall und F1 Score vergleicht. Wir nutzen den F1 Score als Referenz für die Performance, da er die Werte zusammenführt und als harmonisches Mittel zwischen den Metriken agiert. Wie hier zu erkennen ist, ist die Precision bei allen Modellen ähnlich schlecht, bis auf das Modell der Architektur 1 mit 100 Samples und einer doppelten negativen Ratio. Dies lässt sich folgendermaßen begründen: Da die Modelle so simpel sind, könnte es Zufall gewesen sein, dass genau dieses Modell die wenigsten falschen Predictions abgibt, dies werden wir in den folgenden Kapiteln weiter

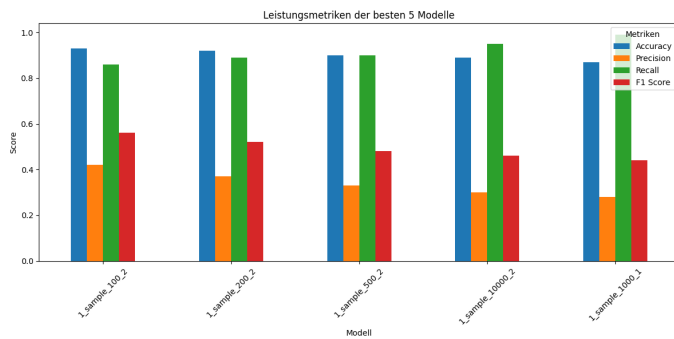


Abbildung 60: Performance 5 besten der Modelle

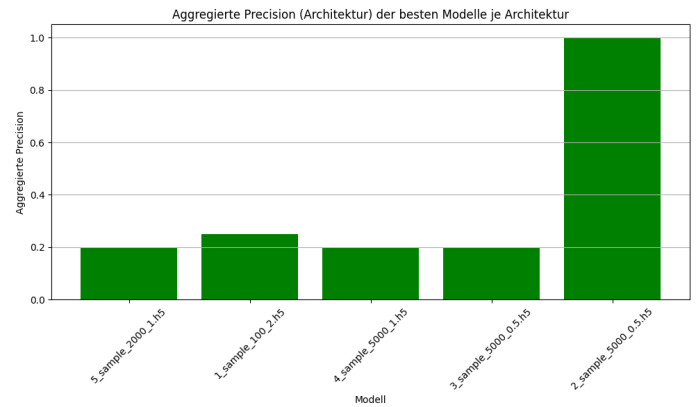


Abbildung 62: Zusammengeführte Liste des Precision Scores je Architektur

vertiefen.

In dem folgenden Graphen werden die fünf besten Modelle insgesamt illustriert.

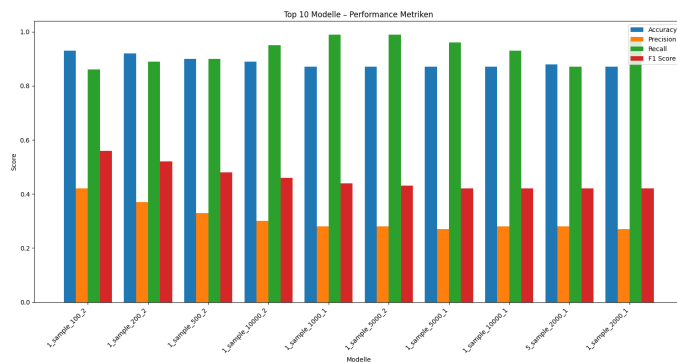


Abbildung 61: Performance der Top 10 Modelle

In diesem Graphen61 werden die Ergebnisse der besten 10 Modelle dargestellt. Hier ist zu erkennen, dass ein Modell der Architektur 5 zwischen den anderen Modellen liegt. Dies zeigt, dass die Modelle der Architektur 1 zwar gut darin sind, diese Art von Ergebnissen zu produzieren. Beobachtungen haben aber gezeigt, dass die Performance der Architekturen drei, vier und fünf bessere Ergebnisse erzielen könnten, wenn das Problem mit der Fehlklassifizierung nicht bestehen würde.

In den folgenden zwei Graphen6263 zeigen wir die Precision-Werte der besten fünf Modelle sowie der besten Modelle jeder Architektur, basierend auf dem F1-Score.

Wie hier zu erkennen ist, zeigen alle Modelle eine ähnliche Performance. Der Unterschied zwischen den beiden Graphen ist dadurch zu rechtfertigen, dass die Fehlfunktion des Top-Modells der Architektur 2 dafür sorgt, dass der Precision-Wert bei 100% liegt. Dies liegt daran, dass das Modell sämtliche Samples einfach als nicht zu Benutzer 1 gehörend einordnet und somit natürlich einen Precision-Wert von 100% erhält, da kein Sample fälschlicherweise als positiv behandelt wurde. Dies lässt den Graphen62 etwas unübersichtlicher wirken, dennoch kann dieser Wert für die aktuelle Analyse ausgeschlossen werden.

Wie bereits erwähnt, sind die Precision-Werte eine wieder Spiegelung dafür, wie viele Benutzer fälschlicherweise als

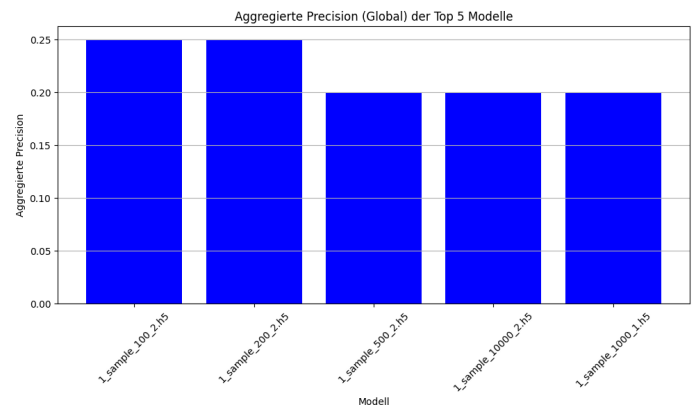


Abbildung 63: Zusammengeführte Liste des Precision Scores der 5 besten Modelle

Benutzer 1 klassifiziert wurden. Da jedoch jedes der trainierten Modelle drei Benutzer konstant falsch klassifiziert, werden die Performance-Metriken negativ beeinflusst und verfälscht.

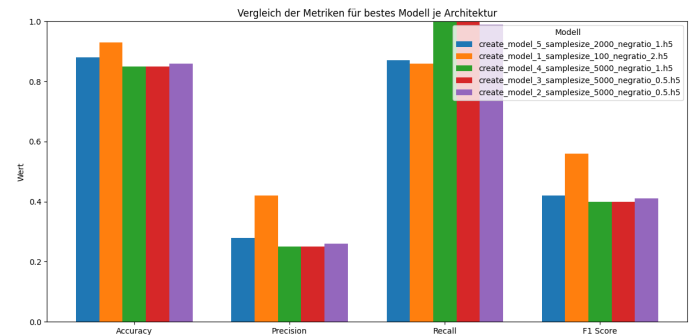


Abbildung 64: Metriken Vergleich der besten Modelle je Architektur

In den beiden folgenden Graphen6465 werden die Ergebnisse übersichtlich dargestellt, sowohl für die besten fünf Modelle (alle aus Architektur 1) als auch für die besten Modelle jeder Architektur. Der F1-Score, der als harmonisches Mittel dient

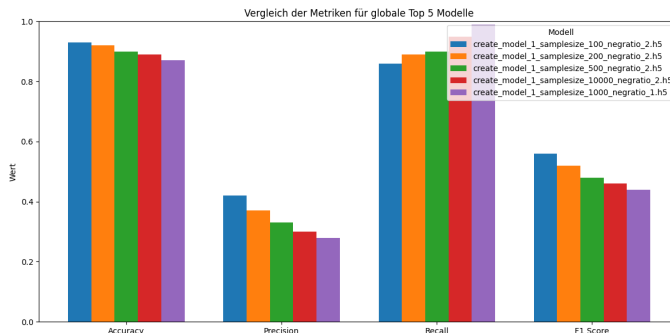


Abbildung 65: Metriken Vergleich der besten 5 Modelle

und eine zentrale Rolle bei der Bewertung der Modelle spielt, ist bei den meisten Modellen leider nicht besonders hoch. Wie bereits erwähnt, ist dies auf die Fehlklassifikationen durch die Modelle zurückzuführen. Diese drei betroffenen Nutzer sollten jedoch nicht die Gesamtpformance der Modelle und deren Ergebnisse dominieren. Sollte unsere Annahme korrekt sein, dass diese Probleme durch einen Fehler in der Datensammlung verursacht wurden, könnten die Modelle problemlos mit den Modellen des ersten Ansatzes konkurrieren, trotz der höheren Daten und Nutzeranzahl. Es lässt sich vermuten, dass, wenn die drei betroffenen Nutzer erneut korrekt in die Datenbank eingetragen würden und keine weiteren Probleme verursachen, die aktuellen Modelle auf Basis der vorliegenden Ergebnisse die Modelle des ersten Ansatzes übertreffen könnten. Wie dieses Problem entstanden sein könnte, klären wir in den folgenden Kapiteln.

5) Optimierungsversuche und finale Modellarchitektur

Nach den erfolgreichen Tests der verschiedenen CNN-Modelle wurde ein weiterer Optimierungsversuch mit Fokus auf Hyperparameter-Tuning durchgeführt. Dabei kam der Keras Tuner[65][66] zum Einsatz, der automatisch verschiedene Hyperparameter-Kombinationen testete und optimierte. Parameter wie Lernrate, Anzahl der Layer, Filtergröße und Dropout-Raten wurden systematisch variiert, um die optimale Konfiguration zu finden. Die negative Ratio, die sich bereits in den vorherigen Modellen als optimal erwiesen hatte, wurde für diese Optimierung beibehalten. Die folgende Architektur stellt das Ergebnis dieses Tuning-Prozesses dar und repräsentiert die finale, optimierte Version des Modells. Der Keras Tuner[65][66] hat diese Architektur als optimal identifiziert, nachdem er verschiedene Konfigurationen automatisch getestet und verglichen hat. Die folgende Architektur66 war das Ergebnis.

Die in der Abbildung66 dargestellte Architektur besteht aus zwei Convolutional-Blöcken, jeweils gefolgt von Max-Pooling, einer Flatten-Schicht und zwei Dense-Schichten. Zusätzlich wird eine Dropout-Schicht integriert, um Überanpassung zu vermeiden. Die Eingabegröße beträgt (224, 224, 3), was typisch für Bildklassifikationsaufgaben ist. Diese Architektur ähnelt insbesondere **Model 3** und **Model 4** aus dem vorherigen Ansatz 1. Wie bei **Model 3** werden hier zwei Convolutional-Schichten verwendet. Im Vergleich zu **Model**

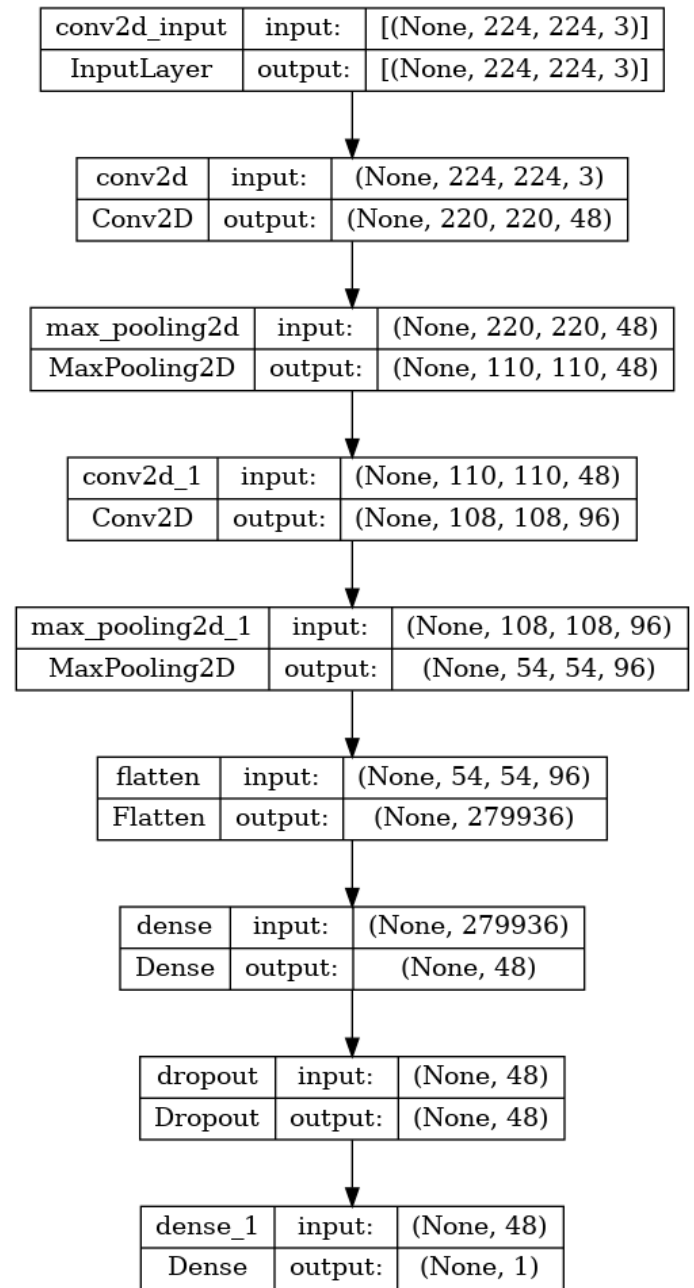


Abbildung 66: Architektur des optimierten CNN-Modells

4, das vier Convolutional-Schichten enthält, ist die Architektur jedoch weniger tief und damit weniger komplex. Die Dense-Schicht mit 48 Neuronen ist zudem kleiner als in **Model 4**, das mit 512 Neuronen arbeitet.

Zusammenfassend lässt sich sagen, dass diese neu entwickelte Architektur den bisherigen Architekturen ähnelt, was sich auch in der erzielten Performance widerspiegelt.

Metrik	Ursprüngliches Modell aus Ansatz 1	Optimiertes Modell aus Ansatz 2
Accuracy	0.9548	0.8356
Precision	0.9600	0.2202
Recall	0.9400	0.9867
F1-Score	0.9500	0.3601
ROC AUC	0.9700	0.9200

Tabelle IV: Vergleich der initialen Leistungsmetriken

Wie in der Abbildung IV zu erkennen ist, zeigt das Modell eine Präzision, die den Werten der zuvor getesteten Modelle sehr ähnlich ist. Dies könnte darauf zurückzuführen sein, dass die Architektur ebenfalls den bisherigen Modellen ähnelt und diese bereits nahe an der optimalen Architektur bzw. der besten Performance lagen. Dennoch liegt der Precision Wert des optimierten Modells leicht über dem der anderen bisher getesteten Modelle. Es bleibt jedoch denkbar, dass mit zusätzlicher Rechenzeit und mehr Ressourcen noch leistungsfähigere Architekturen entdeckt werden könnten. Vor einer weiteren Optimierung müsste jedoch zunächst die Problematik im Zusammenhang mit dem Datensatz gelöst werden, da dieser einen entscheidenden Einfluss auf das Training und die Optimierung der Modelle hat.

Zusätzlich wurden weitere Vergleiche mit früheren Modellversionen durchgeführt:

Modell	Accuracy	Precision	Recall	F1-Score
Model 1 (Alt)	0.9820	0.9776	0.9887	0.9831
Model 4 (Alt)	0.9900	1.0000	0.9811	0.9905
Optimiertes Modell (Neu)	0.8356	0.2202	0.9867	0.3601

Tabelle V: Vergleich mit früheren Modellversionen

Das Ergebnis des mit dem Keras Tuner optimierten CNN-Modells kann in der Konfusionsmatrix 67 eingesehen werden.

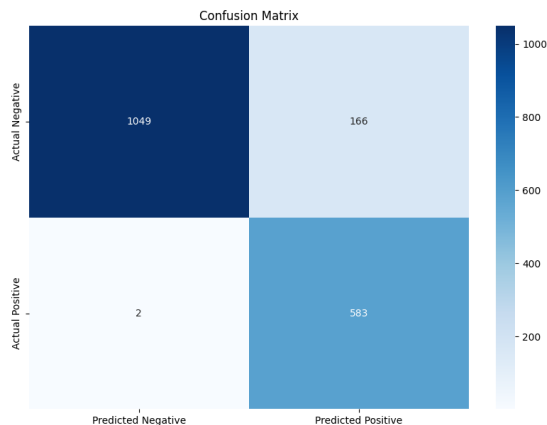


Abbildung 67: CNN Konfusionsmatrix

Interessanterweise zeigt sich, dass die zusätzlichen Optimierungsversuche nicht durchgehend zu einer Verbesserung der Gesamtleistung führten. Das ursprüngliche Modell erreichte in fast allen Metriken bessere Werte, mit Ausnahme des Recall-Wertes. Die älteren Modelle aus Ansatz 1 zeigen zwar höhere Werte bei Accuracy, Precision und F1-Score, was jedoch durch die kleinere Datenbasis begründet sein könnte und durch die Problematik mit dem Datensatz. Das neue optimierte Modell zeigt hingegen eine robuste Leistung bei einer größeren Datenbasis, was auf eine bessere Generalisierungsfähigkeit hinweist.

6) Diskussion der finalen Modellarchitektur

Die optimierte CNN-Architektur zeigt, dass diese Architektur sich am besten für die Klassifizierung für unsere Zwecke eignet und sich als einzige Methode erwiesen hat, die sinnvolle Ergebnisse geliefert hat. Dies wurde bereits im Ansatz 1 herausgefunden und nun bestätigt. Die Architektur der optimierten Modelle, die mit dem Keras Tuner[65] entwickelt wurden, ist ähnlich, wenn nicht sogar identisch mit den Modellen, die wir zuvor verwendet haben, insbesondere der Architektur 5 aus Ansatz 2. Die endgültige Struktur bzw. die Strukturen können sowohl komplex, wie in der Architektur 5, als auch simpel, wie in der Architektur 1, sein. Des Weiteren können die bereits etablierten Modelle durch weiteres Hyperparameter-Tuning weiter optimiert werden.

7) Zusammenfassung

Das Modell CNNV7 hat sich als eines der leistungsstärksten Modelle im Rahmen dieses Projekts erwiesen. Die Kombination aus tiefer Architektur, effektiven Trainingstechniken und optimierten Hyperparametern führte zu einer hohen Klassifikationsgenauigkeit. Die Modelle würden, wenn wir nicht die Fehlfunktion mit den drei verschiedenen Benutzern hätten, besser abschneiden als die Modelle aus Ansatz 1. Ergänzend wurden weitere Optimierungen durchgeführt, um die Leistungsfähigkeit bei größeren Datensätzen und unterschiedlichen Parametern zu evaluieren. Außerdem soll das Problem mit den Daten adressiert und weiter analysiert werden. Im Rahmen dieser Arbeit wurden verschiedene Machine-Learning-Modelle getestet, um die Benutzererkennung anhand von Fingerabdruckdaten zu optimieren. Die untersuchten Ansätze umfassten Isolation Forests, Autoencoder und Convolutional Neural Networks (CNNs). Dieses Kapitel fasst die Implementierung und Ergebnisse der Modelle noch einmal kurz zusammen.

8) Isolation Forest

Isolation Forest wurde als erster Ansatz implementiert, um Anomalien in Fingerabdruckdaten zu erkennen. Dieser Ansatz sollte zeigen, ob es möglich ist, auch mit simpleren Machine-Learning-Methoden genaue Klassifikationen zu erstellen. Wie die Ergebnisse jedoch zeigten, war dies leider kaum möglich. Selbst mit verschiedenen Architekturen und anderen Ansätzen hat sich diese Methode als kaum geeignet erwiesen.

9) Autoencoder

Autoencoder wurden in mehreren Varianten getestet:

- **Einfache Autoencoder:** Modelle mit Encoder-Decoder-Struktur zur Rekonstruktion von Eingabedaten.
- **CNN-Autoencoder:** Erweiterte Version mit Convolutional Layers zur besseren Verarbeitung von Bilddaten.

Die Autoencoder zeigten eine verbesserte Differenzierung zwischen Nutzern. Die beiden Modelle bzw. Methoden für das Autoencoder-Modell haben jedoch leider keine zufriedenstellenden oder gar stabilen Ergebnisse erzielen können. Daher haben wir uns nicht weiter mit diesen Methoden beschäftigt. Obwohl sie komplexer zu implementieren waren, wurden diese beiden Ansätze ebenfalls nicht weiterverfolgt, da keine zuverlässigen Ergebnisse geliefert wurden. Dies wurde durch die geringe Abweichung zwischen den verschiedenen Nutzern

dargestellt und in den Graphen 45 und 42 gezeigt. Dies führt uns zu dem bisherigen Favoriten, dem CNN-Netzwerk.

10) Convolutional Neural Networks (CNNs)

CNNs haben sich im Rahmen dieser Arbeit als die leistungsstärksten Modelle herausgestellt. Die Architektur umfasst insgesamt sechs Varianten: fünf Testmodelle, die den Modellen aus Ansatz 1 ähneln und auf bewährten Techniken basieren, sowie ein zusätzliches Modell, das mithilfe eines Tuners automatisch erstellt wurde. Alle Modelle lieferten gute Ergebnisse, stießen jedoch wiederholt auf das Problem, drei spezifische Nutzer nicht zuverlässig vom ursprünglichen Nutzer unterscheiden zu können. Dieses wiederkehrende Problem beeinflusste die Gesamtperformance der Modelle und wird im weiteren Verlauf detaillierter analysiert.

Im Folgenden möchten wir kurz auf die Problematik der Daten bzw. der Fehlklassifikationen eingehen und unsere These näher erläutern.

H. Klassifikationsproblematik

Bei der Analyse wurde festgestellt, dass drei Benutzer irrtümlich als Benutzer 1 identifiziert wurden. Der tatsächliche Benutzer 1, auf welchem alle Modelle trainiert wurden, verfügt über folgende Konfiguration: **ThinkPad E14 Gen 5 mit Windows 11 (Benutzer 1, Chrome-Browser)**.

Eine genauere Untersuchung ergab, dass alle diese Nutzer Gerätekonfigurationen aufweisen, die denen von Benutzer 1 ähneln. Im Folgenden sind diese Konfigurationen aufgeführt:

Tabelle VI: Geräte- und Benutzerinformationen

Gerät	Betriebssystem	Benutzer-ID	Browser
Home-PC	Windows 11	Benutzer 11	Chrome
Lenovo 21N10007GE	Windows 11	Benutzer 19	Chrome
iPhone 11	iOS 17.5.1	Benutzer 13	Chrome

Bemerkenswert ist, dass zwei der drei Geräte Windows 11 nutzen und auf allen Geräten der Chrome-Browser zum Einsatz kommt. Zudem fällt auf, dass die Geräte von Benutzer 11 und Benutzer 1 jeweils zwei Geräte aus meinem Privathaushalt umfassen, die bereits in Ansatz 1 verwendet wurden und sich damals klar differenzieren ließen. Dies wirft die Frage auf, ob zwischen dem Zeitraum von Ansatz 1 und Experiment 1 sowie Experiment 2 aus Ansatz 2 Änderungen an der Software vorgenommen wurden in Form von dem Betriebssystem oder dem Browser.

Die Beobachtungen deuten darauf hin, dass die Fingerabdrücke dieser Geräte sehr ähnliche Merkmale aufweisen, die vom aktuellen Modell nicht ausreichend differenziert werden können. Ein möglicher Grund könnte darin liegen, dass alle drei Systeme den Chrome-Browser in Verbindung mit vergleichbaren Office-Plugins oder Normalisierungseinstellungen nutzen, welche die Canvas-Rendering-Eigenschaften angleichen. Insbesondere in Unternehmensnetzwerken oder Office-Umgebungen können derartige Plugins standardmäßig installiert sein und zu homogenisierten Fingerabdrücken führen.

Dieses Problem könnte zukünftig durch den Einsatz größerer Trainingsdatensätze und genauere Analyse der Testgeräte

verbessert werden. Dabei sollten auch die spezifischen Browserkonfigurationen sowie der Einfluss der verwendeten Plugins genauer untersucht werden.

I. Diskussion der Ergebnisse

Die Ergebnisse zeigen, dass CNNs das größte Potenzial für die Benutzererkennung bieten. Auf Grundlage dieser Erkenntnisse konnten wir feststellen und belegen, dass Ansätze mit CNNs am besten für diese Art der Klassifizierung geeignet sind. Andere Modellansätze können daher zunächst ausgeschlossen werden. Dies stellt einen entscheidenden Vorteil dar, da der Fokus nun auf die vielversprechendsten Modelle gelegt werden kann, insbesondere im Hinblick auf eine mögliche Implementierung dieses Systems im Unternehmen KOBIL[58].

J. Zusammenfassung

Die Masterarbeit demonstriert erfolgreich, dass der ML Ansatz auch bei großen Datenmengen für die Benutzererkennung anhand von Fingerabdruckdaten effektiv eingesetzt werden kann. Die durchgeführten Untersuchungen mit Datensätzen von 100.000 Samples pro Nutzer über 20 Nutzer hinweg haben gezeigt, dass insbesondere CNN-basierte Modelle die besten Ergebnisse liefern. Dies bestätigt die ursprünglichen Annahmen aus Ansatz 1 und zeigt die besondere Eignung von CNNs für diese spezifische Aufgabe der Fingerabdruckererkennung.

Außerdem wurde unsere Fragetsellung gegenüber der besten Methode für das Training ebenfalls bestätigt, welches die CNN Ansätze waren.

Das Modell CNNV7 hat sich im Rahmen dieses Projekts als das leistungsstärkste Modell erwiesen, mit einer Klassifikationsgenauigkeit von 83,20% und einem F1-Score von 0,3601.

Ein wichtiger Aspekt, der während der Untersuchungen identifiziert wurde, ist die gelegentliche Fehlklassifikation einiger Benutzer als Nutzer 1. Dieses bereits diskutierte Problem könnte verschiedene Ursachen haben, wie beispielsweise ähnliche Browserumgebungen oder Hardware-Konfigurationen innerhalb der KOBIL Systemlandschaft. Trotz dieser Herausforderung zeigen die Ergebnisse, dass CNNV7 in der Lage ist, Unterschiede zwischen den meisten Nutzern zuverlässig zu erkennen.

VIII. DISKUSSION DER ERGEBNISSE

Die Diskussion der Ergebnisse dieser Masterarbeit zeigt eine klare Entwicklung in der Anwendung von Machine-Learning-Modellen zur Benutzererkennung. Die Untersuchung umfasste zwei Hauptansätze, die sich in der Datenmenge und der Komplexität der verwendeten Modelle unterschieden.

A. Ansatz I: Begrenzte Datenmenge

Der erste Ansatz basierte auf einer begrenzten Datenmenge von 1.900 Samples von 5 verschiedenen Geräten. Hier wurden sechs verschiedene CNN-Architekturen entwickelt und getestet, wobei die besten Modelle beeindruckende Ergebnisse erzielten. Model 6 erreichte beispielsweise eine Accuracy von 0,9980 und einen F1-Score von 0,9981. Diese Ergebnisse waren sehr vielversprechend und zeigten das Potenzial von CNNs für die Canvas-Fingerprint-Erkennung. Die Modelle konnten effektiv zwischen verschiedenen Benutzern unterscheiden.

B. Ansatz II: Erweiterte Datenbasis

Der zweite Ansatz erweiterte die Untersuchung auf eine größere Datenbasis von 100.000 Samples pro Nutzer, gesammelt von 20 verschiedenen Geräten. Die Daten wurden analysiert und erwiesen sich als ausreichend divers sowie qualitativ hochwertig für die durchgeführten Experimente. Es wurden verschiedene Modelltypen getestet, darunter Isolation Forest, Autoencoder und optimierte CNN-Modelle. Die optimierte CNN-Architekturen zeigten eine leichte Leistungsminderung im Vergleich zum ersten Ansatz, was auf die größere und diversere Datenmenge zurückzuführen sein könnte bzw die Problematik mit der Fehlklassifikation. Eine Hyperparameter-optimierte Version erreichte eine Accuracy von 89% und einen F1-Score von 36%.

C. Zentrale Herausforderungen

Die Arbeit zeigt mehrere zentrale Herausforderungen:

- Die Leistung der Modelle sank bei der Erweiterung des Datensatzes, was auf Optimierungsbedarf bei wachsenden Datenmengen sowie auf die Notwendigkeit einer besseren Kontrolle der Daten in zukünftigen Ansätzen oder Implementierungen hinweist.
- Tiefere Netze erfordern erhebliche Rechenressourcen, was eine Balance zwischen Modelltiefe und Effizienz notwendig macht.
- Browser-Updates und Hardware-Änderungen können die Fingerprint-Generierung beeinflussen und zu Erkennungsproblemen führen.

Insgesamt zeigt die Arbeit das große Potenzial von Canvas-Fingerprinting für die Benutzererkennung, weist aber auch auf die Notwendigkeit weiterer Forschung und Optimierung hin, um die Technik für den praktischen Einsatz zu verbessern.

IX. FAZIT UND AUSBLICK

Diese Masterarbeit untersuchte die Anwendung von Canvas Fingerprinting zur Benutzererkennung im Rahmen des Projekts IDentify. Dabei wurden zwei Hauptansätze verfolgt: ein initialer Ansatz mit begrenzten Daten und ein erweiterter Ansatz mit einer deutlich größeren Datenbasis.

Die Ergebnisse beider Ansätze haben gezeigt, dass Canvas Fingerprinting in der beschriebenen Form grundsätzlich funktioniert. Ansatz 1 verdeutlichte das Potenzial dieser Technik bereits auf Basis einer kleinen Datenmenge. Ansatz 2 testete verschiedene Machine-Learning-Ansätze, um die effektivste Methode für diese Technik zu identifizieren. Dabei wurde festgestellt, dass die CNN-Architektur (Convolutional Neural Network) die besten Ergebnisse liefert.

Im Verlauf der Arbeit traten jedoch auch Herausforderungen auf, insbesondere im Zusammenhang mit den Daten. Diese führten in Ansatz 2 zu Fehlklassifikationen, deren Ursachen weiter untersucht werden müssen. Zudem zeigte sich, dass der Rechenaufwand bei komplexeren Modellen steigt. Dennoch konnte nachgewiesen werden, dass auch kleinere Modelle in der Lage sind, Klassifikationen effektiv durchzuführen.

Trotz dieser Herausforderungen verdeutlicht die Arbeit das große Potenzial von Canvas Fingerprinting für die Benutzererkennung. Im Vergleich zu klassischen Authentifizierungsmethoden bietet dieser Ansatz mehrere Vorteile: erhöhte Sicherheit durch hardwarebasierte Merkmale, eine verbesserte Nutzererfahrung durch passive Authentifizierung und höhere Erkennungsraten, die als zusätzliche Sicherheitsebene genutzt werden können.

Zusammenfassend lässt sich sagen, dass die zentrale These bzw. Forschungsfrage dieser Arbeit erfolgreich beantwortet wurde. Das effektivste Modell bzw. die geeignetste Architektur für das zugrunde liegende Prinzip konnte identifiziert und erfolgreich getestet werden. Zudem zeigten Tests mit größeren Datenmengen sowohl neue Herausforderungen als auch Bestätigungen. Insbesondere wurde bestätigt, dass die Technik auch mit größeren Datenmengen funktioniert, was jedoch eine verstärkte Kontrolle der Daten erforderlich macht. Die Ergebnisse leisten einen relevanten Beitrag zur Erforschung und Weiterentwicklung von Canvas Fingerprinting als Methode zur Benutzererkennung. Sie zeigen sowohl das Potenzial als auch die Herausforderungen dieser Technologie auf und bieten eine solide Grundlage für zukünftige Forschungen in diesem Bereich. Gleichzeitig unterstreichen sie die Notwendigkeit weiterer Optimierungen und Untersuchungen, um Canvas Fingerprinting als zuverlässige und sichere Methode in realen Anwendungen einsetzen zu können.

A. Ausblick

Auf Basis dieser Forschung könnte das entwickelte System potenziell innerhalb von KOBIL[58] genutzt und als zusätzliche Sicherheitsebene oder Protokoll eingeführt werden. Vor einer produktiven Implementierung sind jedoch noch folgende Schritte erforderlich:

- Das System sowie das Frontend und Backend müssen erweitert werden, sodass bei jedem Besuch eines Nutzers

erneut Fingerabdrücke erstellt und an den Server übermittelt werden. Dadurch könnten Änderungen im Browser oder Betriebssystem direkt erkannt und die Modelle entsprechend neu trainiert werden.

- Es sollte eine Abwägung zwischen Leistung und Genauigkeit vorgenommen werden, insbesondere im Hinblick auf die Performance der Modelle. Die optimale Architektur muss dabei ausgewählt werden.
- Die Problematik der Fehlklassifikationen verschiedener Nutzer sollte genauer untersucht und behoben werden.

Sobald diese Schritte abgeschlossen sind, könnte das System implementiert und effektiv weiterentwickelt und genutzt werden.

X. DANKSAGUNG

Ich möchte mich herzlich bei Herrn Dr. Hossein Federroni von der Firma KOBIL für seine Themengabe während dieser Arbeit bedanken. Sein Fachwissen und seine Anregungen haben zum Erfolg dieses Projekts beigetragen.

Mein besonderer Dank gilt zudem Prof. Dr. Jens Köhler und Frau Prof. Dr. Elisabeth Heinemann, deren Begleitung und konstruktives Feedback eine wichtige Grundlage für die Qualität dieser Arbeit waren.

A. Zusätzliche Informationen

Sämtliche Bilder und Daten, die nicht in der Arbeit gezeigt wurden, sowie zusätzliche Informationen zu den verwendeten Modellen, sind im folgenden Google-Drive-Ordner gesammelt: [https://drive.google.com/file/d/1M41B06hJLM9w7olZCfxm3QQ62XagVZT2/view?usp=sharing\[67\]](https://drive.google.com/file/d/1M41B06hJLM9w7olZCfxm3QQ62XagVZT2/view?usp=sharing[67]). Falls Bilder in der Arbeit nicht ausreichend groß oder erkennbar sind, können diese ebenfalls dort eingesehen werden.

LITERATUR

- [1] Google Developers. „Klassifizierung: Genauigkeit, Trefferquote, Genauigkeit und zugehörige Messwerte.“ Zugriff am 13. Januar 2025. (Jan. 2025), Adresse: <https://developers.google.com/machine-learning/crash-course/classification/accuracy-precision-recall?hl=de>.
- [2] G. Developers. „Klassifizierungsgrenzwerte und Wahrheitsmatrizen“, Google. (2024), Adresse: <https://developers.google.com/machine-learning/crash-course/classification/thresholding> (besucht am 27.01.2024).
- [3] SoSafe. „Brute-Force-Angriff: Beispiele, Anzeichen & Schutz“, SoSafe GmbH. (2024), Adresse: <https://sosafe-awareness.com/de/glossar/brute-force-angriff/> (besucht am 27.01.2024).
- [4] K. Thomas, J. Pullman, K. Yeo u. a., „Data breaches, phishing, or malware? Understanding the risks of stolen credentials“, Englisch, 2017. Adresse: <https://research.google/pubs/data-breaches-phishing-or-malware-understanding-the-risks-of-stolen-credentials/> (besucht am 27.01.2024).

- [5] SGNL. „Why token theft is the next big threat.“ Englisch, SGNL. (Okt. 2024), Adresse: <https://sgnl.ai/2024/10/why-token-theft-is-the-next-big-threat> (besucht am 27.01.2024).
- [6] W. contributors. „Session hijacking.“ Zuletzt abgerufen am 10. Februar 2025. (2025), Adresse: https://en.wikipedia.org/wiki/Session_hijacking.
- [7] S. M. Bellovin, „Problem Areas for the IP Security Protocols,“ *Proceedings of the 6th ACM Conference on Computer and Communications Security (CCS '99)*, S. 1–6, 1999, Zuletzt abgerufen am 10. Februar 2025. DOI: 10.1145/322796.322806. Adresse: <https://dl.acm.org/doi/10.1145/322796.322806>.
- [8] LastPass. „Psychologie der Passwörter: Riskante Verhaltensmuster im Internet,“ LastPass. (2020), Adresse: <https://www.lastpass.com/de/archives/psychology-of-passwords-2020> (besucht am 27.01.2024).
- [9] U. Diaz-Orueta, E. Omeragic, J. Ruiz-Vanoye, A. Diaz-Perez und V. Fernandez-Torres, „User perceptions and use of authentication methods: insights from youth in Mexico and Bosnia and Herzegovina,“ Englisch, *ResearchGate*, 2022. Adresse: https://www.researchgate.net/publication/359327471_User_perceptions_and_use_of_authentication_methods_insights_from_youth_in_Mexico_and_Bosnia_and_Herzegovina (besucht am 27.01.2024).
- [10] L. Witte, *IDentify – GitHub Repository*, <https://github.com/lionboomer/IDentify>, Zugriff am 01.06.2025, 2025.
- [11] t3n, *Canvas-Fingerprinting: Was ist das eigentlich?* <https://t3n.de/news/canvas-fingerprinting-1132510/>, Zugriff am: 15.07.2024, 2024.
- [12] Wikipedia-Autoren. „Fingerprinting (Trackingtechnik).“ Zuletzt abgerufen am 10. Februar 2025. (2025), Adresse: [https://de.wikipedia.org/wiki/Fingerprinting_\(Trackingtechnik\)](https://de.wikipedia.org/wiki/Fingerprinting_(Trackingtechnik)) (besucht am 10.02.2025).
- [13] E. Systems, *Browser-Fingerprinting: Chancen und Risiken des ... - Esko Systems*, <https://esko-systems.de/browser-fingerprinting/>, Zugriff am: 15.07.2024, 2024.
- [14] Fingerprint, *FingerprintJS*, Version 4, 2025. Adresse: <https://github.com/fingerprintjs/fingerprintjs> (besucht am 28.01.2025).
- [15] AdsPower. „Cross Browser Fingerprinting: User Tracking On Steroids.“ Englisch, AdsPower. (2024), Adresse: <https://www.adspower.com/blog/cross-browser-fingerprinting-user-tracking-on-steroids> (besucht am 27.01.2024).
- [16] D. Datenschutz. „Zustimmung bei Device Fingerprinting erforderlich,“ Dr. Datenschutz. (2024), Adresse: <https://www.dr-datenschutz.de/zustimmung-bei-device-fingerprinting-erforderlich> (besucht am 27.01.2024).
- [17] A. Lerner, A. K. Simpson, T. Kohno und F. Roesner, „Internet Jones and the Raiders of the Lost Trackers: An Archaeological Study of Web Tracking from 1996 to 2016,“ Englisch, in *Proceedings of the 25th USENIX Security Symposium*, 2016. Adresse: <https://dl.acm.org/doi/10.5555/3241094.3241172> (besucht am 27.01.2024).
- [18] W3Schools. „HTML5 Canvas Tutorial.“ Zugriff am 5. Januar 2025. (2025), Adresse: https://www.w3schools.com/html/html5_canvas.asp.
- [19] K. Mowery und H. Shacham, „Pixel Perfect: Fingerprinting Canvas in HTML5,“ in *Proceedings of the Web 2.0 Security and Privacy Workshop (W2SP)*, IEEE Computer Society, 2012. Adresse: <https://hovav.net/ucsd/dist/canvas.pdf>.
- [20] Fingerprint. „Canvas fingerprinting: What it is and how it works.“ Englisch, Fingerprint. (2024), Adresse: <https://fingerprint.com/blog/canvas-fingerprinting> (besucht am 27.01.2024).
- [21] Wikipedia, *Canvas fingerprinting*, https://en.wikipedia.org/wiki/Canvas_fingerprinting, Zugriff am: 15.07.2024, 2024.
- [22] BrowserLeaks. „Canvas Fingerprinting.“ Zuletzt abgerufen am 10. Februar 2025. (2025), Adresse: <https://browserleaks.com/canvas> (besucht am 10.02.2025).
- [23] R. Wiki, *Canvas Fingerprinting einfach erklärt*, https://de.ryte.com/wiki/Canvas_Fingerprinting/, Zugriff am: 15.07.2024, 2024.
- [24] Fingerprint, *Canvas fingerprinting: What it is and how it works*, <https://fingerprint.com/blog/canvas-fingerprinting/>, Zugriff am: 15.07.2024, 2024.
- [25] H. Fereidooni. „LinkedIn-Profil von Hossein Fereidooni.“ Zugriff am 3. Januar 2025. (2025), Adresse: https://www.linkedin.com/in/hossein-fereidooni-7839bb6b/?locale=de_DE.
- [26] H. Fereidooni. „Google Scholar-Profil von Hossein Fereidooni.“ Zugriff am 3. Januar 2025. (2025), Adresse: https://scholar.google.com/citations?view_op=list_works&hl=en&user=nwjsHDYAAAAJ.
- [27] Wikipedia contributors. „Isolation Forest.“ Zugriff am 13. Januar 2025. (Jan. 2025), Adresse: https://en.wikipedia.org/wiki/Isolation_forest.
- [28] Wikipedia-Autoren. „Autoencoder.“ Zuletzt abgerufen am 10. Februar 2025. (2025), Adresse: <https://de.wikipedia.org/wiki/Autoencoder> (besucht am 10.02.2025).
- [29] D. Bergmann und C. Stryker. „Autoencoder: Eine Einführung in die Architektur für neuronale Netze.“ Zuletzt abgerufen am 10. Februar 2025. (2023), Adresse: <https://www.ibm.com/de-de/topics/autoencoder> (besucht am 10.02.2025).
- [30] Wikipedia contributors. „Convolutional Neural Network,“ Wikipedia, Die freie Enzyklopädie. (2025), Adresse: https://de.wikipedia.org/wiki/Convolutional_Neural_Network (besucht am 01.02.2025).
- [31] Wikipedia-Autoren. „Siamese Neural Network.“ Zuletzt abgerufen am 10. Februar 2025. (2025), Adresse: https://en.wikipedia.org/wiki/Siamese_neural_network (besucht am 10.02.2025).
- [32] TechTarget. „What is data splitting and why is it important?“ Englisch, TechTarget. (2024), Adresse: <https://www.techtarget.com/whatis/definition/data-splitting>

www.techtarget.com/searchenterpriseai/definition/data-splitting (besucht am 27.01.2024).

- [33] WeLiveSecurity, *Überall Fingerabdrücke: Kann Browser-Fingerprinting die Sicherheit von Websites erhöhen?* <https://www.welivesecurity.com/de/privatsphäre/ueberall-fingerabdrucke-kann-browser-fingerprinting-die-sicherheit-von-websites-erhoehen/>, Zugriff am: 15.07.2024, 2024.
- [34] Experte.de, *Browser-Fingerprinting - Was ist es und wie kann ich mich schützen?* <https://www.experte.de/it-sicherheit/browser-fingerprint>, Zugriff am: 15.07.2024, 2024.
- [35] P. Laperdrix, N. Bielova, B. Baudry und G. Avoine, „Browser Fingerprinting: A Survey,“ *ACM Computing Surveys*, Jg. 53, Nr. 1, S. 1–36, 2020, Also available as arXiv preprint: <https://arxiv.org/abs/1905.01051>. DOI: 10.1145/3386040. Adresse: <https://dl.acm.org/doi/10.1145/3386040>.
- [36] P. Raschke und A. Küpper, „Uncovering Canvas Fingerprinting in Real-Time and Analyzing Its Usage for Web Tracking,“ in *Workshops der INFORMATIK 2018 - Architekturen, Prozesse, Sicherheit und Nachhaltigkeit*, Auch verfügbar unter: <https://dl.gi.de/server/api/core/bitstreams/19378bcf-d522-46d1-b59c-a31f35560742/content>, Bonn: Köllen Druck+Verlag GmbH, 2018, S. 97–108. Adresse: https://www.researchgate.net/publication/327263882_Uncovering_Canvas_Fingerprinting_in_Real-Time_and_Analyzing_Its_Usage_for_Web_Tracking (besucht am 27.01.2024).
- [37] F. Rochet, K. Efthymiadis, F. Koeune und O. Pereira, „SWAT: Seamless Web Authentication Technology,“ in *Proceedings of the World Wide Web Conference, WWW 2019*, ACM, 2019, S. 1579–1589, ISBN: 9781450366748. DOI: 10.1145/3308558.3313637. Adresse: <https://researchportal.vub.be/en/publications/swat-seamless-web-authentication-technology>.
- [38] G. Acar, M. Juarez, N. Nikiforakis u. a., „FPDetective: Dusting the Web for Fingerprinters,“ S. 1129–1140, 2013. DOI: 10.1145/2508859.2516674. Adresse: <https://dl.acm.org/doi/10.1145/2508859.2516674>.
- [39] F. Alaca, A. Abdou und P. C. van Oorschot, „Scalable and Secure HTML5 Canvas-Based User Authentication,“ in *Information Security*, Springer International Publishing, 2022, S. 527–547, ISBN: 978-3-031-16815-4. DOI: 10.1007/978-3-031-16815-4_30. Adresse: https://link.springer.com/chapter/10.1007/978-3-031-16815-4_30.
- [40] FingerprintJS. „FingerprintJS: The Most Advanced Browser Fingerprinting Library.“ Accessed: 14 February 2025. (2023), Adresse: <https://github.com/fingerprintjs/fingerprintjs>.
- [41] D. Zhang, J. Zhang, Y. Bu, B. Chen, C. Sun und T. Wang, „A Survey of Browser Fingerprint Research and Application,“ *Wireless Communications and Mobile Computing*, Jg. 2022, S. 1–14, 2022. DOI: 10.1155/2022/3363335. Adresse: <https://doi.org/10.1155/2022/3363335>.
- [42] Bundesamt für Sicherheit in der Informationstechnik. „JavaScript, Cookies und Fingerprints.“ (2025), Adresse: <https://www.bsi.bund.de/DE/Themen/Verbraucherinnen-und-Verbraucher/Informationen-und-Empfehlungen/Cyber-Sicherheitsempfehlungen/Updates-Browser-Open-Source-Software/Der-Browser/JavaScript-Cookies-Fingerprints/javascript-cookies-fingerprints.html> (besucht am 28.01.2025).
- [43] European Data Protection Board. „Europäischer Datenschutzausschuss.“ (2025), Adresse: https://www.edpb.europa.eu/edpb_de (besucht am 28.01.2025).
- [44] The Tor Project. „Download Tor.“ (2025), Adresse: <https://www.torproject.org/download/> (besucht am 28.01.2025).
- [45] Brave Software Inc. „Brave Browser.“ (2025), Adresse: <https://brave.com/de/> (besucht am 28.01.2025).
- [46] A. A. Salomatin, A. Y. Iskhakov und R. V. Meshcheryakov, „Comparison of the Effectiveness of Countermeasures Against Tracking User Browser Fingerprints,“ *IFAC-PapersOnLine*, Jg. 55, Nr. 9, S. 244–249, 2022. DOI: 10.1016/j.ifacol.2022.07.043. Adresse: <https://www.sciencedirect.com/science/article/pii/S2405896322004281>.
- [47] Mozilla Developer Network. „Web APIs.“ (2024), Adresse: https://developer.mozilla.org/de/docs/Learn_web_development/Extensions/Client-side_APIs/Introduction (besucht am 28.01.2025).
- [48] „uBlock Origin.“ (2025), Adresse: <https://ublockorigin.com/de> (besucht am 28.01.2025).
- [49] Electronic Frontier Foundation. „Privacy Badger.“ (2025), Adresse: <https://privacybadger.org/> (besucht am 28.01.2025).
- [50] kkapsner. „CanvasBlocker.“ (2025), Adresse: <https://github.com/kkapsner/CanvasBlocker> (besucht am 28.01.2025).
- [51] Bundesamt für Sicherheit in der Informationstechnik (BSI). „BSI - Bundesamt für Sicherheit in der Informationstechnik.“ Zugriff am 14. Februar 2025. (2025), Adresse: https://www.bsi.bund.de/DE/Home/home_node.html.
- [52] Wikipedia. „Überanpassung.“ (2025), Adresse: <https://de.wikipedia.org/wiki/%C3%9Cberanpassung> (besucht am 28.01.2025).
- [53] Express.js. „Express - Node.js web application framework.“ (2025), Adresse: <https://expressjs.com/> (besucht am 29.01.2025).
- [54] Node.js. „Node.js: JavaScript runtime built on Chrome's V8 JavaScript engine.“ (2025), Adresse: <https://nodejs.org/en> (besucht am 29.01.2025).
- [55] Docker. „Docker Swarm.“ (2025), Adresse: <https://docs.docker.com/engine/swarm/> (besucht am 29.01.2025).
- [56] Kubernetes. „Kubernetes.“ (2025), Adresse: <https://kubernetes.io/> (besucht am 29.01.2025).

- [57] MongoDB. „MongoDB Atlas: Die beliebteste Dokumentendatenbank der Welt.“ (2025), Adresse: <https://www.mongodb.com/de-de> (besucht am 29.01.2025).
- [58] K. S. GmbH. „KOBIL - Secure Digital Identity and Access Management.“ Zuletzt abgerufen am 13. Februar 2025. (2025), Adresse: <https://www.kobil.com/en/index.html> (besucht am 13.02.2025).
- [59] S. Varshney. „VGG-Net Architecture Explained.“ (2025), Adresse: <https://medium.com/@siddheshb008/vgg-net-architecture-explained-71179310050f> (besucht am 30.01.2025).
- [60] Wikipedia-Autoren. „Residual Neural Network.“ Zuletzt abgerufen am 11. Februar 2025. (2025), Adresse: https://en.wikipedia.org/wiki/Residual_neural_network (besucht am 11.02.2025).
- [61] Viso Suite. „VGG: Very Deep Convolutional Networks.“ Zuletzt abgerufen am 11. Februar 2025. (2025), Adresse: <https://viso.ai/deep-learning/vgg-very-deep-convolutional-networks/> (besucht am 11.02.2025).
- [62] GeeksforGeeks. „VGG-16 CNN Model.“ Zuletzt abgerufen am 11. Februar 2025. (2025), Adresse: <https://www.geeksforgeeks.org/vgg-16-cnn-model/> (besucht am 11.02.2025).
- [63] Wikipedia. „Strukturelle Ähnlichkeit.“ Zuletzt abgerufen am 13. Februar 2025. (2025), Adresse: https://de.wikipedia.org/wiki/Strukturelle_Ähnlichkeit (besucht am 13.02.2025).
- [64] O. Olu-Ipinlaye. „Convolutional Autoencoders.“ DigitalOcean Community Tutorial, n.d. (), Adresse: <https://www.digitalocean.com/community/tutorials/convolutional-autoencoder> (besucht am 14.02.2025).
- [65] TensorFlow. „Introduction to the Keras Tuner.“ Abgerufen am 12. Februar 2025. (2024), Adresse: https://www.tensorflow.org/tutorials/keras/keras_tuner.
- [66] P. Kashyap. „Mastering Hyperparameter Tuning for Neural Networks with Keras Tuner.“ Abgerufen am 12. Februar 2025. (n.d.), Adresse: <https://medium.com/@piyushkashyap045/mastering-hyperparameter-tuning-for-neural-networks-with-keras-tuner-c0b64d549188>.
- [67] Unbekannt, *Zusätzliche Daten und Bilder zu der Arbeit*, Zugriff am 14. Februar 2025, 2025. Adresse: <https://drive.google.com/file/d/1M41B06hJLM9w7olZCfxm3QQ62XagVZT2/view?usp=sharing>.